

	This method can be overridden to achieve a custom naming scheme for generic BaseModels.
<code>model_post_init</code>	Override this method to perform additional initialization after <i>init</i> and <i>model_construct</i> . This is useful if you want to do some validation that requires the entire model to be initialized.
<code>model_rebuild</code>	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
<code>model_validate</code>	Validate a pydantic model instance.
<code>model_validate_json</code>	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
<code>model_validate_strings</code>	Validate the given object with string data against the Pydantic model.
<code>parse_file</code>	
<code>parse_obj</code>	
<code>parse_raw</code>	
<code>schema</code>	
<code>schema_json</code>	
<code>to_dict</code>	Return the error payload as a plain mapping.
<code>to_json</code>	Return the error payload serialized to JSON.
<code>update_forward_refs</code>	
<code>validate</code>	

`__init__`

Create a new model by parsing and validating input data from keyword arguments.

Raises `[ValidationError][pydantic_core.ValidationError]` if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

```
__init__(**data: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
data Required*	Any

Returns

[Expand table](#)

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
_fields_set	set[str] None Default value: None
values Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude:
AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None
= None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
include Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>update</code>	Default value: None
<code>deep</code>	Default value: False

Returns

[Expand table](#)

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

create

Create a standard OpenAI error response.

Python

```
create(message: str, type: str = 'invalid_request_error', code: str | None = None)
-> OpenAIError
```

Parameters

[Expand table](#)

Name	Description
<code>message</code> Required*	str
<code>type</code>	str Default value: "invalid_request_error"
<code>code</code>	str None Default value: None

Returns

[Expand table](#)

Type	Description
OpenAIError	

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>by_alias</code>	Default value: False
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>Dict[str, Any]</code>	

from_orm

Python
<code>from_orm(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
<code>obj</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>by_alias</code>	Default value: False
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False
<code>encoder</code>	Default value: PydanticUndefined
<code>models_as_dict</code>	Default value: PydanticUndefined

Returns

 Expand table

Type	Description
<code>str</code>	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

Python


```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: <code>None</code>
<code>values</code> Required*	<code>Any</code> Trusted or pre-validated data dictionary.

Returns

[Expand table](#)

Type	Description
<code>Self</code>	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's `[dict][object.dict]` attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of `[cached properties][functools.cached_property]`).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

 Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
```

```
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
mode Required*	<code>Literal['json', 'python'] str</code> The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings	<code>bool Literal['none', 'warn', 'error']</code>

Name	Description
Required*	How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's `to_json` method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

 Expand table

Name	Description
indent Required*	<code>int None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool </code>

Name	Description
	<code>Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None

Name	Description
<code>ensure_ascii</code>	Default value: False
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>context</code>	Default value: None
<code>by_alias</code>	Default value: None
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False
<code>exclude_computed_fields</code>	Default value: False
<code>round_trip</code>	Default value: False
<code>warnings</code>	Default value: True
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

[🔗](#) Expand table

Type	Description
<code>str</code>	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal ['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default). 'primitive_type_array': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string</i>, <i>boolean</i>, <i>null</i>, <i>integer</i> or <i>number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	type [<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: GenerateJsonSchema
mode	Literal ['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
union_format	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
dict [str , Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModel.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

 Expand table

Name	Description
params Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model[str, int]</i> , the value <i>(str, int)</i> would be passed to <i>params</i> .

Returns

 Expand table

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

 Expand table

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

[Expand table](#)

Name	Description
context Required*	

Parameters

[Expand table](#)

Name	Description
context Required*	Any

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
    _parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
    None) -> bool | None
```

Parameters

[Expand table](#)

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	bool Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> None The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

[Expand table](#)

Type	Description
bool None	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding was required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context:
```

```
Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool None Whether to extract data from object attributes.
context Required*	Any None Additional context to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

 Expand table

Type	Description
Self	The validated model instance.

Exceptions

 Expand table

Type	Description
<code>ValidationError</code>	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.

Name	Description
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
path Required*	str <xref:Path>
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None

Name	Description
<code>allow_pickle</code>	Default value: False

Returns

[Expand table](#)

Type	Description
Self	

parse_obj

Python
<code>parse_obj(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python
<code>parse_raw(b: str bytes, *, content_type: str None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol None = None, allow_pickle: bool = False) -> Self</code>

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

[Expand table](#)

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT\_REF\_TEMPLATE) ->  
Dict[str, Any]
```

Parameters

 Expand table

Name	Description
by_alias	<code>bool</code> Default value: True
ref_template	<code>str</code> Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
<code>Dict[str, Any]</code>	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
by_alias Required*	<code>bool</code>
ref_template Required*	<code>str</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>by_alias</code>	Default value: True
<code>ref_template</code>	Default value: DEFAULT_REF_TEMPLATE

Returns

[Expand table](#)

Type	Description
<code>str</code>	

to_dict

Return the error payload as a plain mapping.

Python
<code>to_dict() -> dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

to_json

Return the error payload serialized to JSON.

Python
<code>to_json() -> str</code>

Returns

[Expand table](#)

Type	Description
str	

update_forward_refs

Python
<code>update_forward_refs(**locals: Any) -> None</code>

Parameters

[↗](#) Expand table

Name	Description
locals Required*	Any

Returns

[↗](#) Expand table

Type	Description
None	

validate

Python
<code>validate(value: Any) -> Self</code>

Parameters

[↗](#) Expand table

Name	Description
value Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

[Expand table](#)

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

error

Python
<code>field error: dict[str, Any] [Required]</code>

lab Package

mem0 Package

Classes

 Expand table

Mem0Provider	Mem0 Context Provider. Initializes a new instance of the Mem0Provider class.
------------------------------	--

Mem0Provider Class

Mem0 Context Provider.

Initializes a new instance of the Mem0Provider class.

Constructor

Python
<code>Mem0Provider()</code>

Parameters

 Expand table

Name	Description
<code>mem0_client</code>	A pre-created Mem0 MemoryClient or None to create a default client. Default value: None
<code>api_key</code>	The API key for authenticating with the Mem0 API. If not provided, it will attempt to use the MEM0_API_KEY environment variable. Default value: None
<code>application_id</code>	The application ID for scoping memories or None. Default value: None
<code>agent_id</code>	The agent ID for scoping memories or None. Default value: None
<code>thread_id</code>	The thread ID for scoping memories or None. Default value: None
<code>user_id</code>	The user ID for scoping memories or None. Default value: None
<code>scope_to_per_operation_thread_id</code>	Whether to scope memories to per-operation thread ID. Default value: False
<code>context_prompt</code>	The prompt to prepend to retrieved memories. Default value: ## Memories Consider the following memories when answering user questions:

Methods

[Expand table](#)

__init__	Initializes a new instance of the Mem0Provider class.
__new__	
invoked	
invoking	Called before invoking the AI model to provide context.
thread_created	Called when a new thread is created.

[__init__](#)

Initializes a new instance of the Mem0Provider class.

Python

```
__init__(mem0_client: AsyncMemory | AsyncMemoryClient | None = None, api_key: str | None = None, application_id: str | None = None, agent_id: str | None = None, thread_id: str | None = None, user_id: str | None = None, scope_to_per_operation_thread_id: bool = False, context_prompt: str = ContextProvider.DEFAULT_CONTEXT_PROMPT) -> None
```

Parameters

[Expand table](#)

Name	Description
mem0_client	<xref:mem0.memory.main.AsyncMemory> <xref:mem0.client.main.AsyncMemoryClient> None A pre-created Mem0 MemoryClient or None to create a default client. Default value: None
api_key	str None The API key for authenticating with the Mem0 API. If not provided, it will attempt to use the MEM0_API_KEY environment variable. Default value: None
application_id	str None The application ID for scoping memories or None. Default value: None

Name	Description
<code>agent_id</code>	<code>str</code> <code>None</code> The agent ID for scoping memories or None. Default value: None
<code>thread_id</code>	<code>str</code> <code>None</code> The thread ID for scoping memories or None. Default value: None
<code>user_id</code>	<code>str</code> <code>None</code> The user ID for scoping memories or None. Default value: None
<code>scope_to_per_operation_thread_id</code>	<code>bool</code> Whether to scope memories to per-operation thread ID. Default value: False
<code>context_prompt</code>	<code>str</code> The prompt to prepend to retrieved memories. Default value: <code>ContextProvider.DEFAULT_CONTEXT_PROMPT</code>

Returns

 Expand table

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

invoked

Python
<pre>async invoked(request_messages: ChatMessage Sequence[ChatMessage], response_messages: ChatMessage Sequence[ChatMessage] None = None, invoke_exception: Exception None = None, **kwargs: Any) -> None</pre>

Parameters

[Expand table](#)

Name	Description
request_messages Required*	ChatMessage Sequence[ChatMessage]
response_messages	ChatMessage Sequence[ChatMessage] None Default value: None
invoke_exception	Exception None Default value: None
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
None	

invoking

Called before invoking the AI model to provide context.

Python

```
async invoking(messages: ChatMessage | MutableSequence[ChatMessage], **kwargs: Any) -> Context
```

Parameters

[Expand table](#)

Name	Description
messages Required*	ChatMessage MutableSequence[ChatMessage] List of new messages in the thread.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	not used at present.

Returns

[Expand table](#)

Type	Description
Context	Context object containing instructions with memories.

thread_created

Called when a new thread is created.

Python
<pre>async thread_created(thread_id: str None = None) -> None</pre>

Parameters

[Expand table](#)

Name	Description
thread_id	str None The ID of the thread or None. Default value: None

Returns

[Expand table](#)

Type	Description
None	

Attributes

DEFAULT_CONTEXT_PROMPT

Python

```
DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories  
when answering user questions:'
```

microsoft Package

Classes

 Expand table

CacheProvider	Protocol for cache providers used by Purview integration.
CopilotStudioAgent	A Copilot Studio Agent. Initialize the Copilot Studio Agent.
PurviewAppLocation	Identifier representing the app's location for Purview policy evaluation. Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
PurviewAuthenticationError	Authentication / authorization failure (401/403). Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewChatPolicyMiddleware	Chat middleware variant for Purview policy evaluation. This allows users to attach Purview enforcement directly to a chat client Behavior: • Pre-chat: evaluates outgoing (user + context) messages as an upload activity and can terminate execution if blocked. • Post-chat: evaluates the received response messages (streaming is not presently supported) and can replace them with a blocked message. Uses the same user_id from the request to ensure consistent user identity throughout the evaluation. Usage:
PurviewPaymentRequiredError	Payment required (402). Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewPolicyMiddleware	Agent middleware that enforces Purview policies on prompt and response.

	Accepts either a synchronous TokenCredential or an AsyncTokenCredential. Usage:
PurviewRateLimitError	Rate limiting or throttling (429). Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewRequestError	Other non-success HTTP errors. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewServiceError	Base exception for Purview errors. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewSettings	Settings for Purview integration mirroring .NET PurviewSettings. Initialize the settings class. JSON <pre>{ "title": "PurviewSettings", "description": "Settings for Purview integration mirroring .NET PurviewSettings.\n\nAttributes:\napp_name: Public app name.\n app_version: Optional version string of the application.\n tenant_id: Optional tenant id (guid) of the user making the request.\n purview_app_location: Optional app location for policy evaluation.\n graph_base_uri: Base URI for Microsoft Graph.\n blocked_prompt_message: Custom message to return when a prompt is blocked by policy.\n blocked_response_message: Custom message to return when a response is blocked by policy.\n ignore_exceptions: If True, all Purview exceptions will be logged but not thrown in middleware.\n ignore_payment_required: If True, 402 payment required errors will be logged but not thrown.\n cache_ttl_seconds: Time to live for cache entries in seconds (default 14400 = 4 hours).\n max_cache_size_bytes: Maximum cache size in bytes (default 200MB).", "type": "object", "properties": { "env_file_path": { "anyOf": [{ "type": "string"</pre>


```

        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Env File Path"
},
"env_file_encoding": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": "utf-8",
    "title": "Env File Encoding"
},
"app_name": {
    "title": "App Name",
    "type": "string"
},
"app_version": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "App Version"
},
"tenant_id": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Tenant Id"
},
"purview_app_location": {
    "anyOf": [
        {
            "$ref": "#/$defs/PurviewAppLocation"
        },
        {
            "type": "null"
        }
    ]
}

```

```

    }
  ],
  "default": null
},
"graph_base_uri": {
  "default":
"https://graph.microsoft.com/v1.0/",
  "title": "Graph Base Uri",
  "type": "string"
},
"blocked_prompt_message": {
  "default": "Prompt blocked by policy",
  "description": "Message to return when a
prompt is blocked by policy.",
  "title": "Blocked Prompt Message",
  "type": "string"
},
"blocked_response_message": {
  "default": "Response blocked by policy",
  "description": "Message to return when a
response is blocked by policy.",
  "title": "Blocked Response Message",
  "type": "string"
},
"ignore_exceptions": {
  "default": false,
  "description": "If True, all Purview
exceptions will be logged but not thrown in middleware.",
  "title": "Ignore Exceptions",
  "type": "boolean"
},
"ignore_payment_required": {
  "default": false,
  "description": "If True, 402 payment required
errors will be logged but not thrown.",
  "title": "Ignore Payment Required",
  "type": "boolean"
},
"cache_ttl_seconds": {
  "default": 14400,
  "description": "Time to live for cache
entries in seconds (default 14400 = 4 hours).",
  "title": "Cache Ttl Seconds",
  "type": "integer"
},
"max_cache_size_bytes": {
  "default": 209715200,
  "description": "Maximum cache size in bytes
(default 200MB).",
  "title": "Max Cache Size Bytes",
  "type": "integer"
}
},
"$defs": {
  "PurviewAppLocation": {

```

```

        "description": "Identifier representing the
app's location for Purview policy evaluation.",
        "properties": {
            "location_type": {
                "$ref": "#/$defs/PurviewLocationType",
                "description": "The location type."
            },
            "location_value": {
                "description": "The location value.",
                "title": "Location Value",
                "type": "string"
            }
        },
        "required": [
            "location_type",
            "location_value"
        ],
        "title": "PurviewAppLocation",
        "type": "object"
    },
    "PurviewLocationType": {
        "description": "The type of location for
Purview policy evaluation.",
        "enum": [
            "application",
            "uri",
            "domain"
        ],
        "title": "PurviewLocationType",
        "type": "string"
    }
},
"required": [
    "app_name"
]
}

```

Enums

[Expand table](#)

PurviewLocationType

The type of location for Purview policy evaluation.

Functions

acquire_token

Acquire an authentication token using MSAL Public Client Application.

This function attempts to acquire a token silently first (using cached tokens), and falls back to interactive authentication if needed.

Python

```
acquire_token(*, client_id: str, tenant_id: str, username: str | None = None,
token_cache: Any | None = None, scopes: list[str] | None = None) -> str
```

Parameters

 Expand table

Name	Description
client_id Required*	str
tenant_id Required*	str
username Required*	str None
token_cache Required*	Any None
scopes Required*	list[str] None

Keyword-Only Parameters

 Expand table

Name	Description
client_id	The client ID of the application.
tenant_id	The tenant ID for authentication.
username	Optional username to filter accounts. Default value: None
token_cache	Optional token cache for storing tokens. Default value: None
scopes	Optional list of scopes. Defaults to Power Platform API scopes. Default value: None

Returns

 Expand table

Type	Description
str	The access token string.

Exceptions

 Expand table

Type	Description
ServiceException	If authentication token cannot be acquired.

CacheProvider Class

Protocol for cache providers used by Purview integration.

Constructor

Python
<code>CacheProvider()</code>

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>get</code>	Get a value from the cache.
<code>remove</code>	Remove a value from the cache.
<code>set</code>	Set a value in the cache.

`__init__`

Python
<code>__init__(*args, **kwargs)</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`get`

Get a value from the cache.

Python

```
async get(key: str) -> Any | None
```

Parameters

[Expand table](#)

Name	Description
key Required*	<code>str</code> The cache key.

Returns

[Expand table](#)

Type	Description
<code>Any None</code>	The cached value or None if not found or expired.

remove

Remove a value from the cache.

Python

```
async remove(key: str) -> None
```

Parameters

[Expand table](#)

Name	Description
key Required*	<code>str</code> The cache key.

Returns

[Expand table](#)

Type	Description
<code>None</code>	

set

Set a value in the cache.

Python

```
async set(key: str, value: Any, ttl_seconds: int | None = None) -> None
```

Parameters

 Expand table

Name	Description
key Required*	<code>str</code> The cache key.
value Required*	<code>Any</code> The value to cache.
ttl_seconds	<code>int None</code> Time to live in seconds. If None, uses provider default. Default value: None

Returns

 Expand table

Type	Description
<code>None</code>	

CopilotStudioAgent Class

A Copilot Studio Agent.

Initialize the Copilot Studio Agent.

Constructor

Python
<code>CopilotStudioAgent()</code>

Parameters

 Expand table

Name	Description
<code>client</code>	Optional pre-configured CopilotClient instance. If not provided, a new client will be created using the other parameters. Default value: None
<code>settings</code>	Optional pre-configured ConnectionSettings. If not provided, settings will be created from the other parameters. Default value: None

Keyword-Only Parameters

 Expand table

Name	Description
<code>id</code>	id of the CopilotAgent Default value: None
<code>name</code>	Name of the CopilotAgent Default value: None
<code>description</code>	Description of the CopilotAgent Default value: None
<code>context_providers</code>	Context Providers, to be used by the copilot agent. Default value: None

Name	Description
middleware	Agent middlewares used by the agent. Default value: None
environment_id	Environment ID of the Power Platform environment containing the Copilot Studio app. Can also be set via COPILOTSTUDIOAGENT_ENVIRONMENTID environment variable. Default value: None
agent_identifier	The agent identifier or schema name of the Copilot to use. Can also be set via COPILOTSTUDIOAGENT_SCHEMANAME environment variable. Default value: None
client_id	The app ID of the App Registration used for authentication. Can also be set via COPILOTSTUDIOAGENT_AGENTAPPID environment variable. Default value: None
tenant_id	The tenant ID of the App Registration used for authentication. Can also be set via COPILOTSTUDIOAGENT_TENANTID environment variable. Default value: None
token	Optional pre-acquired authentication token. If not provided, token acquisition will be attempted using MSAL. Default value: None
cloud	The Power Platform cloud to use (Public, GCC, etc.). Default value: None
agent_type	The type of Copilot Studio agent (Copilot, Agent, etc.). Default value: None
custom_power_platform_cloud	Custom Power Platform cloud URL if using a custom environment. Default value: None
username	Optional username for token acquisition. Default value: None
token_cache	Optional token cache for storing authentication tokens. Default value: None
scopes	Optional list of authentication scopes. Defaults to Power Platform API scopes if not provided. Default value: None
env_file_path	Optional path to .env file for loading configuration. Default value: None
env_file_encoding	Encoding of the .env file, defaults to 'utf-8'. Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the Copilot Studio Agent.
<code>__new__</code>	
<code>as_tool</code>	Create an <code>AIFunction</code> tool that wraps this agent.
<code>deserialize_thread</code>	Deserialize a thread from its serialized state.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_new_thread</code>	Return a new <code>AgentThread</code> instance that is compatible with the agent.
<code>run</code>	Get a response from the agent. This method returns the final result of the agent's execution as a single <code>AgentRunResponse</code> object. The caller is blocked until the final result is available. Note: For streaming responses, use the <code>run_stream</code> method, which returns intermediate steps and the final result as a stream of <code>AgentRunResponseUpdate</code> objects. Streaming only the final result is not feasible because the timing of the final result's availability is unknown, and blocking the caller until then is undesirable in streaming scenarios.
<code>run_stream</code>	Run the agent as a stream. This method will return the intermediate steps and final results of the agent's execution as a stream of <code>AgentRunResponseUpdate</code> objects to the caller. Note: An <code>AgentRunResponseUpdate</code> object contains a chunk of a message.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

`to_json` Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize the Copilot Studio Agent.

Python

```
__init__(client: CopilotClient | None = None, settings: ConnectionSettings | None = None, *, id: str | None = None, name: str | None = None, description: str | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]] | None = None, environment_id: str | None = None, agent_identifier: str | None = None, client_id: str | None = None, tenant_id: str | None = None, token: str | None = None, cloud: PowerPlatformCloud | None = None, agent_type: AgentType | None = None, custom_power_platform_cloud: str | None = None, username: str | None = None, token_cache: Any | None = None, scopes: list[str] | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>client</code>	<code>CopilotClient</code> <code>None</code> Optional pre-configured CopilotClient instance. If not provided, a new client will be created using the other parameters. Default value: None
<code>settings</code>	<code>ConnectionSettings</code> <code>None</code> Optional pre-configured ConnectionSettings. If not provided, settings will be created from the other parameters. Default value: None
<code>id</code> Required*	<code>str</code> <code>None</code>
<code>name</code>	<code>str</code> <code>None</code>

Name	Description
Required*	
description Required*	str None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] None
environment_id Required*	str None
agent_identifier Required*	str None
client_id Required*	str None
tenant_id Required*	str None
token Required*	str None
cloud Required*	PowerPlatformCloud None
agent_type Required*	AgentType None
custom_power_platform_cloud Required*	str None
username Required*	str None
token_cache Required*	Any None
scopes Required*	list[str] None
env_file_path Required*	str None

Name	Description
env_file_encoding Required*	str None

Keyword-Only Parameters

 Expand table

Name	Description
id	id of the CopilotAgent Default value: None
name	Name of the CopilotAgent Default value: None
description	Description of the CopilotAgent Default value: None
context_providers	Context Providers, to be used by the copilot agent. Default value: None
middleware	Agent middlewares used by the agent. Default value: None
environment_id	Environment ID of the Power Platform environment containing the Copilot Studio app. Can also be set via COPILOTSTUDIOAGENT_ENVIRONMENTID environment variable. Default value: None
agent_identifier	The agent identifier or schema name of the Copilot to use. Can also be set via COPILOTSTUDIOAGENT_SCHEMANAME environment variable. Default value: None
client_id	The app ID of the App Registration used for authentication. Can also be set via COPILOTSTUDIOAGENT_AGENTAPPID environment variable. Default value: None
tenant_id	The tenant ID of the App Registration used for authentication. Can also be set via COPILOTSTUDIOAGENT_TENANTID environment variable. Default value: None
token	Optional pre-acquired authentication token. If not provided, token acquisition will be attempted using MSAL. Default value: None

Name	Description
<code>cloud</code>	The Power Platform cloud to use (Public, GCC, etc.). Default value: None
<code>agent_type</code>	The type of Copilot Studio agent (Copilot, Agent, etc.). Default value: None
<code>custom_power_platform_cloud</code>	Custom Power Platform cloud URL if using a custom environment. Default value: None
<code>username</code>	Optional username for token acquisition. Default value: None
<code>token_cache</code>	Optional token cache for storing authentication tokens. Default value: None
<code>scopes</code>	Optional list of authentication scopes. Defaults to Power Platform API scopes if not provided. Default value: None
<code>env_file_path</code>	Optional path to .env file for loading configuration. Default value: None
<code>env_file_encoding</code>	Encoding of the .env file, defaults to 'utf-8'. Default value: None

Returns

 Expand table

Type	Description
<code>None</code>	

Exceptions

 Expand table

Type	Description
<code>ServiceInitializationError</code>	If required configuration is missing or invalid.

__new__

Python

```
__new__(**kwargs)
```

as_tool

Create an AIFunction tool that wraps this agent.

Python

```
as_tool(*, name: str | None = None, description: str | None = None, arg_name: str = 'task', arg_description: str | None = None, stream_callback: Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate], Awaitable[None]] | None = None) -> AIFunction[BaseModel, str]
```

Parameters

 Expand table

Name	Description
name Required*	str None
description Required*	str None
arg_name Required*	str
arg_description Required*	str None
stream_callback Required*	Callable[[AgentRunResponseUpdate], None] Callable[[AgentRunResponseUpdate], Awaitable[None]] None

Keyword-Only Parameters

 Expand table

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task").

Name	Description
	Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

[Expand table](#)

Type	Description
AIFunction [<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

[Expand table](#)

Type	Description
TypeError	If the agent does not implement AgentProtocol.
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
from agent_framework import ChatAgent

# Create an agent
agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

# Convert the agent to a tool
research_tool = agent.as_tool()

# Use the tool with another agent
coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

 Expand table

Name	Description
serialized_thread Required*	Any The serialized thread data.
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
kwargs	Additional keyword arguments.

Returns

 Expand table

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

from_dict()

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated
```

```

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
<code>from_json()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Return a new AgentThread instance that is compatible with the agent.

Python
<code>get_new_thread(**kwargs: Any) -> AgentThread</code>

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Additional keyword arguments passed to AgentThread.

Returns

[Expand table](#)

Type	Description
AgentThread	A new AgentThread instance configured with the agent's context provider.

run

Get a response from the agent.

This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available.

Note: For streaming responses, use the `run_stream` method, which returns intermediate steps and the final result as a stream of AgentRunResponseUpdate objects. Streaming only the final result is not feasible because the timing of the final result's availability is unknown, and blocking the caller until then is undesirable in streaming scenarios.

Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, **kwargs: Any) -> AgentRunResponse
```

Parameters

[Expand table](#)

Name	Description
messages	str ChatMessage list[str] list[ChatMessage] None The message(s) to send to the agent. Default value: None
thread Required*	AgentThread None
kwargs	Any

Name	Description
Required*	

Keyword-Only Parameters

[Expand table](#)

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

[Expand table](#)

Type	Description
AgentRunResponse	An agent response item.

run_stream

Run the agent as a stream.

This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller.

Note: An AgentRunResponseUpdate object contains a chunk of a message.

Python

```
async run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] |
None = None, *, thread: AgentThread | None = None, **kwargs: Any) ->
AsyncIterable[AgentRunResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages	str ChatMessage list[str] list[ChatMessage] None

Name	Description
	The message(s) to send to the agent. Default value: None
thread Required*	AgentThread None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

 Expand table

Type	Description
AsyncIterable [AgentRunResponseUpdate]	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

`display_name`

Returns the display name of the agent.

This is the name if present, otherwise the id.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

additional_properties

Python

```
additional_properties: dict[str, Any]
```

middleware

Python

```
middleware: list[Middleware] | None
```

PurviewAppLocation Class

Identifier representing the app's location for Purview policy evaluation.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python
pydantic model PurviewAppLocation

Keyword-Only Parameters

 Expand table

Name	Description
location_type Required*	
location_value Required*	

Methods

 Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
__new__	

construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <i>model_copy</i> instead. If you need <i>include</i> or <i>exclude</i>, use: <pre>python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)</pre>
dict	
from_orm	
get_policy_location	
json	
model_construct	Creates a new instance of the <i>Model</i> class with validated data. Creates a new model setting <i>dict</i> and <i>pydantic_fields_set</i> from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note <i>model_construct()</i> generally respects the <i>model_config.extra</i> setting on the provided model. That is, if <i>model_config.extra</i> == 'allow', then all extra passed values are added to the model instance's <i>dict</i> and <i>pydantic_extra</i> fields. If <i>model_config.extra</i> == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to <i>model_construct()</i>, having <i>model_config.extra</i> == 'forbid' does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" model_copy Returns a copy of the model. !!! note The underlying instance's [<i>dict</i>][object.<i>dict</i>] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" model_dump Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" model_dump_json Generates a JSON representation of the model using Pydantic's <i>to_json</i> method.

<code>model_json_schema</code>	Generates a JSON schema for a model class.
<code>model_parametrized_name</code>	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
<code>model_post_init</code>	Override this method to perform additional initialization after <i>init</i> and <i>model_construct</i> . This is useful if you want to do some validation that requires the entire model to be initialized.
<code>model_rebuild</code>	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
<code>model_validate</code>	Validate a pydantic model instance.
<code>model_validate_json</code>	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
<code>model_validate_strings</code>	Validate the given object with string data against the Pydantic model.
<code>parse_file</code>	
<code>parse_obj</code>	
<code>parse_raw</code>	
<code>schema</code>	
<code>schema_json</code>	
<code>update_forward_refs</code>	
<code>validate</code>	

`__init__`

Create a new model by parsing and validating input data from keyword arguments.

Raises `[ValidationError][pydantic_core.ValidationError]` if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.


```
__init__(**data: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
data Required*	Any

Returns

[Expand table](#)

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
_fields_set	set[str] None Default value: None
values Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude:
AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None
= None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
include Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None

Name	Description
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

 Expand table

Type	Description
<code>Dict[str, Any]</code>	

from_orm

Python
<code>from_orm(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

get_policy_location

Python

```
get_policy_location() -> dict[str, str]
```

Returns

[Expand table](#)

Type	Description
dict[str, str]	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |  
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,  
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |  
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |  
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |  
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx  
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =  
False, exclude_defaults: bool = False, exclude_none: bool = False, encoder:  
Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool =  
PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
exclude Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Name	Description
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

[Expand table](#)

Type	Description
<code>str</code>	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

Python
<code>model_construct(_fields_set: set[str] None = None, **values: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None

Name	Description
values Required*	Any Trusted or pre-validated data dictionary.

Returns

 Expand table

Type	Description
Self	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.***dict***] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python
<code>model_copy(*, update: Mapping[str, Any] None = None, deep: bool = False) -> Self</code>

Parameters

 Expand table

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

 Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python
<pre>model_dump(*, mode: Literal['json', 'python'] str = 'python', include: set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool Mapping[str, set[int]] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool None = None, exclude: set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool Mapping[str, set[int]] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool None = None, context: Any None = None, by_alias: bool None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] None = None, serialize_as_any: bool = False) -> dict[str, Any]</pre>

Parameters

 Expand table

Name	Description
mode Required*	Literal ['json', 'python'] str The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.

Name	Description
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

 Expand table

Name	Description
indent Required*	<code>int None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults	<code>bool</code>

Name	Description
Required*	Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any]</code> <code>None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
<code>exclude_none</code>	Default value: False
<code>exclude_computed_fields</code>	Default value: False
<code>round_trip</code>	Default value: False
<code>warnings</code>	Default value: True
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>str</code>	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
<code>by_alias</code>	<code>bool</code> Whether to use attribute aliases or not. Default value: True
<code>ref_template</code>	<code>str</code> The reference template. Default value: DEFAULT_REF_TEMPLATE
<code>union_format</code>	<code>Literal['any_of', 'primitive_type_array']</code>

Name	Description
Required*	The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default). 'primitive_type_array': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string, boolean, null, integer or number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: <i>GenerateJsonSchema</i>
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
union_format	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
dict [str , Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

[Expand table](#)

Name	Description
params Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model[str, int]</i> , the value (<i>str</i> , <i>int</i>) would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

[Expand table](#)

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

[Expand table](#)

Name	Description
context Required*	

Parameters

[Expand table](#)

Name	Description
context Required*	Any

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

[Expand table](#)

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors	bool

Name	Description
Required*	Whether to raise errors, defaults to <i>True</i> .
<code>_parent_namespace_depth</code> Required*	<code>int</code> The depth level of the parent namespace, defaults to 2.
<code>_types_namespace</code> Required*	<code><xref:MappingNamespace></code> <code>None</code> The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>force</code>	Default value: <code>False</code>
<code>raise_errors</code>	Default value: <code>True</code>
<code>_parent_namespace_depth</code>	Default value: <code>2</code>
<code>_types_namespace</code>	Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding was required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool None Whether to extract data from object attributes.
context Required*	Any None Additional context to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python
<pre>model_validate_json(json_data: str bytes bytearray, *, strict: bool None = None, extra: Literal['allow', 'ignore', 'forbid'] None = None, context: Any None = None, by_alias: bool None = None, by_name: bool None = None) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name	bool None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [<i>extra</i> configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

 Expand table

Name	Description
path Required*	<code>str</code> <code><xref:Path></code>
content_type Required*	<code>str</code> <code>None</code>
encoding Required*	<code>str</code>
proto Required*	<code><xref:DeprecatedParseProtocol></code> <code>None</code>
allow_pickle Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: <code>None</code>
encoding	Default value: <code>'utf8'</code>
proto	Default value: <code>None</code>
allow_pickle	Default value: <code>False</code>

Returns

 Expand table

Type	Description
Self	

parse_obj

Python
<pre>parse_obj(obj: Any) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python
<pre>parse_raw(b: str bytes, *, content_type: str None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol None = None, allow_pickle: bool = False) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes

Name	Description
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

 Expand table

Type	Description
Self	

schema

Python
<pre>schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) -> Dict[str, Any]</pre>

Parameters

 Expand table

Name	Description
by_alias	<code>bool</code> Default value: True
ref_template	<code>str</code> Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
<code>Dict[str, Any]</code>	

schema_json

Python
<pre>schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE, **kwargs: Any) -> str</pre>

Parameters

 Expand table

Name	Description
by_alias Required*	<code>bool</code>
ref_template Required*	<code>str</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
by_alias	Default value: True

Name	Description
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

[⌵](#) Expand table

Type	Description
str	

update_forward_refs

Python
<pre>update_forward_refs(**locals: Any) -> None</pre>

Parameters

[⌵](#) Expand table

Name	Description
locals Required*	Any

Returns

[⌵](#) Expand table

Type	Description
None	

validate

Python
<pre>validate(value: Any) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
value Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

[Expand table](#)

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

location_type

The location type.

Python

```
field location_type: PurviewLocationType [Required]
```

location_value

The location value.

Python

```
field location_value: str [Required]
```

PurviewAuthenticationError Class

Authentication / authorization failure (401/403).

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
PurviewAuthenticationError()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

PurviewChatPolicyMiddleware Class

Chat middleware variant for Purview policy evaluation.

This allows users to attach Purview enforcement directly to a chat client

Behavior:

- Pre-chat: evaluates outgoing (user + context) messages as an upload activity and can terminate execution if blocked.
- Post-chat: evaluates the received response messages (streaming is not presently supported) and can replace them with a blocked message. Uses the same user_id from the request to ensure consistent user identity throughout the evaluation.

Usage:

Constructor

Python
<pre>PurviewChatPolicyMiddleware()</pre>

Parameters

 Expand table

Name	Description
credential Required*	
settings Required*	
cache_provider	Default value: None

Methods

 Expand table

<code>__init__</code>

`__new__`

`process`

`__init__`

Python

```
__init__(credential: TokenCredential | AsyncTokenCredential, settings: PurviewSettings, cache_provider: CacheProvider | None = None) -> None
```

Parameters

 Expand table

Name	Description
credential Required*	TokenCredential AsyncTokenCredential
settings Required*	PurviewSettings
cache_provider	CacheProvider None Default value: None

Returns

 Expand table

Type	Description
None	

`__new__`

Python

```
__new__(**kwargs)
```

`process`

Python

```
async process(context: ChatContext, next: Callable[[ChatContext],  
Awaitable[None]]) -> None
```

Parameters

 Expand table

Name	Description
context Required*	ChatContext
next Required*	Callable [[ChatContext], Awaitable [None]]

Returns

 Expand table

Type	Description
None	

PurviewLocationType Enum

The type of location for Purview policy evaluation.

Fields

 Expand table

APPLICATION
DOMAIN
URI

PurviewPaymentRequiredError Class

Payment required (402).

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
PurviewPaymentRequiredError()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	<code>str</code>
inner_exception	<code>Exception None</code> Default value: <code>None</code>
log_level	<code>Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None</code> Default value: <code>10</code>
args Required*	<code>Any</code>
kwargs Required*	<code>Any</code>

`__new__`

Python

```
__new__(**kwargs)
```

`add_note`

`Exception.add_note(note)` – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

PurviewPolicyMiddleware Class

Agent middleware that enforces Purview policies on prompt and response.

Accepts either a synchronous TokenCredential or an AsyncTokenCredential.

Usage:

Constructor

Python
<pre>PurviewPolicyMiddleware()</pre>

Parameters

 Expand table

Name	Description
credential Required*	
settings Required*	
cache_provider	Default value: None

Methods

 Expand table

__init__
__new__
process

[__init__](#)

Python

```
__init__(credential: TokenCredential | AsyncTokenCredential, settings:
PurviewSettings, cache_provider: CacheProvider | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
credential Required*	TokenCredential AsyncTokenCredential
settings Required*	PurviewSettings
cache_provider	CacheProvider None Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

process

Python
<code>async process(context: AgentRunContext, next: Callable[[AgentRunContext], Awaitable[None]]) -> None</code>

Parameters

[Expand table](#)

Name	Description
context Required*	AgentRunContext
next Required*	Callable [[AgentRunContext], Awaitable [None]]

Returns

 Expand table

Type	Description
None	

PurviewRateLimitError Class

Rate limiting or throttling (429).

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
PurviewRateLimitError()

Parameters

[Expand table](#)

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

[Expand table](#)

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

PurviewRequestError Class

Other non-success HTTP errors.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
PurviewRequestError()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

PurviewServiceError Class

Base exception for Purview errors.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
PurviewServiceError()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

PurviewSettings Class

Settings for Purview integration mirroring .NET PurviewSettings.

Initialize the settings class.

JSON

```
{
  "title": "PurviewSettings",
  "description": "Settings for Purview integration mirroring .NET\n\nPurviewSettings.\n\nAttributes:\n  app_name: Public app name.\n  app_version: Optional version string of the application.\n  tenant_id: Optional tenant id (guid) of the user making the request.\n  purview_app_location: Optional app location for policy evaluation.\n  graph_base_uri: Base URI for Microsoft Graph.\n  blocked_prompt_message: Custom message to return when a prompt is blocked by policy.\n  blocked_response_message: Custom message to return when a response is blocked by policy.\n  ignore_exceptions: If True, all Purview exceptions will be logged but not thrown in middleware.\n  ignore_payment_required: If True, 402 payment required errors will be logged but not thrown.\n  cache_ttl_seconds: Time to live for cache entries in seconds (default 14400 = 4 hours).\n  max_cache_size_bytes: Maximum cache size in bytes (default 200MB).",
  "type": "object",
  "properties": {
    "env_file_path": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Env File Path"
    },
    "env_file_encoding": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": "utf-8",
      "title": "Env File Encoding"
    },
    "app_name": {
      "title": "App Name",

```

```
    "type": "string"
  },
  "app_version": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "App Version"
  },
  "tenant_id": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Tenant Id"
  },
  "purview_app_location": {
    "anyOf": [
      {
        "$ref": "#/$defs/PurviewAppLocation"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "graph_base_uri": {
    "default": "https://graph.microsoft.com/v1.0/",
    "title": "Graph Base Uri",
    "type": "string"
  },
  "blocked_prompt_message": {
    "default": "Prompt blocked by policy",
    "description": "Message to return when a prompt is blocked by policy.",
    "title": "Blocked Prompt Message",
    "type": "string"
  },
  "blocked_response_message": {
    "default": "Response blocked by policy",
    "description": "Message to return when a response is blocked by policy.",
    "title": "Blocked Response Message",
    "type": "string"
  },
  "ignore_exceptions": {
```

```

        "default": false,
        "description": "If True, all Purview exceptions will be logged but not
thrown in middleware.",
        "title": "Ignore Exceptions",
        "type": "boolean"
    },
    "ignore_payment_required": {
        "default": false,
        "description": "If True, 402 payment required errors will be logged but
not thrown.",
        "title": "Ignore Payment Required",
        "type": "boolean"
    },
    "cache_ttl_seconds": {
        "default": 14400,
        "description": "Time to live for cache entries in seconds (default 14400 =
4 hours).",
        "title": "Cache Ttl Seconds",
        "type": "integer"
    },
    "max_cache_size_bytes": {
        "default": 209715200,
        "description": "Maximum cache size in bytes (default 200MB).",
        "title": "Max Cache Size Bytes",
        "type": "integer"
    }
},
"$defs": {
    "PurviewAppLocation": {
        "description": "Identifier representing the app's location for Purview
policy evaluation.",
        "properties": {
            "location_type": {
                "$ref": "#/$defs/PurviewLocationType",
                "description": "The location type."
            },
            "location_value": {
                "description": "The location value.",
                "title": "Location Value",
                "type": "string"
            }
        },
        "required": [
            "location_type",
            "location_value"
        ],
        "title": "PurviewAppLocation",
        "type": "object"
    },
    "PurviewLocationType": {
        "description": "The type of location for Purview policy evaluation.",
        "enum": [
            "application",
            "uri",
            "domain"
        ]
    }
}

```

```

    ],
    "title": "PurviewLocationType",
    "type": "string"
  },
},
"required": [
  "app_name"
]
}

```

Constructor

Python

pydantic settings PurviewSettings

Keyword-Only Parameters

[Expand table](#)

Name	Description
env_file_path	Default value: None
env_file_encoding	Default value: utf-8
app_name Required*	
app_version	Default value: None
tenant_id	Default value: None
purview_app_location	Default value: None
graph_base_uri	Default value: https://graph.microsoft.com/v1.0/
blocked_prompt_message	Default value: Prompt blocked by policy
blocked_response_message	Default value: Response blocked by policy
ignore_exceptions	Default value: False
ignore_payment_required	Default value: False
cache_ttl_seconds	Default value: 14400
max_cache_size_bytes	Default value: 209715200

Methods

 Expand table

<code>__init__</code>	Initialize the settings class.
<code>__new__</code>	Override the new method to set the <code>env_prefix</code> .
<code>construct</code>	
<code>copy</code>	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <i>model_copy</i> instead. If you need <i>include</i> or <i>exclude</i>, use: <pre>python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)</pre>
<code>dict</code>	
<code>from_orm</code>	
<code>get_scopes</code>	
<code>json</code>	
<code>model_construct</code>	Creates a new instance of the <i>Model</i> class with validated data. Creates a new model setting <i>dict</i> and <i>pydantic_fields_set</i> from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note <i>model_construct()</i> generally respects the <i>model_config.extra</i> setting on the provided model. That is, if <i>model_config.extra</i> == 'allow', then all extra passed values are added to the model instance's <i>dict</i> and <i>pydantic_extra</i> fields. If <i>model_config.extra</i> == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to <i>model_construct()</i>, having <i>model_config.extra</i> == 'forbid' does not result in an error if extra values are passed, but they will be ignored.
<code>model_copy</code>	!!! abstract "Usage Documentation" <i>model_copy</i> Returns a copy of the model. !!! note The underlying instance's [<i>dict</i>][<code>object.dict</code>] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [<code>cached properties</code>][<code>functools.cached_property</code>]).

<code>model_dump</code>	!!! abstract "Usage Documentation" model_dump Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
<code>model_dump_json</code>	!!! abstract "Usage Documentation" model_dump_json Generates a JSON representation of the model using Pydantic's <i>to_json</i> method.
<code>model_json_schema</code>	Generates a JSON schema for a model class.
<code>model_parametrized_name</code>	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModel.
<code>model_post_init</code>	Override this method to perform additional initialization after <i>__init__</i> and <i>model_construct</i> . This is useful if you want to do some validation that requires the entire model to be initialized.
<code>model_rebuild</code>	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
<code>model_validate</code>	Validate a pydantic model instance.
<code>model_validate_json</code>	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
<code>model_validate_strings</code>	Validate the given object with string data against the Pydantic model.
<code>parse_file</code>	
<code>parse_obj</code>	
<code>parse_raw</code>	
<code>schema</code>	
<code>schema_json</code>	
<code>update_forward_refs</code>	
<code>validate</code>	

`__init__`

Initialize the settings class.

Python

```
__init__(**kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

Python

```
static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings
```

Parameters

[Expand table](#)

Name	Description
cls Required*	type [<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._pydantic.TSettings></code>	

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> Default value: None
<code>values</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,  
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =  
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
include Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code>
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code>
by_alias Required*	<code>bool</code>
exclude_unset Required*	<code>bool</code>
exclude_defaults Required*	<code>bool</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False

Name	Description
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>Dict[str, Any]</code>	

from_orm

Python
<code>from_orm(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
<code>obj</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

get_scopes

Python
<code>get_scopes() -> list[str]</code>

Returns

[Expand table](#)

Type	Description
<code>list[str]</code>	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code>
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code>
by_alias Required*	<code>bool</code>
exclude_unset Required*	<code>bool</code>
exclude_defaults Required*	<code>bool</code>
exclude_none Required*	<code>bool</code>

Name	Description
encoder Required*	Callable [[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

[⌵](#) Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

[⌵](#) Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note `model_construct()` generally respects the `model_config.extra` setting on the provided model. That is, if `model_config.extra == 'allow'`, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If `model_config.extra == 'ignore'` (the default), then all extra passed values are ignored. Because no validation is performed with a call to `model_construct()`, having `model_config.extra == 'forbid'` does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None
<code>values</code> Required*	<code>Any</code> Trusted or pre-validated data dictionary.

Returns

[Expand table](#)

Type	Description
<code>Self</code>	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's `[dict][object.dict]` attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of `[cached properties][functools.cached_property]`).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

[Expand table](#)

Name	Description
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int]
| set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] |
Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
mode Required*	<code>Literal['json', 'python'] str</code> The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .

Name	Description
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	bool Literal ['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	Callable [[Any], Any] None A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True

Name	Description
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's `to_json` method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

 Expand table

Name	Description
<code>indent</code> Required*	<code>int None</code> Indentation to use in the JSON output. If None is passed, the output will be compact.
<code>ensure_ascii</code> Required*	<code>bool</code>

Name	Description
	If <i>True</i> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <i>False</i> (the default), these characters will be output as-is.
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If <i>True</i> , dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool Literal['none', 'warn', 'error']</code> How to handle serialization errors. <i>False</i> /"none" ignores them, <i>True</i> /"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.

Name	Description
<code>serialize_as_any</code> Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>indent</code>	Default value: None
<code>ensure_ascii</code>	Default value: False
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>context</code>	Default value: None
<code>by_alias</code>	Default value: None
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False
<code>exclude_computed_fields</code>	Default value: False
<code>round_trip</code>	Default value: False
<code>warnings</code>	Default value: True
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>str</code>	A JSON string representation of the model.

`model_json_schema`

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal ['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default). 'primitive_type_array': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string</i>, <i>boolean</i>, <i>null</i>, <i>integer</i> or <i>number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	type [<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: GenerateJsonSchema
mode	Literal ['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>union_format</code>	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModel.

Python
<code>model_parametrized_name(params: tuple[type[Any], ...]) -> str</code>

Parameters

[Expand table](#)

Name	Description
params Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model[str, int]</i> , the value <i>(str, int)</i> would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

[Expand table](#)

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python
<code>model_post_init(context: Any, /) -> None</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
context Required*	

Parameters

[Expand table](#)

Name	Description
context Required*	Any

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

[Expand table](#)

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	bool Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> None The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

[Expand table](#)

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
obj Required*	<code>Any</code> The object to validate.
strict Required*	<code>bool</code> <code>None</code> Whether to enforce types strictly.
extra Required*	<code>Literal</code> ['allow', 'ignore', 'forbid'] <code>None</code> Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	<code>bool</code> <code>None</code> Whether to extract data from object attributes.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the validator.
by_alias Required*	<code>bool</code> <code>None</code> Whether to use the field's alias when validating against the provided input data.
by_name Required*	<code>bool</code> <code>None</code> Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
<code>strict</code>	Default value: None
<code>extra</code>	Default value: None
<code>from_attributes</code>	Default value: None
<code>context</code>	Default value: None
<code>by_alias</code>	Default value: None
<code>by_name</code>	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
<code>ValidationError</code>	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 [Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

 [Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

 [Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None

Name	Description
<code>extra</code>	Default value: None
<code>context</code>	Default value: None
<code>by_alias</code>	Default value: None
<code>by_name</code>	Default value: None

Returns

[↗](#) Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[↗](#) Expand table

Name	Description
<code>path</code> Required*	str <xref:Path>
<code>content_type</code> Required*	str None
<code>encoding</code> Required*	str
<code>proto</code> Required*	<xref:DeprecatedParseProtocol> None
<code>allow_pickle</code> Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

 Expand table

Type	Description
Self	

parse_obj

Python
<code>parse_obj(obj: Any) -> Self</code>

Parameters

 Expand table

Name	Description
obj Required*	Any

Returns

 Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

[Expand table](#)

Type	Description
Self	

schema

Python
<pre> schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) -> Dict[str, Any]</pre>

Parameters

[⌵](#) Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

[⌵](#) Expand table

Type	Description
Dict[str, Any]	

schema_json

Python
<pre> schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE, **kwargs: Any) -> str</pre>

Parameters

[⌵](#) Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

[Expand table](#)

Type	Description
str	

update_forward_refs

Python
update_forward_refs(**localns: Any) -> None

Parameters

[Expand table](#)

Name	Description
localns Required*	Any

Returns

[Expand table](#)

Type	Description
None	

validate

Python
<pre>validate(value: Any) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
value Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

 Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

app_name

Public app name.

Python
<code>field app_name: str [Required]</code>

app_version

Optional version string of the application.

Python
<code>field app_version: str None = None</code>

tenant_id

Optional tenant id (guid) of the user making the request.

Python
<code>field tenant_id: str None = None</code>

purview_app_location

Optional app location for policy evaluation.

Python

```
field purview_app_location: PurviewAppLocation | None = None
```

graph_base_uri

Base URI for Microsoft Graph.

Python

```
field graph_base_uri: str = 'https://graph.microsoft.com/v1.0/'
```

blocked_prompt_message

Custom message to return when a prompt is blocked by policy.

Python

```
field blocked_prompt_message: str = 'Prompt blocked by policy'
```

blocked_response_message

Custom message to return when a response is blocked by policy.

Python

```
field blocked_response_message: str = 'Response blocked by policy'
```

ignore_exceptions

If True, all Purview exceptions will be logged but not thrown in middleware.

Python

```
field ignore_exceptions: bool = False
```

ignore_payment_required

If True, 402 payment required errors will be logged but not thrown.

Python

```
field ignore_payment_required: bool = False
```

cache_ttl_seconds

Time to live for cache entries in seconds (default 14400 = 4 hours).

Python

```
field cache_ttl_seconds: int = 14400
```

max_cache_size_bytes

Maximum cache size in bytes (default 200MB).

Python

```
field max_cache_size_bytes: int = 209715200
```

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = ''
```


ollama Package

Classes

 Expand table

OllamaChatClient	Ollama Chat completion class. Initialize an Ollama Chat client.
OllamaSettings	Ollama settings. Initialize the settings class. JSON <pre>{ "title": "OllamaSettings", "description": "Ollama settings.", "type": "object", "properties": { "env_file_path": { "anyOf": [{ "type": "string" }, { "type": "null" }], "default": null, "title": "Env File Path" }, "env_file_encoding": { "anyOf": [{ "type": "string" }, { "type": "null" }], "default": "utf-8", "title": "Env File Encoding" }, "host": { "anyOf": [{ "type": "string" }] } } }</pre>

```
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Host"
},
"model_id": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Model Id"
}
}
```

OllamaChatClient Class

Ollama Chat completion class.

Initialize an Ollama Chat client.

Constructor

Python
<code>OllamaChatClient()</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>host</code>	Ollama server URL, if none <i>http://localhost:11434</i> is used. Can be set via the OLLAMA_HOST env variable. Default value: None
<code>client</code>	An optional Ollama Client instance. If not provided, a new instance will be created. Default value: None
<code>model_id</code>	The Ollama chat model ID to use. Can be set via the OLLAMA_MODEL_ID env variable. Default value: None
<code>env_file_path</code>	An optional path to a dotenv (.env) file to load environment variables from. Default value: None
<code>env_file_encoding</code>	The encoding to use when reading the dotenv (.env) file. Defaults to 'utf-8'. Default value: None
<code>**kwargs</code>	Additional keyword arguments passed to BaseChatClient.

Methods

[Expand table](#)

<code>__init__</code>	Initialize an Ollama Chat client.
<code>__new__</code>	

<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an Ollama Chat client.

Python

```
__init__(*, host: str | None = None, client: AsyncClient | None = None, model_id: str | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
host Required*	str None
client Required*	<xref:ollama._client.AsyncClient> None
model_id Required*	str None
env_file_path Required*	str None
env_file_encoding Required*	str None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
host	Ollama server URL, if none <i>http://localhost:11434</i> is used. Can be set via the OLLAMA_HOST env variable. Default value: None
client	An optional Ollama Client instance. If not provided, a new instance will be created. Default value: None
model_id	The Ollama chat model ID to use. Can be set via the OLLAMA_MODEL_ID env variable. Default value: None
env_file_path	An optional path to a dotenv (.env) file to load environment variables from.

Name	Description
	Default value: None
<code>env_file_encoding</code>	The encoding to use when reading the dotenv (.env) file. Defaults to 'utf-8'. Default value: None
<code>**kwargs</code>	Additional keyword arguments passed to BaseChatClient.

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`create_agent`

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python
<pre>create_agent(*, id: str None = None, name: str None = None, description: str None = None, instructions: str None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] None = None, context_providers: ContextProvider list[ContextProvider] AggregateContextProvider None = None, middleware: AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] None = None, allow_multiple_tool_calls: bool None = None, conversation_id: str None = None, frequency_penalty: float None = None,</pre>

```
logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None,
metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

 Expand table

Name	Description
id Required*	str None
name Required*	str None
description Required*	str None
instructions Required*	str None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]]] None
allow_multiple_tool_calls Required*	bool None

Name	Description
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options	dict[str, Any] None

Name	Description
Required*	
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None

Name	Description
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

 Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
    temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) -> str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```

```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

[Expand table](#)

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None

Name	Description
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
<code>allow_multiple_tool_calls</code>	Whether to allow multiple tool calls in a single response. Default value: None
<code>frequency_penalty</code>	The frequency penalty to use. Default value: None
<code>logit_bias</code>	The logit bias to use. Default value: None
<code>max_tokens</code>	The maximum number of tokens to generate. Default value: None
<code>metadata</code>	Additional metadata to include in the request. Default value: None
<code>model_id</code>	The <code>model_id</code> to use for the agent. Default value: None
<code>presence_penalty</code>	The presence penalty to use. Default value: None
<code>response_format</code>	The format of the response. Default value: None
<code>seed</code>	The random seed to use. Default value: None
<code>stop</code>	The stop sequence(s) for the request. Default value: None
<code>store</code>	Whether to store the response. Default value: None
<code>temperature</code>	The sampling temperature to use. Default value: None
<code>tool_choice</code>	The tool choice for the request. Default is <i>auto</i> . Default value: None
<code>tools</code>	The tools to use for the request. Default value: None
<code>top_p</code>	The nucleus sampling probability to use. Default value: None

Name	Description
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Name	Description
messages Required*	<code>str</code> <code>ChatMessage</code> <code>list[str]</code> <code>list[ChatMessage]</code> The message or messages to send to the model.
allow_multiple_tool_calls Required*	<code>bool</code> <code>None</code>
frequency_penalty Required*	<code>float</code> <code>None</code>
logit_bias Required*	<code>dict[str int, float]</code> <code>None</code>
max_tokens Required*	<code>int</code> <code>None</code>
metadata Required*	<code>dict[str, Any]</code> <code>None</code>
model_id Required*	<code>str</code> <code>None</code>
presence_penalty Required*	<code>float</code> <code>None</code>
response_format Required*	<code>type[<xref:pydantic.main.BaseModel>]</code> <code>None</code>
seed Required*	<code>int</code> <code>None</code>
stop Required*	<code>str</code> <code>Sequence[str]</code> <code>None</code>
store Required*	<code>bool</code> <code>None</code>
temperature Required*	<code>float</code> <code>None</code>
tool_choice Required*	<code>ToolMode</code> <code>Literal['auto', 'required', 'none']</code> <code>dict[str, typing.Any]</code> <code>None</code>
tools Required*	<code>ToolProtocol</code> <code>Callable[[...], Any]</code> <code>MutableMapping[str, Any]</code> <code>Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]]</code> <code>None</code>
top_p Required*	<code>float</code> <code>None</code>

Name	Description
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

[🔗](#) Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None

Name	Description
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

 Expand table

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python
<pre>service_url() -> str</pre>

Returns

 Expand table

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'ollama'
```

OllamaSettings Class

Ollama settings.

Initialize the settings class.

JSON

```
{
  "title": "OllamaSettings",
  "description": "Ollama settings.",
  "type": "object",
  "properties": {
    "env_file_path": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Env File Path"
    },
    "env_file_encoding": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": "utf-8",
      "title": "Env File Encoding"
    },
    "host": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Host"
    },
    "model_id": {
```

```

        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Model Id"
    }
}

```

Constructor

Python
pydantic settings OllamaSettings

Keyword-Only Parameters

Expand table

Name	Description
env_file_path	Default value: None
env_file_encoding	Default value: utf-8
host	Default value: None
model_id	Default value: None

Methods

Expand table

<code>__init__</code>	Initialize the settings class.
<code>__new__</code>	Override the new method to set the env_prefix.
<code>construct</code>	
<code>copy</code>	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <i>model_copy</i> instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied
= self.model_validate(data)
```

[dict](#)

[from_orm](#)

[json](#)

[model_construct](#)

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

[model_copy](#)

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.***dict***] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

[model_dump](#)

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

[model_dump_json](#)

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's *to_json* method.

[model_json_schema](#)

Generates a JSON schema for a model class.

[model_parametrized_name](#)

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

[model_post_init](#)

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires

	the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

`__init__`

Initialize the settings class.

Python

`__init__(**kwargs: Any) -> None`

Parameters

 Expand table

Name	Description
<code>kwargs</code> Required*	Any

Returns

[Expand table](#)

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

Python

```
static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings
```

Parameters

[Expand table](#)

Name	Description
cls Required*	type [<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_pydantic.TSettings>	

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> Default value: None
<code>values</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude:
AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None
= None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>include</code> Required*	<code><xref:AbstractSetIntStr> <xref:MappingIntStrAny> None</code> Optional set or mapping specifying which fields to include in the copied model.
<code>exclude</code>	<code><xref:AbstractSetIntStr> <xref:MappingIntStrAny> None</code>

Name	Description
Required*	Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	<code>Dict[str, Any] None</code> Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	<code>bool</code> If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
<code>Self</code>	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
exclude Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

[Expand table](#)

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
exclude Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined

Name	Description
<code>models_as_dict</code>	Default value: PydanticUndefined

Returns

 Expand table

Type	Description
<code>str</code>	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

Python
<pre>model_construct(_fields_set: set[str] None = None, **values: Any) -> Self</pre>

Parameters

 Expand table

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None
<code>values</code> Required*	<code>Any</code> Trusted or pre-validated data dictionary.

Returns

[Expand table](#)

Type	Description
Self	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [*dict*][object.dict] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

[Expand table](#)

Name	Description
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
mode Required*	Literal ['json', 'python'] str The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set [int] set [str] Mapping [int, set [int]] set [str] Mapping [int, IncEx] bool Mapping [str, IncEx] bool bool Mapping [str, set [int]] set [str] Mapping [int, IncEx] bool Mapping [str, IncEx] bool bool None A set of fields to include in the output.

Name	Description
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's `to_json` method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] |
```



```
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

[Expand table](#)

Name	Description
indent Required*	<code>int None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <code>None</code> .

Name	Description
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	bool Literal ['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	Callable [[Any], Any] None A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False

Name	Description
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

[Expand table](#)

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal ['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default).

Name	Description
	'<i>primitive_type_array</i>': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string</i>, <i>boolean</i>, <i>null</i>, <i>integer</i> or <i>number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	type [<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: <i>GenerateJsonSchema</i>
mode	Literal ['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
union_format	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
dict [str , Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python
<code>model_parametrized_name(params: tuple[type[Any], ...]) -> str</code>

Parameters

[Expand table](#)

Name	Description
params Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model[<i>str</i>, <i>int</i>]</i> , the value (<i>str</i> , <i>int</i>) would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

[Expand table](#)

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python
<pre>model_post_init(context: Any, /) -> None</pre>

Positional-Only Parameters

[Expand table](#)

Name	Description
context Required*	

Parameters

[Expand table](#)

Name	Description
context Required*	Any

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
    _parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
    None) -> bool | None
```

Parameters

[Expand table](#)

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	bool Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> None The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

[Expand table](#)

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	<code>Any</code> The object to validate.
strict Required*	<code>bool</code> <code>None</code> Whether to enforce types strictly.
extra	<code>Literal</code> ['allow', 'ignore', 'forbid'] <code>None</code>

Name	Description
Required*	Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool None Whether to extract data from object attributes.
context Required*	Any None Additional context to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
<code>strict</code>	Default value: None
<code>extra</code>	Default value: None
<code>context</code>	Default value: None
<code>by_alias</code>	Default value: None
<code>by_name</code>	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
<code>ValidationError</code>	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>obj</code> Required*	Any The object containing string data to validate.

Name	Description
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value] [pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

 Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
```

-> Self

Parameters

 Expand table

Name	Description
path Required*	str <xref:Path>
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

 Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None

Name	Description
<code>allow_pickle</code> Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>content_type</code>	Default value: None
<code>encoding</code>	Default value: 'utf8'
<code>proto</code>	Default value: None
<code>allow_pickle</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
<code>by_alias</code>	<code>bool</code> Default value: True
<code>ref_template</code>	<code>str</code> Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
<code>Dict[str, Any]</code>	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
by_alias Required*	<code>bool</code>
ref_template Required*	<code>str</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
str	

update_forward_refs

Python
update_forward_refs(**locals: Any) -> None

Parameters

[↗](#) Expand table

Name	Description
locals Required*	Any

Returns

[↗](#) Expand table

Type	Description
None	

validate

Python
validate(value: Any) -> Self

Parameters

[↗](#) Expand table

Name	Description
value Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

[Expand table](#)

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'OLLAMA_'
```

host

Python

```
field host: str | None = None
```

model_id

Python

```
field model_id: str | None = None
```

openai Package

Classes

 Expand table

OpenAIAssistantsClient	OpenAI Assistants client. Initialize an OpenAI Assistants client.
OpenAIChatClient	OpenAI Chat completion class. Initialize an OpenAI Chat completion client.
OpenAIContentFilterException	AI exception for an error from Azure OpenAI's content filter. Initializes a new instance of the ContentFilterAIException class.
OpenAIResponsesClient	OpenAI Responses client class. Initialize an OpenAI Responses client.
OpenAISettings	OpenAI environment settings. The settings are first loaded from environment variables with the prefix '>>OPENAI_<<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing. Initialize the settings class. JSON <pre>{ "title": "OpenAISettings", "description": "OpenAI environment settings.\n\nThe settings are first loaded from environment variables with the prefix 'OPENAI_'.\nIf the environment variables are not found, the settings can be loaded from a .env file with the\nencoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored;\nhowever, validation will fail alerting that the settings are missing.\n\nKeyword Args:\n api_key: OpenAI API key, see https://platform.openai.com/account/api-keys.\n Can be set via environment variable OPENAI_API_KEY.\n base_url: The base URL for the OpenAI API. Can be set via environment variable OPENAI_BASE_URL.\n org_id: This is usually optional unless your account</pre>

belongs to multiple organizations.\n Can be set via environment variable OPENAI_ORG_ID.\n

chat_model_id: The OpenAI chat model ID to use, for example, gpt-3.5-turbo or gpt-4.\n Can be set via environment variable OPENAI_CHAT_MODEL_ID.\n

responses_model_id: The OpenAI responses model ID to use, for example, gpt-4o or o1.\n Can be set via environment variable OPENAI_RESPONSES_MODEL_ID.\n

env_file_path: The path to the .env file to load settings from.\n

env_file_encoding: The encoding of the .env file, defaults to 'utf-8'.\n\nExamples:\n .. code-block:: python\n\n from agent_framework.openai import OpenAISettings\n\n # Using environment variables\n # Set OPENAI_API_KEY=sk-...\n # Set OPENAI_CHAT_MODEL_ID=gpt-4\n settings = OpenAISettings()\n\n # Or passing parameters directly\n settings = OpenAISettings(api_key=\"sk-...\", chat_model_id=\"gpt-4\")\n\n # Or loading from a .env file\n settings = OpenAISettings(env_file_path=\"path/to/.env\"),

```

    "type": "object",
    "properties": {
      "env_file_path": {
        "anyOf": [
          {
            "type": "string"
          },
          {
            "type": "null"
          }
        ],
        "default": null,
        "title": "Env File Path"
      },
      "env_file_encoding": {
        "anyOf": [
          {
            "type": "string"
          },
          {
            "type": "null"
          }
        ],
        "default": "utf-8",
        "title": "Env File Encoding"
      },
      "api_key": {
        "anyOf": [
          {
            "format": "password",
            "type": "string",
            "writeOnly": true
          },
          {

```

```
        "type": "null"
      }
    ],
    "default": null,
    "title": "Api Key"
  },
  "base_url": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Base Url"
  },
  "org_id": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Org Id"
  },
  "chat_model_id": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Chat Model Id"
  },
  "responses_model_id": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Responses Model Id"
  }
}
```

```
}  
}
```

Enums

 Expand table

ContentFilterResultSeverity	The severity of the content filter result.
---	--

ContentFilterResultSeverity Enum

The severity of the content filter result.

Fields

 Expand table

HIGH
LOW
MEDIUM
SAFE

OpenAIAssistantClient Class

OpenAI Assistants client.

Initialize an OpenAI Assistants client.

Constructor

Python
<code>OpenAIAssistantClient()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>model_id</code>	OpenAI model name, see https://platform.openai.com/docs/models . Can also be set via environment variable <code>OPENAI_CHAT_MODEL_ID</code> . Default value: None
<code>assistant_id</code>	The ID of an OpenAI assistant to use. If not provided, a new assistant will be created (and deleted after the request). Default value: None
<code>assistant_name</code>	The name to use when creating new assistants. Default value: None
<code>assistant_description</code>	The description to use when creating new assistants. Default value: None
<code>thread_id</code>	Default thread ID to use for conversations. Can be overridden by <code>conversation_id</code> property when making a request. If not provided, a new thread will be created (and deleted after the request). Default value: None
<code>api_key</code>	The API key to use. If provided will override the env vars or <code>.env</code> file value. Can also be set via environment variable <code>OPENAI_API_KEY</code> . Default value: None
<code>org_id</code>	The org ID to use. If provided will override the env vars or <code>.env</code> file value. Can also be set via environment variable <code>OPENAI_ORG_ID</code> . Default value: None
<code>base_url</code>	

Name	Description
	The base URL to use. If provided will override the standard value. Can also be set via environment variable OPENAI_BASE_URL. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables. Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None
kwargs	Other keyword parameters.

Examples

Python

```
from agent_framework.openai import OpenAIAssistantsClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
client = OpenAIAssistantsClient()

# Or passing parameters directly
client = OpenAIAssistantsClient(model_id="gpt-4", api_key="sk-...")

# Or loading from a .env file
client = OpenAIAssistantsClient(env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize an OpenAI Assistants client.
<code>__new__</code>	
<code>close</code>	Clean up any assistants we created.
<code>create_agent</code>	Create a ChatAgent with this client.

	This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an OpenAI Assistants client.

Python

```
__init__(*, model_id: str | None = None, assistant_id: str | None = None,
assistant_name: str | None = None, assistant_description: str | None = None,
thread_id: str | None = None, api_key: str | Callable[[], str | Awaitable[str]] |
None = None, org_id: str | None = None, base_url: str | None = None,
default_headers: Mapping[str, str] | None = None, async_client: AsyncOpenAI | None
= None, env_file_path: str | None = None, env_file_encoding: str | None = None,
**kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
model_id Required*	str None
assistant_id Required*	str None
assistant_name Required*	str None
assistant_description Required*	str None
thread_id Required*	str None
api_key Required*	str Callable[[], str Awaitable[str]] None
org_id Required*	str None
base_url Required*	str None
default_headers Required*	Mapping[str, str] None
async_client Required*	<xref:openai.AsyncOpenAI> None
env_file_path Required*	str None
env_file_encoding Required*	str None

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
model_id	OpenAI model name, see https://platform.openai.com/docs/models  . Can also be set via environment variable OPENAI_CHAT_MODEL_ID. Default value: None
assistant_id	The ID of an OpenAI assistant to use. If not provided, a new assistant will be created (and deleted after the request). Default value: None
assistant_name	The name to use when creating new assistants. Default value: None
assistant_description	The description to use when creating new assistants. Default value: None
thread_id	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. If not provided, a new thread will be created (and deleted after the request). Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_API_KEY. Default value: None
org_id	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_ORG_ID. Default value: None
base_url	The base URL to use. If provided will override the standard value. Can also be set via environment variable OPENAI_BASE_URL. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables.

Name	Description
	Default value: None
<code>env_file_encoding</code>	The encoding of the environment settings file. Default value: None
<code>kwargs</code>	Other keyword parameters.

Returns

 Expand table

Type	Description
<code>None</code>	

Examples

Python

```
from agent_framework.openai import OpenAIAssistantsClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
client = OpenAIAssistantsClient()

# Or passing parameters directly
client = OpenAIAssistantsClient(model_id="gpt-4", api_key="sk-...")

# Or loading from a .env file
client = OpenAIAssistantsClient(env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

close

Clean up any assistants we created.

Python

```
async close() -> None
```

Returns

 Expand table

Type	Description
None	

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Name	Description
id Required*	str None
name Required*	str None
description Required*	str None
instructions Required*	str None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata	dict[str, Any] None

Name	Description
Required*	
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
id	

Name	Description
	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the <code>chat_client</code> <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

[Expand table](#)

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient
```

```
# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
    temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

from_dict()

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any]</code> <code>None</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
```

```

client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext

```

```

from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value	

Name	Description
Required*	

Parameters

 Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

 Expand table

Name	Description
messages Required*	<code>str ChatMessage list[str] list[ChatMessage]</code> The message or messages to send to the model.
allow_multiple_tool_calls Required*	<code>bool None</code>
frequency_penalty Required*	<code>float None</code>
logit_bias Required*	<code>dict[str int, float] None</code>
max_tokens Required*	<code>int None</code>
metadata Required*	<code>dict[str, Any] None</code>
model_id Required*	<code>str None</code>

Name	Description
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None

Name	Description
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str |
Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None

Name	Description
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
<code>allow_multiple_tool_calls</code>	Whether to allow multiple tool calls in a single response. Default value: None
<code>frequency_penalty</code>	The frequency penalty to use. Default value: None
<code>logit_bias</code>	The logit bias to use. Default value: None
<code>max_tokens</code>	The maximum number of tokens to generate. Default value: None
<code>metadata</code>	Additional metadata to include in the request. Default value: None
<code>model_id</code>	The <code>model_id</code> to use for the agent. Default value: None
<code>presence_penalty</code>	The presence penalty to use. Default value: None
<code>response_format</code>	The format of the response. Default value: None
<code>seed</code>	The random seed to use. Default value: None
<code>stop</code>	The stop sequence(s) for the request. Default value: None
<code>store</code>	Whether to store the response. Default value: None
<code>temperature</code>	The sampling temperature to use. Default value: None
<code>tool_choice</code>	The tool choice for the request. Default is <i>auto</i> . Default value: None
<code>tools</code>	The tools to use for the request. Default value: None
<code>top_p</code>	The nucleus sampling probability to use. Default value: None

Name	Description
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python
<code>service_url() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'openai'
```

OpenAIChatClient Class

OpenAI Chat completion class.

Initialize an OpenAI Chat completion client.

Constructor

Python
<code>OpenAIChatClient()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>model_id</code>	OpenAI model name, see https://platform.openai.com/docs/models . Can also be set via environment variable <code>OPENAI_CHAT_MODEL_ID</code> . Default value: None
<code>api_key</code>	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable <code>OPENAI_API_KEY</code> . Default value: None
<code>org_id</code>	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable <code>OPENAI_ORG_ID</code> . Default value: None
<code>default_headers</code>	The default headers mapping of string keys to string values for HTTP requests. Default value: None
<code>async_client</code>	An existing client to use. Default value: None
<code>instruction_role</code>	The role to use for 'instruction' messages, for example, "system" or "developer". If not provided, the default is "system". Default value: None
<code>base_url</code>	The base URL to use. If provided will override the standard value for an OpenAI connector, the env vars or .env file value. Can also be set via environment variable <code>OPENAI_BASE_URL</code> . Default value: None
<code>env_file_path</code>	Use the environment settings file as a fallback to environment variables.

Name	Description
	Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None

Examples

Python

```
from agent_framework.openai import OpenAIChatClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
client = OpenAIChatClient()

# Or passing parameters directly
client = OpenAIChatClient(model_id="gpt-4", api_key="sk-...")

# Or loading from a .env file
client = OpenAIChatClient(env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize an OpenAI Chat completion client.
<code>__new__</code>	
<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string.

	This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return <code>None</code>.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an OpenAI Chat completion client.

Python

```
__init__(*, model_id: str | None = None, api_key: str | Callable[[], str | Awaitable[str]] | None = None, org_id: str | None = None, default_headers: Mapping[str, str] | None = None, async_client: AsyncOpenAI | None = None, instruction_role: str | None = None, base_url: str | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None) -> None
```

Parameters

 Expand table

Name	Description
model_id Required*	str None
api_key Required*	str Callable[[], str] Awaitable[str] None
org_id Required*	str None
default_headers Required*	Mapping[str, str] None
async_client Required*	<xref:openai.AsyncOpenAI> None
instruction_role Required*	str None
base_url Required*	str None
env_file_path Required*	str None
env_file_encoding Required*	str None

Keyword-Only Parameters

[Expand table](#)

Name	Description
model_id	OpenAI model name, see https://platform.openai.com/docs/models . Can also be set via environment variable OPENAI_CHAT_MODEL_ID. Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_API_KEY. Default value: None
org_id	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_ORG_ID. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None

Name	Description
<code>async_client</code>	An existing client to use. Default value: None
<code>instruction_role</code>	The role to use for 'instruction' messages, for example, "system" or "developer". If not provided, the default is "system". Default value: None
<code>base_url</code>	The base URL to use. If provided will override the standard value for an OpenAI connector, the env vars or .env file value. Can also be set via environment variable OPENAI_BASE_URL. Default value: None
<code>env_file_path</code>	Use the environment settings file as a fallback to environment variables. Default value: None
<code>env_file_encoding</code>	The encoding of the environment settings file. Default value: None

Returns

 Expand table

Type	Description
<code>None</code>	

Examples

Python

```
from agent_framework.openai import OpenAIChatClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
client = OpenAIChatClient()

# Or passing parameters directly
client = OpenAIChatClient(model_id="gpt-4", api_key="sk-...")

# Or loading from a .env file
client = OpenAIChatClient(env_file_path="path/to/.env")
```

`__new__`

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

 Expand table

Name	Description
id Required*	str None

Name	Description
name Required*	str None
description Required*	str None
instructions Required*	str None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None

Name	Description
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent.

Name	Description
	Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request.

Name	Description
	Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

 Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent()
```

```
name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
from_dict()

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```

```

    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}

```

```

context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

 Expand table

Name	Description
messages Required*	<code>str ChatMessage list[str] list[ChatMessage]</code> The message or messages to send to the model.
allow_multiple_tool_calls Required*	<code>bool None</code>
frequency_penalty Required*	<code>float None</code>
logit_bias Required*	<code>dict[str int, float] None</code>
max_tokens Required*	<code>int None</code>
metadata Required*	<code>dict[str, Any] None</code>
model_id Required*	<code>str None</code>
presence_penalty Required*	<code>float None</code>

Name	Description
response_format Required*	type [<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence [str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal ['auto', 'required', 'none'] dict [str , typing.Any] None
tools Required*	ToolProtocol Callable [... , Any] MutableMapping [str , Any] Sequence [ToolProtocol Callable [... , Any] MutableMapping [str , Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict [str , Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None

Name	Description
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |  
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,  
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None  
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,  
model_id: str | None = None, presence_penalty: float | None = None,  
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str |  
Sequence[str] | None = None, store: bool | None = None, temperature: float |  
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |  
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |  
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |  
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |  
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -  
> AsyncIterable[ChatResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None

Name	Description
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None

Name	Description
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python
<code>service_url() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

to_dict

Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.


```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

--

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'openai'
```

OpenAIContentFilterException Class

AI exception for an error from Azure OpenAI's content filter.

Initializes a new instance of the ContentFilterAIException class.

Constructor

Python
<code>OpenAIContentFilterException()</code>

Parameters

[Expand table](#)

Name	Description
<code>message</code> Required*	The error message.
<code>inner_exception</code> Required*	The inner exception.

Methods

[Expand table](#)

<code>__init__</code>	Initializes a new instance of the ContentFilterAIException class.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Initializes a new instance of the ContentFilterAIException class.

Python

```
__init__(message: str, inner_exception: BadRequestError) -> None
```

Parameters

[Expand table](#)

Name	Description
message Required*	str The error message.
inner_exception Required*	<xref:openai.BadRequestError> The inner exception.

Returns

[Expand table](#)

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.traceback to tb and return self.

Python

```
with_traceback()
```

Attributes

args

content_filter_code

Python

```
content_filter_code: ContentFilterCodes
```

content_filter_result

Python

```
content_filter_result: dict[str, ContentFilterResult]
```

param

Python

```
param: str | None
```

OpenAIResponsesClient Class

OpenAI Responses client class.

Initialize an OpenAI Responses client.

Constructor

Python
<code>OpenAIResponsesClient()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>model_id</code>	OpenAI model name, see https://platform.openai.com/docs/models . Can also be set via environment variable <code>OPENAI_RESPONSES_MODEL_ID</code> . Default value: None
<code>api_key</code>	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable <code>OPENAI_API_KEY</code> . Default value: None
<code>org_id</code>	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable <code>OPENAI_ORG_ID</code> . Default value: None
<code>base_url</code>	The base URL to use. If provided will override the standard value. Can also be set via environment variable <code>OPENAI_BASE_URL</code> . Default value: None
<code>default_headers</code>	The default headers mapping of string keys to string values for HTTP requests. Default value: None
<code>async_client</code>	An existing client to use. Default value: None
<code>instruction_role</code>	The role to use for 'instruction' messages, for example, "system" or "developer". If not provided, the default is "system". Default value: None
<code>env_file_path</code>	Use the environment settings file as a fallback to environment variables. Default value: None

Name	Description
env_file_encoding	The encoding of the environment settings file. Default value: None
kwargs	Other keyword parameters.

Examples

Python

```
from agent_framework.openai import OpenAIResponsesClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_RESPONSES_MODEL_ID=gpt-4o
client = OpenAIResponsesClient()

# Or passing parameters directly
client = OpenAIResponsesClient(model_id="gpt-4o", api_key="sk-...")

# Or loading from a .env file
client = OpenAIResponsesClient(env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize an OpenAI Responses client.
<code>__new__</code>	
<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection

	capabilities are available through the <code>dependencies</code> parameter.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an OpenAI Responses client.

Python

```
__init__(*, model_id: str | None = None, api_key: str | Callable[[], str] | Awaitable[str] | None = None, org_id: str | None = None, base_url: str | None = None, default_headers: Mapping[str, str] | None = None, async_client: AsyncOpenAI | None = None, instruction_role: str | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>model_id</code>	<code>str</code> <code>None</code>

Name	Description
Required*	
api_key Required*	str Callable[[], str] Awaitable[str] None
org_id Required*	str None
base_url Required*	str None
default_headers Required*	Mapping[str, str] None
async_client Required*	<xref:openai.AsyncOpenAI> None
instruction_role Required*	str None
env_file_path Required*	str None
env_file_encoding Required*	str None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
model_id	OpenAI model name, see https://platform.openai.com/docs/models ↗ . Can also be set via environment variable OPENAI_RESPONSES_MODEL_ID. Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_API_KEY. Default value: None
org_id	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_ORG_ID. Default value: None
base_url	The base URL to use. If provided will override the standard value. Can also be set via environment variable OPENAI_BASE_URL.

Name	Description
	Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, "system" or "developer". If not provided, the default is "system". Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables. Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None
kwargs	Other keyword parameters.

Returns

 Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.openai import OpenAIResponsesClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_RESPONSES_MODEL_ID=gpt-4o
client = OpenAIResponsesClient()

# Or passing parameters directly
client = OpenAIResponsesClient(model_id="gpt-4o", api_key="sk-...")

# Or loading from a .env file
client = OpenAIResponsesClient(env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

[Expand table](#)

Name	Description
id Required*	str None

Name	Description
name Required*	str None
description Required*	str None
instructions Required*	str None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None

Name	Description
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent.

Name	Description
	Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request.

Name	Description
	Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

 Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent()
```

```
name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
from_dict()

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```

```

    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}

```

```

context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

 Expand table

Name	Description
messages Required*	<code>str ChatMessage list[str] list[ChatMessage]</code> The message or messages to send to the model.
allow_multiple_tool_calls Required*	<code>bool None</code>
frequency_penalty Required*	<code>float None</code>
logit_bias Required*	<code>dict[str int, float] None</code>
max_tokens Required*	<code>int None</code>
metadata Required*	<code>dict[str, Any] None</code>
model_id Required*	<code>str None</code>
presence_penalty Required*	<code>float None</code>

Name	Description
response_format Required*	type [<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence [str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal ['auto', 'required', 'none'] dict [str , typing.Any] None
tools Required*	ToolProtocol Callable [... , Any] MutableMapping [str , Any] Sequence [ToolProtocol Callable [... , Any] MutableMapping [str , Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict [str , Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None

Name	Description
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |  
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,  
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None  
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,  
model_id: str | None = None, presence_penalty: float | None = None,  
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str |  
Sequence[str] | None = None, store: bool | None = None, temperature: float |  
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |  
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |  
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |  
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |  
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -  
> AsyncIterable[ChatResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None

Name	Description
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None

Name	Description
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
AsyncIterable [ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python
<code>service_url() -> str</code>

Returns

[Expand table](#)

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

--

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

FILE_SEARCH_MAX_RESULTS

Python

```
FILE_SEARCH_MAX_RESULTS: int = 50
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'openai'
```

OpenAISettings Class

OpenAI environment settings.

The settings are first loaded from environment variables with the prefix '>>OPENAI_<<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing.

Initialize the settings class.

JSON

```
{
  "title": "OpenAISettings",
  "description": "OpenAI environment settings.\n\nThe settings are first loaded from environment variables with the prefix 'OPENAI_'. If the environment variables are not found, the settings can be loaded from a .env file with the\nencoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored;\nhowever, validation will fail alerting that the settings are missing.\n\nKeyword Args:\napi_key: OpenAI API key, see https://platform.openai.com/account/api-keys.\nCan be set via environment variable OPENAI_API_KEY.\nbase_url: The base URL for the OpenAI API.\nCan be set via environment variable OPENAI_BASE_URL.\norg_id: This is usually optional unless your account belongs to multiple organizations.\nCan be set via environment variable OPENAI_ORG_ID.\nchat_model_id: The OpenAI chat model ID to use, for example, gpt-3.5-turbo or gpt-4.\nCan be set via environment variable OPENAI_CHAT_MODEL_ID.\nresponses_model_id: The OpenAI responses model ID to use, for example, gpt-4o or o1.\nCan be set via environment variable OPENAI_RESPONSES_MODEL_ID.\nenv_file_path: The path to the .env file to load settings from.\nenv_file_encoding: The encoding of the .env file, defaults to 'utf-8'.\n\nExamples:\n.. code-block:: python\nfrom agent_framework.openai import OpenAISettings\n\n# Using environment variables\n# Set OPENAI_API_KEY=sk-...\n# Set OPENAI_CHAT_MODEL_ID=gpt-4\nsettings = OpenAISettings()\n\n# Or passing parameters directly\nsettings = OpenAISettings(api_key=\"sk-...\", chat_model_id=\"gpt-4\")\n\n# Or loading from a .env file\nsettings = OpenAISettings(env_file_path=\"path/to/.env\"),
  "type": "object",
  "properties": {
    "env_file_path": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ]
    }
  }
},
```

```
    "default": null,
    "title": "Env File Path"
  },
  "env_file_encoding": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": "utf-8",
    "title": "Env File Encoding"
  },
  "api_key": {
    "anyOf": [
      {
        "format": "password",
        "type": "string",
        "writeOnly": true
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Api Key"
  },
  "base_url": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Base Url"
  },
  "org_id": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Org Id"
  },
  "chat_model_id": {
    "anyOf": [
```



```

        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Chat Model Id"
},
"responses_model_id": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Responses Model Id"
}
}
}

```

Constructor

Python

pydantic settings OpenAISettings

Keyword-Only Parameters

[Expand table](#)

Name	Description
api_key	OpenAI API key, see https://platform.openai.com/account/api-keys . Can be set via environment variable OPENAI_API_KEY. Default value: None
base_url	The base URL for the OpenAI API. Can be set via environment variable OPENAI_BASE_URL. Default value: None
org_id	This is usually optional unless your account belongs to multiple organizations. Can be set via environment variable OPENAI_ORG_ID. Default value: None

Name	Description
chat_model_id	The OpenAI chat model ID to use, for example, gpt-3.5-turbo or gpt-4. Can be set via environment variable OPENAI_CHAT_MODEL_ID. Default value: None
responses_model_id	The OpenAI responses model ID to use, for example, gpt-4o or o1. Can be set via environment variable OPENAI_RESPONSES_MODEL_ID. Default value: None
env_file_path	The path to the .env file to load settings from. Default value: None
env_file_encoding	The encoding of the .env file, defaults to 'utf-8'. Default value: utf-8

Examples

Python

```
from agent_framework.openai import OpenAISettings

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
settings = OpenAISettings()

# Or passing parameters directly
settings = OpenAISettings(api_key="sk-...", chat_model_id="gpt-4")

# Or loading from a .env file
settings = OpenAISettings(env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize the settings class.
<code>__new__</code>	Override the new method to set the env_prefix.
<code>construct</code>	
<code>copy</code>	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <i>model_copy</i> instead. If you need <i>include</i> or <i>exclude</i> , use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied
= self.model_validate(data)
```

dict

from_orm

json

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting **dict** and **pydantic_fields_set** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's **dict** and **pydantic_extra** fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [**dict**][object.dict] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's *to_json* method.

model_json_schema

Generates a JSON schema for a model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

model_post_init

Override this method to perform additional initialization after **init** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

`__init__`

Initialize the settings class.

Python
<code>__init__(**kwargs: Any) -> None</code>

Parameters

[Expand table](#)

Name	Description
<code>kwargs</code> Required*	Any

Returns

[Expand table](#)

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

Python

```
static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings
```

Parameters

[Expand table](#)

Name	Description
cls Required*	type [<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_pydantic.TSettings>	

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> Default value: None
<code>values</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude:
AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None
= None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>include</code> Required*	<code><xref:AbstractSetIntStr> <xref:MappingIntStrAny> None</code> Optional set or mapping specifying which fields to include in the copied model.
<code>exclude</code>	<code><xref:AbstractSetIntStr> <xref:MappingIntStrAny> None</code>

Name	Description
Required*	Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	<code>Dict[str, Any] None</code> Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	<code>bool</code> If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
<code>Self</code>	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
exclude Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

[Expand table](#)

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
exclude Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined

Name	Description
<code>models_as_dict</code>	Default value: PydanticUndefined

Returns

 Expand table

Type	Description
<code>str</code>	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

Python
<pre>model_construct(_fields_set: set[str] None = None, **values: Any) -> Self</pre>

Parameters

 Expand table

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None
<code>values</code> Required*	<code>Any</code> Trusted or pre-validated data dictionary.

Returns

[Expand table](#)

Type	Description
Self	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [*dict*][object.dict] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

[Expand table](#)

Name	Description
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
mode Required*	Literal ['json', 'python'] str The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set [int] set [str] Mapping [int, set [int]] set [str] Mapping [int, IncEx] Mapping [str, IncEx] bool Mapping [str, set [int]] set [str] Mapping [int, IncEx] Mapping [str, IncEx] bool None A set of fields to include in the output.

Name	Description
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's `to_json` method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] |
```

```
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

[Expand table](#)

Name	Description
indent Required*	<code>int None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <code>None</code> .

Name	Description
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	bool Literal ['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	Callable [[Any], Any] None A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False

Name	Description
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

[Expand table](#)

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal ['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default).

Name	Description
	'<i>primitive_type_array</i>': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string</i>, <i>boolean</i>, <i>null</i>, <i>integer</i> or <i>number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	type [<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: <i>GenerateJsonSchema</i>
mode	Literal ['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
union_format	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
dict [str , Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python
<code>model_parametrized_name(params: tuple[type[Any], ...]) -> str</code>

Parameters

[Expand table](#)

Name	Description
params Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model[<i>str</i>, <i>int</i>]</i> , the value (<i>str</i> , <i>int</i>) would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

[Expand table](#)

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python
<pre>model_post_init(context: Any, /) -> None</pre>

Positional-Only Parameters

[Expand table](#)

Name	Description
context Required*	

Parameters

[Expand table](#)

Name	Description
context Required*	Any

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
    _parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
    None) -> bool | None
```

Parameters

[Expand table](#)

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	bool Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> None The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

[Expand table](#)

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	<code>Any</code> The object to validate.
strict Required*	<code>bool</code> <code>None</code> Whether to enforce types strictly.
extra	<code>Literal</code> ['allow', 'ignore', 'forbid'] <code>None</code>

Name	Description
Required*	Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool None Whether to extract data from object attributes.
context Required*	Any None Additional context to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```

model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None =
None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any |
None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self

```

Parameters

 Expand table

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python
<pre> model_validate_strings(obj: Any, *, strict: bool None = None, extra: Literal['allow', 'ignore', 'forbid'] None = None, context: Any None = None, by_alias: bool None = None, by_name: bool None = None) -> Self </pre>

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object containing string data to validate.

Name	Description
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value] [pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

 Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
```

-> Self

Parameters

 Expand table

Name	Description
path Required*	str <xref:Path>
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

 Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None

Name	Description
<code>allow_pickle</code> Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>content_type</code>	Default value: None
<code>encoding</code>	Default value: 'utf8'
<code>proto</code>	Default value: None
<code>allow_pickle</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
<code>by_alias</code>	<code>bool</code> Default value: True
<code>ref_template</code>	<code>str</code> Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
<code>Dict[str, Any]</code>	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
by_alias Required*	<code>bool</code>
ref_template Required*	<code>str</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
str	

update_forward_refs

Python
<code>update_forward_refs(**locals: Any) -> None</code>

Parameters

[↗](#) Expand table

Name	Description
locals Required*	Any

Returns

[↗](#) Expand table

Type	Description
None	

validate

Python
<code>validate(value: Any) -> Self</code>

Parameters

[↗](#) Expand table

Name	Description
value Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

[Expand table](#)

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

api_key

Python

```
field api_key: SecretStr | Callable[[], str | Awaitable[str]] | None = None
```

base_url

Python

```
field base_url: str | None = None
```


chat_model_id

Python

```
field chat_model_id: str | None = None
```

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'OPENAI_'
```

org_id

Python

```
field org_id: str | None = None
```

responses_model_id

Python

```
field responses_model_id: str | None = None
```

redis Package

Classes

 Expand table

RedisChatMessageStore Redis-backed implementation of ChatMessageStoreProtocol using Redis Lists.

This implementation provides persistent, thread-safe chat message storage using Redis Lists. Messages are stored as JSON-serialized strings in chronological order, with each conversation thread isolated by a unique Redis key.

Key Features:

- **Persistent Storage:** Messages survive application restarts and crashes
- **Thread Isolation:** Each conversation thread has its own Redis key namespace
- **Auto Message Limits:** Configurable automatic trimming of old messages using LTRIM
- **Performance Optimized:** Uses native Redis operations for efficiency
- **State Serialization:** Full compatibility with Agent Framework thread serialization
- **Initial Message Support:** Pre-load conversations with existing message history
- **Production Ready:** Atomic operations, error handling, connection pooling

Redis Operations:

- **R PUSH:** Add messages to the end of the list (chronological order)
- **L RANGE:** Retrieve messages in chronological order
- **L TRIM:** Maintain message limits by trimming old messages
- **DELETE:** Clear all messages for a thread

Initialize the Redis chat message store.

Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.

param redis_url: Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with credential_provider.

param credential_provider: Redis credential provider (redis.credentials.CredentialProvider) for Azure AD authentication. Requires host parameter. Mutually exclusive with redis_url.

param host: Redis host name (e.g., "myredis.redis.cache.windows.net"). Required when using credential_provider.

param port: Redis port number. Defaults to 6380 (Azure Redis SSL port).

param ssl: Enable SSL/TLS connection. Defaults to True.

param username: Redis username. Defaults to None.

param thread_id: Unique identifier for this conversation thread. If not provided, a UUID will be auto-generated. This becomes part of the Redis key: {key_prefix}:{thread_id}

param key_prefix: Prefix for Redis keys to namespace different applications. Defaults to 'chat_messages'. Useful for multi-tenant scenarios.

param max_messages: Maximum number of messages to retain in Redis. When exceeded, oldest messages are automatically trimmed using LTRIM. None means unlimited storage.

param messages: Initial messages to pre-populate the conversation. These are added to Redis on first access if the Redis key is empty. Useful for resuming conversations or seeding with context.

raises ValueError: If neither redis_url nor credential_provider is provided.

raises ValueError: If both redis_url and credential_provider are provided.

raises ValueError: If credential_provider is used without host parameter.

Examples

```
Traditional connection: store = RedisChatMessageStore(  
    redis_url="redis://localhost:6379", thread_id="conversation_123"  
)
```

Azure Managed Redis with credential provider: from redis.credentials import
CredentialProvider from azure.identity.aio import DefaultAzureCredential

```
store = RedisChatMessageStore(  
    credential_provider=CredentialProvider(DefaultAzureCredential()),  
    host="myredis.redis.cache.windows.net", thread_id="conversation_123"  
)
```

```
init(redis_url: str | None = None, credential_provider: CredentialProvider | None =  
None, host: str | None = None, port: int = 6380, ssl: bool = True, username: str |  
None = None, thread_id: str | None = None, key_prefix: str = 'chat_messages',  
max_messages: int | None = None, messages: Sequence[ChatMessage] | None =  
None) -> None
```

Initialize the Redis chat message store.

Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.

Parameters: * **redis_url** ([str](#)* | *[None](#)) – Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with **credential_provider**.

```
* **credential_provider**  
(<xref:redis.credentials.CredentialProvider>* | *<xref:None>)  
– Redis credential provider  
(redis.credentials.CredentialProvider) for Azure AD  
authentication. Requires host parameter. Mutually exclusive  
with redis_url.  
  
* **host** (<xref:str>* | *<xref:None>) – Redis host name  
(e.g., "myredis.redis.cache.windows.net"). Required when using  
credential_provider.  
  
* **port** (<xref:int>) – Redis port number. Defaults to  
6380 (Azure Redis SSL port).  
  
* **ssl** (<xref:bool>) – Enable SSL/TLS connection.  
Defaults to True.  
  
* **username** (<xref:str>* | *<xref:None>) – Redis  
username. Defaults to None.  
  
* **thread_id** (<xref:str>* | *<xref:None>) – Unique  
identifier for this conversation thread. If not provided, a  
UUID will be auto-generated. This becomes part of the Redis  
key: {key_prefix}:{thread_id}  
  
* **key_prefix** (<xref:str>) – Prefix for Redis keys to  
namespace different applications. Defaults to 'chat_messages'.  
Useful for multi-tenant scenarios.  
  
* **max_messages** (<xref:int>* | *<xref:None>) – Maximum  
number of messages to retain in Redis. When exceeded, oldest  
messages are automatically trimmed using LTRIM. None means  
unlimited storage.  
  
* **messages** (<xref:collections.abc.Sequence>*[*  
<xref:agent_framework.ChatMessage>*] ** | *<xref:None>) –  
Initial messages to pre-populate the conversation. These are  
added to Redis on first access if the Redis key is empty.  
Useful for resuming conversations or seeding with context.
```

Raises: * [ValueError](#) – If neither **redis_url** nor **credential_provider** is provided.

```
* <xref:ValueError> - If both redis_url and
credential_provider are provided.

* <xref:ValueError> - If credential_provider is used without
host parameter.
```

Return type: [None](#)

Examples

Traditional connection: store = RedisChatMessageStore(
redis_url="redis://localhost:6379", thread_id="conversation_123"

```
)
```

Azure Managed Redis with credential provider: from redis.credentials import
CredentialProvider from azure.identity.aio import DefaultAzureCredential

```
store = RedisChatMessageStore(

credential_provider=CredentialProvider(DefaultAzureCredential(
)),
    host="myredis.redis.cache.windows.net",
    thread_id="conversation_123"

)
```

new(kwargs)**

async aclose() -> [None](#)

Close the Redis connection.

This method provides a clean way to close the underlying Redis connection when the store is no longer needed. This is particularly useful in samples and applications where explicit resource cleanup is desired.

Return type: [None](#)

async add_messages(messages: Sequence[ChatMessage]) -> [None](#)

Add messages to the Redis store (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for adding messages. Messages are appended to the Redis list in chronological order, with automatic trimming if message limits are configured.

Parameters: **messages** ([Sequence\[ChatMessage\]](#)) – Sequence of ChatMessage objects to add to the store. Can be empty (no-op) or contain multiple messages.

Return type: [None](#)

Thread Safety:

- Atomic pipeline ensures all messages are added together
- LTRIM operation is atomic for consistent message limits

Example

Python

```
messages = [ChatMessage(role="user", text="Hello"),
             ChatMessage(role="assistant", text="Hi there!")]
await store.add_messages(messages)
....
```

[async](#) `append(item: ChatMessage)` -> [<xref:None>](#)

Append a message to the end of the store.

Parameters:

****item**** ([<xref:agent_framework.ChatMessage>](#)) – The ChatMessage to append.

Return type:

[<xref:None>](#)

[async](#) `clear()` -> [<xref:None>](#)

Remove all messages [from](#) the store.

Permanently deletes all messages [for](#) this conversation thread by removing the Redis key. This operation cannot be undone.

Warning:

- This permanently deletes all conversation history
- Consider exporting messages before clearing [if](#) backup [is](#) needed

[#### Example](#)

```
<!-- literal_block {"ids": [], "classes": [], "names": [],
"dupnames": [], "backrefs": [], "xml:space": "preserve",
"force": false, "language": "python", "highlight_args": {}},
"linenos": false} -->
```

```

python

# Clear conversation history
await store.clear()

# Verify messages are gone
messages = await store.list_messages()
assert len(messages) == 0

```

Return type:

<xref:None>

`async count()` -> <xref:int>

Return the number of messages `in` the Redis store.

Returns:

The count of messages currently stored `in` Redis.

Return type:

<xref:int>

`async classmethod deserialize(serialized_store_state: Any, **kwargs: Any) -> <xref:agent_framework_redis._chat_message_store.RedisChatMessageStore>`

Deserialize state data into a new store instance (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol `for` state deserialization. Creates a new RedisChatMessageStore instance `from` previously serialized data, allowing the store to reconnect to the same conversation data `in` Redis.

Parameters:

* `**serialized_store_state**` (<xref:agent_framework.exceptions.Any>) - Previously serialized state data `from` `serialize_state()`. Should be a dictionary `with` `thread_id`, `redis_url`, etc.

* `**kwargs**` (<xref:agent_framework.exceptions.Any>) -

Keyword Arguments:

`****kwargs**` - Additional arguments passed to Pydantic model validation.

Returns:

A new RedisChatMessageStore instance configured `from` the

serialized state.

Raises:

<xref:ValueError> – If required fields are missing or invalid in the serialized state.

Return type:

<xref:agent_framework_redis._chat_message_store.RedisChatMessageStore>

`async` `extend(items: Sequence[ChatMessage]) -> <xref:None>`

Extend the store by appending all messages from the iterable.

Parameters:

`**items**` (<xref:collections.abc.Sequence>[*] * <xref:agent_framework.ChatMessage>[*]) – Sequence of ChatMessage objects to append.

Return type:

<xref:None>

`async` `getitem(index: int) -> <xref:agent_framework.ChatMessage>`

Get a message by index using Redis LINDEX.

Parameters:

`**index**` (<xref:int>) – The index of the message to retrieve.

Returns:

The ChatMessage at the specified index.

Raises:

<xref:IndexError> – If the index is out of range.

Return type:

<xref:agent_framework.ChatMessage>

`async` `index(item: ChatMessage) -> <xref:int>`

Return the index of the first occurrence of the specified message.

Uses Redis LINDEX to iterate through the list without loading all messages.

Still O(N) but more memory efficient for large lists.

Parameters:

`**item**` (<xref:agent_framework.ChatMessage>) – The

ChatMessage to find.

Returns:

The index of the first occurrence of the message.

Raises:

<xref:ValueError> - If the message **is not** found **in** the store.

Return type:

<xref:int>

```
async list_messages() -> <xref:list>
[<xref:agent_framework.ChatMessage>]
```

Get all messages **from** the store **in** chronological order (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol **for** retrieving messages.

Returns all messages stored **in** Redis, ordered **from** oldest (index 0) to newest (index -1).

Returns:

List of ChatMessage objects **in** chronological order (oldest first).

Returns empty list **if** no messages exist **or if** Redis connection fails.

Return type:

<xref:list>[<xref:agent_framework.ChatMessage>]

Example

```
<!-- literal_block {"ids": [], "classes": [], "names": [],
"dupnames": [], "backrefs": [], "xml:space": "preserve",
"force": false, "language": "python", "highlight_args": {}},
"linenos": false} -->
```

```
```python
```

```
 # Get all conversation history
 messages = await store.list_messages()
    ```
```

```
async ping() -> <xref:bool>
```

Test the Redis connection.

Returns:

True **if** the connection **is** successful, **False** otherwise.

Return type:
<xref:bool>

property redis_key: <xref:str>

Get the Redis key **for** this thread's messages.

The key format is: {key_prefix}:{thread_id}

Returns:

Redis key string used for storing this thread's messages.

```
#### ExampleFor key_prefix="chat_messages" and  
thread_id="user_123_session_456":  
Returns "chat_messages:user_123_session_456"
```

async remove(item: ChatMessage) -> <xref:None>

Remove the first occurrence of the specified message **from** the store.

Uses Redis LREM command **for** efficient removal by value.
O(N) but performed natively **in** Redis without data transfer.

Parameters:

****item**** (<xref:agent_framework.ChatMessage>) - The ChatMessage to remove.

Raises:

<xref:ValueError> - If the message **is not** found **in** the store.

Return type:

<xref:None>

async serialize(**kwargs: Any) ->
<xref:agent_framework.exceptions.Any>

Serialize the current store state **for** persistence
(ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol
protocol **for** state serialization.

Captures the Redis connection configuration **and** thread
information needed to
reconstruct the store **and** reconnect to the same conversation
data.

Keyword Arguments:

******kwargs**** - Additional arguments passed to Pydantic `model_dump()` for serialization.

Common options: `exclude_none=True`, `by_alias=True`

Returns:

Dictionary containing serialized store configuration that can be persisted to databases, files, or other storage mechanisms.

Parameters:

******kwargs**** (`<xref:agent_framework.exceptions.Any>`) -

Return type:

`<xref:agent_framework.exceptions.Any>`

async `setitem(index: int, item: ChatMessage) -> <xref:None>`

Set a message at the specified index using Redis LSET.

Parameters:

* ****index**** (`<xref:int>`) - The index at which to set the message.

* ****item**** (`<xref:agent_framework.ChatMessage>`) - The ChatMessage to set at the specified index.

Raises:

`<xref:IndexError>` - If the index is out of range.

Return type:

`<xref:None>`

async `update_from_state(serialized_store_state: Any, **kwargs: Any) -> <xref:None>`

Deserialize state data into this store instance (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for state deserialization.

Restores the store configuration from previously serialized data, allowing the store to reconnect to the same conversation data in Redis.

Parameters:

* ****serialized_store_state**** (`<xref:agent_framework.exceptions.Any>`) - Previously serialized state data from `serialize_state()`. Should be a dictionary with `thread_id`, `redis_url`, etc.

* ******kwargs**** (`<xref:agent_framework.exceptions.Any>`) -

Keyword Arguments:

****kwargs** – Additional arguments passed to Pydantic model validation.

Return type:
<xref:None>

RedisProvider

Redis context provider with dynamic, filterable schema.

Stores context in Redis and retrieves scoped context. Uses full-text or optional hybrid vector search to ground model responses.

Create a Redis Context Provider.

RedisChatMessageStore Class

Redis-backed implementation of ChatMessageStoreProtocol using Redis Lists.

This implementation provides persistent, thread-safe chat message storage using Redis Lists. Messages are stored as JSON-serialized strings in chronological order, with each conversation thread isolated by a unique Redis key.

Key Features:

- **Persistent Storage:** Messages survive application restarts and crashes
- **Thread Isolation:** Each conversation thread has its own Redis key namespace
- **Auto Message Limits:** Configurable automatic trimming of old messages using LTRIM
- **Performance Optimized:** Uses native Redis operations for efficiency
- **State Serialization:** Full compatibility with Agent Framework thread serialization
- **Initial Message Support:** Pre-load conversations with existing message history
- **Production Ready:** Atomic operations, error handling, connection pooling

Redis Operations:

- **R PUSH:** Add messages to the end of the list (chronological order)
- **L RANGE:** Retrieve messages in chronological order
- **L TRIM:** Maintain message limits by trimming old messages
- **DELETE:** Clear all messages for a thread

Initialize the Redis chat message store.

Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.

param `redis_url`: Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with `credential_provider`.

param `credential_provider`: Redis credential provider (`redis.credentials.CredentialProvider`) for Azure AD authentication. Requires `host` parameter. Mutually exclusive with `redis_url`.

param `host`: Redis host name (e.g., "myredis.redis.cache.windows.net"). Required when using `credential_provider`.

param `port`: Redis port number. Defaults to 6380 (Azure Redis SSL port).

param ssl: Enable SSL/TLS connection. Defaults to True.

param username: Redis username. Defaults to None.

param thread_id: Unique identifier for this conversation thread. If not provided, a UUID will be auto-generated. This becomes part of the Redis key: {key_prefix}:{thread_id}

param key_prefix: Prefix for Redis keys to namespace different applications. Defaults to 'chat_messages'. Useful for multi-tenant scenarios.

param max_messages: Maximum number of messages to retain in Redis. When exceeded, oldest messages are automatically trimmed using LTRIM. None means unlimited storage.

param messages: Initial messages to pre-populate the conversation. These are added to Redis on first access if the Redis key is empty. Useful for resuming conversations or seeding with context.

raises ValueError: If neither redis_url nor credential_provider is provided.

raises ValueError: If both redis_url and credential_provider are provided.

raises ValueError: If credential_provider is used without host parameter.

Examples

```
Traditional connection: store = RedisChatMessageStore( redis_url="redis://localhost:6379",
thread_id="conversation_123"
)
```

```
Azure Managed Redis with credential provider: from redis.credentials import CredentialProvider
from azure.identity.aio import DefaultAzureCredential
```

```
store = RedisChatMessageStore(
credential_provider=CredentialProvider(DefaultAzureCredential()),
host="myredis.redis.cache.windows.net", thread_id="conversation_123"
)
```

```
init(redis_url: str | None = None, credential_provider: CredentialProvider | None = None, host: str |
None = None, port: int = 6380, ssl: bool = True, username: str | None = None, thread_id: str |
None = None, key_prefix: str = 'chat_messages', max_messages: int | None = None, messages:
Sequence[ChatMessage] | None = None) -> None
```

Initialize the Redis chat message store.

Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.

Parameters: * **redis_url** ([str](#)* | *[None](#)) – Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with **credential_provider**.

```
* credential_provider (<xref:redis.credentials.CredentialProvider>* | *
<xref:None>) – Redis credential provider (redis.credentials.CredentialProvider) for
Azure AD authentication. Requires host parameter. Mutually exclusive with redis_url.

* host (<xref:str>* | *<xref:None>) – Redis host name (e.g.,
"myredis.redis.cache.windows.net"). Required when using credential_provider.

* port (<xref:int>) – Redis port number. Defaults to 6380 (Azure Redis SSL
port).

* ssl (<xref:bool>) – Enable SSL/TLS connection. Defaults to True.

* username (<xref:str>* | *<xref:None>) – Redis username. Defaults to None.

* thread_id (<xref:str>* | *<xref:None>) – Unique identifier for this
conversation thread. If not provided, a UUID will be auto-generated. This becomes part
of the Redis key: {key_prefix}:{thread_id}

* key_prefix (<xref:str>) – Prefix for Redis keys to namespace different
applications. Defaults to 'chat_messages'. Useful for multi-tenant scenarios.

* max_messages (<xref:int>* | *<xref:None>) – Maximum number of messages to
retain in Redis. When exceeded, oldest messages are automatically trimmed using LTRIM.
None means unlimited storage.

* messages (<xref:collections.abc.Sequence>*[*
<xref:agent_framework.ChatMessage>*] **| *<xref:None>) – Initial messages to pre-
populate the conversation. These are added to Redis on first access if the Redis key
is empty. Useful for resuming conversations or seeding with context.
```

Raises: * [ValueError](#) – If neither **redis_url** nor **credential_provider** is provided.

```
* <xref:ValueError> – If both redis_url and credential_provider are provided.

* <xref:ValueError> – If credential_provider is used without host parameter.
```

Return type: [None](#)

Examples

Traditional connection: `store = RedisChatMessageStore( redis_url="redis://localhost:6379", thread_id="conversation_123"`

```
)
```

Azure Managed Redis with credential provider: `from redis.credentials import CredentialProvider
from azure.identity.aio import DefaultAzureCredential`

```
store = RedisChatMessageStore(  
    credential_provider=CredentialProvider(DefaultAzureCredential()),  
    host="myredis.redis.cache.windows.net",  
    thread_id="conversation_123"  
)
```

`new(**kwargs)`

`async aclose()` -> [None](#)

Close the Redis connection.

This method provides a clean way to close the underlying Redis connection when the store is no longer needed. This is particularly useful in samples and applications where explicit resource cleanup is desired.

Return type: [None](#)

`async add_messages(messages: Sequence[ChatMessage])` -> [None](#)

Add messages to the Redis store (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for adding messages. Messages are appended to the Redis list in chronological order, with automatic trimming if message limits are configured.

Parameters: **messages** ([Sequence\[ChatMessage\]](#)) – Sequence of ChatMessage objects to add to the store. Can be empty (no-op) or contain multiple messages.

Return type: [None](#)

Thread Safety:

- Atomic pipeline ensures all messages are added together
- LTRIM operation is atomic for consistent message limits

Example

Python

```
messages = [ChatMessage(role="user", text="Hello"), ChatMessage(role="assistant",
text="Hi there!")]
await store.add_messages(messages)
````
```

`async` `append(item: ChatMessage) -> <xref:None>`

Append a message to the end of the store.

Parameters:

`**item**` (<xref:agent\_framework.ChatMessage>) – The ChatMessage to append.

Return type:

<xref:None>

`async` `clear()` -> <xref:None>

Remove all messages `from` the store.

Permanently deletes all messages `for` this conversation thread by removing the Redis key. This operation cannot be undone.

Warning:

- This permanently deletes all conversation history
- Consider exporting messages before clearing `if` backup `is` needed

### #### Example

```
<!-- literal_block {"ids": [], "classes": [], "names": [], "dupnames": [],
"backrefs": [], "xml:space": "preserve", "force": false, "language": "python",
"highlight_args": {}, "linenos": false} -->
```

```
```python
```

```
# Clear conversation history
await store.clear()
```

```
# Verify messages are gone
messages = await store.list_messages()
```

```
assert len(messages) == 0
....
```

Return type:
<xref:None>

`async` count() -> <xref:int>

Return the number of messages `in` the Redis store.

Returns:
The count of messages currently stored `in` Redis.

Return type:
<xref:int>

`async` classmethod deserialize(serialized_store_state: Any, **kwargs: Any) -> <xref:agent_framework_redis._chat_message_store.RedisChatMessageStore>

Deserialize state data into a new store instance (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol `for` state deserialization.

Creates a new RedisChatMessageStore instance `from` previously serialized data, allowing the store to reconnect to the same conversation data `in` Redis.

Parameters:
* `**serialized_store_state**` (<xref:agent_framework.exceptions.Any>) - Previously serialized state data `from` `serialize_state()`. Should be a dictionary `with` `thread_id`, `redis_url`, etc.

* `**kwargs**` (<xref:agent_framework.exceptions.Any>) -

Keyword Arguments:
`****kwargs**` - Additional arguments passed to Pydantic model validation.

Returns:
A new RedisChatMessageStore instance configured `from` the serialized state.

Raises:
<xref:ValueError> - If required fields are missing `or` invalid `in` the serialized state.

Return type:
<xref:agent_framework_redis._chat_message_store.RedisChatMessageStore>

`async` extend(items: Sequence[ChatMessage]) -> <xref:None>

Extend the store by appending all messages `from` the iterable.

Parameters:

****items**** (<xref:collections.abc.Sequence>[*<xref:agent_framework.ChatMessage>*]*)
- Sequence of ChatMessage objects to append.

Return type:
<xref:None>

async getitem(index: int) -> <xref:agent_framework.ChatMessage>

Get a message by index using Redis LINDEX.

Parameters:
****index**** (<xref:int>) - The index of the message to retrieve.

Returns:
The ChatMessage at the specified index.

Raises:
<xref:IndexError> - If the index **is** out of range.

Return type:
<xref:agent_framework.ChatMessage>

async index(item: ChatMessage) -> <xref:int>

Return the index of the first occurrence of the specified message.

Uses Redis LINDEX to iterate through the list without loading all messages.
Still O(N) but more memory efficient **for** large lists.

Parameters:
****item**** (<xref:agent_framework.ChatMessage>) - The ChatMessage to find.

Returns:
The index of the first occurrence of the message.

Raises:
<xref:ValueError> - If the message **is not** found **in** the store.

Return type:
<xref:int>

async list_messages() -> <xref:list>[<xref:agent_framework.ChatMessage>]

Get all messages **from** the store **in** chronological order (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol **for** retrieving messages.

Returns all messages stored **in** Redis, ordered **from** oldest (index 0) to newest (index -1).

Returns:

List of ChatMessage objects **in** chronological order (oldest first).
Returns empty list **if** no messages exist **or if** Redis connection fails.

Return type:

<xref:list>[<xref:agent_framework.ChatMessage>]

Example

```
<!-- literal_block {"ids": [], "classes": [], "names": [], "dupnames": [],  
"backrefs": [], "xml:space": "preserve", "force": false, "language": "python",  
"highlight_args": {}, "linenos": false} -->
```

```
```python
```

```
 # Get all conversation history
 messages = await store.list_messages()
    ```
```

async ping() -> <xref:bool>

Test the Redis connection.

Returns:

True **if** the connection **is** successful, **False** otherwise.

Return type:

<xref:bool>

property redis_key: <xref:str>

Get the Redis key **for** this thread's messages.

The key format is: {key_prefix}:{thread_id}

Returns:

Redis key string used for storing this thread's messages.

ExampleFor key_prefix="chat_messages" and thread_id="user_123_session_456":

Returns "chat_messages:user_123_session_456"

async remove(item: ChatMessage) -> <xref:None>

Remove the first occurrence of the specified message **from** the store.

Uses Redis LREM command **for** efficient removal by value.

O(N) but performed natively **in** Redis without data transfer.

Parameters:

****item**** (<xref:agent_framework.ChatMessage>) - The ChatMessage to remove.

Raises:

<xref:ValueError> - If the message **is not** found **in** the store.

Return type:

<xref:None>

async serialize(**kwargs: Any) -> <xref:agent_framework.exceptions.Any>

Serialize the current store state **for** persistence (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol **for** state serialization.

Captures the Redis connection configuration **and** thread information needed to reconstruct the store **and** reconnect to the same conversation data.

Keyword Arguments:

*****kwargs**** - Additional arguments passed to Pydantic model_dump() **for** serialization.

Common options: exclude_none=**True**, by_alias=**True**

Returns:

Dictionary containing serialized store configuration that can be persisted to databases, files, **or** other storage mechanisms.

Parameters:

****kwargs**** (<xref:agent_framework.exceptions.Any>) -

Return type:

<xref:agent_framework.exceptions.Any>

async setitem(index: int, item: ChatMessage) -> <xref:None>

Set a message at the specified index using Redis LSET.

Parameters:

* ****index**** (<xref:int>) - The index at which to set the message.

* ****item**** (<xref:agent_framework.ChatMessage>) - The ChatMessage to set at the specified index.

Raises:

<xref:IndexError> - If the index **is** out of range.

Return type:

<xref:None>

async update_from_state(serialized_store_state: Any, **kwargs: Any) -> <xref:None>

Deserialize state data into this store instance (ChatMessageStoreProtocol protocol

method).

This method implements the required ChatMessageStoreProtocol protocol [for](#) state deserialization.

Restores the store configuration [from](#) previously serialized data, allowing the store to reconnect to the same conversation data [in](#) Redis.

Parameters:

*** **serialized_store_state**** ([<xref:agent_framework.exceptions.Any>](#)) - Previously serialized state data [from](#) `serialize_state()`. Should be a dictionary [with](#) `thread_id`, `redis_url`, etc.

*** **kwargs**** ([<xref:agent_framework.exceptions.Any>](#)) -

Keyword Arguments:

******kwargs**** - Additional arguments passed to Pydantic model validation.

Return type:

[<xref:None>](#)

Constructor

Python

RedisChatMessageStore()

Parameters

[Expand table](#)

Name	Description
redis_url	Default value: None
credential_provider	Default value: None
host	Default value: None
port	Default value: 6380
ssl	Default value: True
username	Default value: None
thread_id	Default value: None
key_prefix	Default value: chat_messages
max_messages	Default value: None

Name	Description
messages	Default value: None

Methods

 Expand table

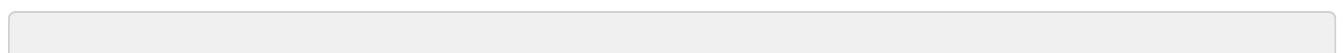
<code>__init__</code>	Initialize the Redis chat message store. Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.
<code>__new__</code>	
<code>aclose</code>	Close the Redis connection. This method provides a clean way to close the underlying Redis connection when the store is no longer needed. This is particularly useful in samples and applications where explicit resource cleanup is desired.
<code>add_messages</code>	Add messages to the Redis store (ChatMessageStoreProtocol protocol method). This method implements the required ChatMessageStoreProtocol protocol for adding messages. Messages are appended to the Redis list in chronological order, with automatic trimming if message limits are configured. Thread Safety: • Atomic pipeline ensures all messages are added together • LTRIM operation is atomic for consistent message limits
<code>append</code>	Append a message to the end of the store.
<code>clear</code>	Remove all messages from the store. Permanently deletes all messages for this conversation thread by removing the Redis key. This operation cannot be undone. Warning: • This permanently deletes all conversation history • Consider exporting messages before clearing if backup is needed
<code>count</code>	Return the number of messages in the Redis store.
<code>deserialize</code>	Deserialize state data into a new store instance (ChatMessageStoreProtocol protocol method).

	This method implements the required ChatMessageStoreProtocol protocol for state deserialization. Creates a new RedisChatMessageStore instance from previously serialized data, allowing the store to reconnect to the same conversation data in Redis.
<code>extend</code>	Extend the store by appending all messages from the iterable.
<code>getitem</code>	Get a message by index using Redis LINDEX.
<code>index</code>	Return the index of the first occurrence of the specified message. Uses Redis LINDEX to iterate through the list without loading all messages. Still O(N) but more memory efficient for large lists.
<code>list_messages</code>	Get all messages from the store in chronological order (ChatMessageStoreProtocol protocol method). This method implements the required ChatMessageStoreProtocol protocol for retrieving messages. Returns all messages stored in Redis, ordered from oldest (index 0) to newest (index -1).
<code>ping</code>	Test the Redis connection.
<code>remove</code>	Remove the first occurrence of the specified message from the store. Uses Redis LREM command for efficient removal by value. O(N) but performed natively in Redis without data transfer.
<code>serialize</code>	Serialize the current store state for persistence (ChatMessageStoreProtocol protocol method). This method implements the required ChatMessageStoreProtocol protocol for state serialization. Captures the Redis connection configuration and thread information needed to reconstruct the store and reconnect to the same conversation data.
<code>setitem</code>	Set a message at the specified index using Redis LSET.
<code>update_from_state</code>	Deserialize state data into this store instance (ChatMessageStoreProtocol protocol method). This method implements the required ChatMessageStoreProtocol protocol for state deserialization. Restores the store configuration from previously serialized data, allowing the store to reconnect to the same conversation data in Redis.

`__init__`

Initialize the Redis chat message store.

Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.



Python

```
__init__(redis_url: str | None = None, credential_provider: CredentialProvider |
None = None, host: str | None = None, port: int = 6380, ssl: bool = True,
username: str | None = None, thread_id: str | None = None, key_prefix: str =
'chat_messages', max_messages: int | None = None, messages: Sequence[ChatMessage]
| None = None) -> None
```

Parameters

 Expand table

Name	Description
redis_url	str None Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with credential_provider. Default value: None
credential_provider	<xref:redis.credentials.CredentialProvider> None Redis credential provider (redis.credentials.CredentialProvider) for Azure AD authentication. Requires host parameter. Mutually exclusive with redis_url. Default value: None
host	str None Redis host name (e.g., "myredis.redis.cache.windows.net"). Required when using credential_provider. Default value: None
port	int Redis port number. Defaults to 6380 (Azure Redis SSL port). Default value: 6380
ssl	bool Enable SSL/TLS connection. Defaults to True. Default value: True
username	str None Redis username. Defaults to None. Default value: None
thread_id	str None Unique identifier for this conversation thread. If not provided, a UUID will be auto-generated. This becomes part of the Redis key: {key_prefix}:{thread_id} Default value: None
key_prefix	str Prefix for Redis keys to namespace different applications. Defaults to 'chat_messages'. Useful for multi-tenant scenarios. Default value: "chat_messages"

Name	Description
<code>max_messages</code>	<code>int</code> <code>None</code> Maximum number of messages to retain in Redis. When exceeded, oldest messages are automatically trimmed using LTRIM. None means unlimited storage. Default value: None
<code>messages</code>	<code>Sequence[ChatMessage]</code> <code>None</code> Initial messages to pre-populate the conversation. These are added to Redis on first access if the Redis key is empty. Useful for resuming conversations or seeding with context. Default value: None

Returns

 Expand table

Type	Description
<code>None</code>	

Exceptions

 Expand table

Type	Description
<code>ValueError</code>	If neither <code>redis_url</code> nor <code>credential_provider</code> is provided.
<code>ValueError</code>	If both <code>redis_url</code> and <code>credential_provider</code> are provided.
<code>ValueError</code>	If <code>credential_provider</code> is used without host parameter.

Examples

```
Traditional connection: store = RedisChatMessageStore( redis_url="redis://localhost:6379",
thread_id="conversation_123"

)
```

```
Azure Managed Redis with credential provider: from redis.credentials import
CredentialProvider from azure.identity.aio import DefaultAzureCredential

store = RedisChatMessageStore(
credential_provider=CredentialProvider(DefaultAzureCredential()),
```

```
host="myredis.redis.cache.windows.net", thread_id="conversation_123"
```

```
)
```

__new__

Python

```
__new__(**kwargs)
```

aclose

Close the Redis connection.

This method provides a clean way to close the underlying Redis connection when the store is no longer needed. This is particularly useful in samples and applications where explicit resource cleanup is desired.

Python

```
async aclose() -> None
```

Returns

[Expand table](#)

Type	Description
None	

add_messages

Add messages to the Redis store (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for adding messages. Messages are appended to the Redis list in chronological order, with automatic trimming if message limits are configured.

Thread Safety:

- Atomic pipeline ensures all messages are added together
- LTRIM operation is atomic for consistent message limits

Python

```
async add_messages(messages: Sequence[ChatMessage]) -> None
```

Parameters

[Expand table](#)

Name	Description
messages Required*	Sequence[ChatMessage] Sequence of ChatMessage objects to add to the store. Can be empty (no-op) or contain multiple messages.

Returns

[Expand table](#)

Type	Description
None	

Examples

Python

```
messages = [ChatMessage(role="user", text="Hello"),  
ChatMessage(role="assistant", text="Hi there!")]  
await store.add_messages(messages)
```

append

Append a message to the end of the store.

Python

```
async append(item: ChatMessage) -> None
```

Parameters

[Expand table](#)

Name	Description
item Required*	ChatMessage The ChatMessage to append.

Returns

 Expand table

Type	Description
None	

clear

Remove all messages from the store.

Permanently deletes all messages for this conversation thread by removing the Redis key. This operation cannot be undone.

Warning:

- This permanently deletes all conversation history
- Consider exporting messages before clearing if backup is needed

Python
<code>async clear() -> None</code>

Returns

 Expand table

Type	Description
None	

Examples

Python
<pre># Clear conversation history await store.clear()</pre>

```
# Verify messages are gone
messages = await store.list_messages()
assert len(messages) == 0
```

count

Return the number of messages in the Redis store.

Python

```
async count() -> int
```

Returns

[Expand table](#)

Type	Description
int	The count of messages currently stored in Redis.

deserialize

Deserialize state data into a new store instance (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for state deserialization. Creates a new RedisChatMessageStore instance from previously serialized data, allowing the store to reconnect to the same conversation data in Redis.

Python

```
async classmethod deserialize(serialized_store_state: Any, **kwargs: Any) ->
RedisChatMessageStore
```

Parameters

[Expand table](#)

Name	Description
serialized_store_state Required*	Any Previously serialized state data from serialize_state(). Should be a dictionary with thread_id, redis_url, etc.
kwargs	Any

Name	Description
Required*	

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional arguments passed to Pydantic model validation.

Returns

[Expand table](#)

Type	Description
RedisChatMessageStore	A new RedisChatMessageStore instance configured from the serialized state.

Exceptions

[Expand table](#)

Type	Description
ValueError	If required fields are missing or invalid in the serialized state.

extend

Extend the store by appending all messages from the iterable.

Python
<code>async extend(items: Sequence[ChatMessage]) -> None</code>

Parameters

[Expand table](#)

Name	Description
items Required*	Sequence[ChatMessage] Sequence of ChatMessage objects to append.

Returns

[Expand table](#)

Type	Description
None	

getitem

Get a message by index using Redis LINDEX.

Python

```
async getitem(index: int) -> ChatMessage
```

Parameters

[Expand table](#)

Name	Description
index Required*	int The index of the message to retrieve.

Returns

[Expand table](#)

Type	Description
ChatMessage	The ChatMessage at the specified index.

Exceptions

[Expand table](#)

Type	Description
IndexError	If the index is out of range.

index

Return the index of the first occurrence of the specified message.

Uses Redis LINDEX to iterate through the list without loading all messages. Still O(N) but more memory efficient for large lists.

Python

```
async index(item: ChatMessage) -> int
```

Parameters

[Expand table](#)

Name	Description
item Required*	ChatMessage The ChatMessage to find.

Returns

[Expand table](#)

Type	Description
int	The index of the first occurrence of the message.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the message is not found in the store.

list_messages

Get all messages from the store in chronological order (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for retrieving messages. Returns all messages stored in Redis, ordered from oldest (index 0) to newest (index -1).

Python

```
async list_messages() -> list[ChatMessage]
```

Returns

[Expand table](#)

Type	Description
<code>list[ChatMessage]</code>	List of ChatMessage objects in chronological order (oldest first). Returns empty list if no messages exist or if Redis connection fails.

Examples

Python

```
# Get all conversation history
messages = await store.list_messages()
```

ping

Test the Redis connection.

Python

```
async ping() -> bool
```

Returns

[Expand table](#)

Type	Description
<code>bool</code>	True if the connection is successful, False otherwise.

remove

Remove the first occurrence of the specified message from the store.

Uses Redis LREM command for efficient removal by value. O(N) but performed natively in Redis without data transfer.

Python

```
async remove(item: ChatMessage) -> None
```

Parameters

[Expand table](#)

Name	Description
item Required*	ChatMessage The ChatMessage to remove.

Returns

[Expand table](#)

Type	Description
None	

Exceptions

[Expand table](#)

Type	Description
ValueError	If the message is not found in the store.

serialize

Serialize the current store state for persistence (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for state serialization. Captures the Redis connection configuration and thread information needed to reconstruct the store and reconnect to the same conversation data.

Python

```
async serialize(**kwargs: Any) -> Any
```

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional arguments passed to Pydantic model_dump() for serialization. Common options: exclude_none=True, by_alias=True

Returns

[Expand table](#)

Type	Description
Any	Dictionary containing serialized store configuration that can be persisted to databases, files, or other storage mechanisms.

setitem

Set a message at the specified index using Redis LSET.

Python
<pre>async setitem(index: int, item: ChatMessage) -> None</pre>

Parameters

[Expand table](#)

Name	Description
index Required*	int The index at which to set the message.
item Required*	ChatMessage The ChatMessage to set at the specified index.

Returns

[Expand table](#)

Type	Description
None	

Exceptions

[Expand table](#)

Type	Description
IndexError	If the index is out of range.

update_from_state

Deserialize state data into this store instance (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for state deserialization. Restores the store configuration from previously serialized data, allowing the store to reconnect to the same conversation data in Redis.

Python

```
async update_from_state(serialized_store_state: Any, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
serialized_store_state Required*	Any Previously serialized state data from <code>serialize_state()</code> . Should be a dictionary with <code>thread_id</code> , <code>redis_url</code> , etc.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional arguments passed to Pydantic model validation.

Returns

 Expand table

Type	Description
None	

Attributes

redis_key

Get the Redis key for this thread's messages.

The key format is: {key_prefix}:{thread_id}

Returns

 Expand table

Type	Description
	Redis key string used for storing this thread's messages.

Examples

For key_prefix="chat_messages" and thread_id="user_123_session_456": Returns "chat_messages:user_123_session_456"

RedisProvider Class

Redis context provider with dynamic, filterable schema.

Stores context in Redis and retrieves scoped context. Uses full-text or optional hybrid vector search to ground model responses.

Create a Redis Context Provider.

Constructor

Python
<code>RedisProvider()</code>

Parameters

 Expand table

Name	Description
<code>redis_url</code>	The Redis server URL. Default value: redis://localhost:6379
<code>index_name</code>	The name of the Redis index. Default value: context
<code>prefix</code>	The prefix for all keys in the Redis database. Default value: context
<code>redis_vectorizer</code>	The vectorizer to use for Redis. Default value: None
<code>vector_field_name</code>	The name of the vector field in Redis. Default value: None
<code>vector_algorithm</code>	The algorithm to use for vector search. Default value: None
<code>vector_distance_metric</code>	The distance metric to use for vector search. Default value: None
<code>application_id</code>	The application ID to scope the context. Default value: None

Name	Description
agent_id	The agent ID to scope the context. Default value: None
user_id	The user ID to scope the context. Default value: None
thread_id	The thread ID to scope the context. Default value: None
scope_to_per_operation_thread_id	Whether to scope to the per-operation thread ID. Default value: False
context_prompt	The context prompt to use for the provider. Default value: ## Memories Consider the following memories when answering user questions:
redis_index	The Redis index to use for the provider. Default value: None
overwrite_index	Whether to overwrite the existing Redis index. Default value: False

Methods

 Expand table

<code>__init__</code>	Create a Redis Context Provider.
<code>__new__</code>	
<code>invoked</code>	
<code>invoking</code>	Called before invoking the model to provide scoped context. Concatenates recent messages into a query, fetches matching memories from Redis. Prepends them as instructions.
<code>search_all</code>	Returns all documents in the index. Streams results via pagination to avoid excessive memory and response sizes.
<code>thread_created</code>	Called when a new thread is created. Captures the per-operation thread id when scoping is enabled to enforce single-thread usage.

`__init__`

Create a Redis Context Provider.

Python

```
__init__(redis_url: str = 'redis://localhost:6379', index_name: str = 'context',
prefix: str = 'context', redis_vectorizer: BaseVectorizer | None = None,
vector_field_name: str | None = None, vector_algorithm: Literal['flat', 'hnsw'] |
None = None, vector_distance_metric: Literal['cosine', 'ip', 'l2'] | None = None,
application_id: str | None = None, agent_id: str | None = None, user_id: str |
None = None, thread_id: str | None = None, scope_to_per_operation_thread_id: bool
= False, context_prompt: str = ContextProvider.DEFAULT_CONTEXT_PROMPT,
redis_index: Any = None, overwrite_index: bool = False)
```

Parameters

[Expand table](#)

Name	Description
<code>redis_url</code>	str The Redis server URL. Default value: "redis://localhost:6379"
<code>index_name</code>	str The name of the Redis index. Default value: "context"
<code>prefix</code>	str The prefix for all keys in the Redis database. Default value: "context"
<code>redis_vectorizer</code>	<xref:redisvl.utils.vectorize.base.BaseVectorizer> None The vectorizer to use for Redis. Default value: None
<code>vector_field_name</code>	str None The name of the vector field in Redis. Default value: None
<code>vector_algorithm</code>	Literal ['flat', 'hnsw'] None The algorithm to use for vector search. Default value: None
<code>vector_distance_metric</code>	Literal ['cosine', 'ip', 'l2'] None The distance metric to use for vector search. Default value: None
<code>application_id</code>	str None The application ID to scope the context. Default value: None

Name	Description
agent_id	str None The agent ID to scope the context. Default value: None
user_id	str None The user ID to scope the context. Default value: None
thread_id	str None The thread ID to scope the context. Default value: None
scope_to_per_operation_thread_id	bool Whether to scope to the per-operation thread ID. Default value: False
context_prompt	str The context prompt to use for the provider. Default value: ContextProvider.DEFAULT_CONTEXT_PROMPT
redis_index	Any The Redis index to use for the provider. Default value: None
overwrite_index	bool Whether to overwrite the existing Redis index. Default value: False

__new__

Python
<code>__new__(**kwargs)</code>

invoked

Python
<code>async invoked(request_messages: ChatMessage Sequence[ChatMessage], response_messages: ChatMessage Sequence[ChatMessage] None = None, invoke_exception: Exception None = None, **kwargs: Any) -> None</code>

Parameters

Name	Description
request_messages Required*	ChatMessage Sequence[ChatMessage]
response_messages	ChatMessage Sequence[ChatMessage] None Default value: None
invoke_exception	Exception None Default value: None
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
None	

invoking

Called before invoking the model to provide scoped context.

Concatenates recent messages into a query, fetches matching memories from Redis. Prepends them as instructions.

Python
<pre>async invoking(messages: ChatMessage MutableSequence[ChatMessage], **kwargs: Any) -> Context</pre>

Parameters

[Expand table](#)

Name	Description
messages Required*	ChatMessage MutableSequence[ChatMessage] List of new messages in the thread.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	not used at present at present.

Returns

[Expand table](#)

Type	Description
Context	Context object containing instructions with memories.

search_all

Returns all documents in the index.

Streams results via pagination to avoid excessive memory and response sizes.

Python

```
async search_all(page_size: int = 200) -> list[dict[str, Any]]
```

Parameters

[Expand table](#)

Name	Description
page_size	int Page size used for pagination under the hood. Default value: 200

Returns

[Expand table](#)

Type	Description
list[dict[str, Any]]	List of all documents.

thread_created

Called when a new thread is created.

Captures the per-operation thread id when scoping is enabled to enforce single-thread usage.

Python

```
async thread_created(thread_id: str | None) -> None
```

Parameters

 Expand table

Name	Description
thread_id Required*	<code>str None</code> The ID of the thread or None.

Returns

 Expand table

Type	Description
<code>None</code>	

Attributes

schema_dict

Get the Redis schema dictionary, computing and caching it on first access.

DEFAULT_CONTEXT_PROMPT

Python

```
DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories when answering user questions:'
```

exceptions Module

Classes

 Expand table

AdditionItemMismatch	An error occurred while adding two types. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentException	Base class for all agent exceptions. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentExecutionException	An error occurred while executing the agent. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentFrameworkException	Base exceptions for the Agent Framework. Automatically logs the message as debug. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentInitializationError	An error occurred while initializing the agent. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentThreadException	An error occurred while managing the agent thread. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.

Any	Special type indicating an unconstrained type. - Any is compatible with every type. - Any assumed to have all methods. - All values assumed to be instances of Any. Note that all the above statements are true from the point of view of static type checkers. At runtime, Any should not be used with instance checks.
ChatClientException	An error occurred while dealing with a chat client. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ChatClientInitializationError	An error occurred while initializing the chat client. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ContentError	An error occurred while processing content. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
MiddlewareException	An error occurred during middleware execution. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceContentFilterException	An error was raised by the content filter of the service. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceException	Base class for all service exceptions. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceInitializationError	An error occurred while initializing the service.

	Create an <code>AgentFrameworkException</code>. This emits a debug log (by default), with the <code>inner_exception</code> if provided.
<code>ServiceInvalidAuthError</code>	An error occurred while authenticating the service. Create an <code>AgentFrameworkException</code>. This emits a debug log (by default), with the <code>inner_exception</code> if provided.
<code>ServiceInvalidExecutionSettingsError</code>	An error occurred while validating the execution settings of the service. Create an <code>AgentFrameworkException</code>. This emits a debug log (by default), with the <code>inner_exception</code> if provided.
<code>ServiceInvalidRequestError</code>	An error occurred while validating the request to the service. Create an <code>AgentFrameworkException</code>. This emits a debug log (by default), with the <code>inner_exception</code> if provided.
<code>ServiceInvalidResponseError</code>	An error occurred while validating the response from the service. Create an <code>AgentFrameworkException</code>. This emits a debug log (by default), with the <code>inner_exception</code> if provided.
<code>ServiceResponseException</code>	Base class for all service response exceptions. Create an <code>AgentFrameworkException</code>. This emits a debug log (by default), with the <code>inner_exception</code> if provided.
<code>ToolException</code>	An error occurred while executing a tool. Create an <code>AgentFrameworkException</code>. This emits a debug log (by default), with the <code>inner_exception</code> if provided.
<code>ToolExecutionException</code>	An error occurred while executing a tool. Create an <code>AgentFrameworkException</code>. This emits a debug log (by default), with the <code>inner_exception</code> if provided.

AdditionItemMismatch Class

An error occurred while adding two types.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
AdditionItemMismatch()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

AgentException Class

Base class for all agent exceptions.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
AgentException()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	<code>str</code>
inner_exception	<code>Exception None</code> Default value: <code>None</code>
log_level	<code>Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None</code> Default value: <code>10</code>
args Required*	<code>Any</code>
kwargs Required*	<code>Any</code>

`__new__`

Python

```
__new__(**kwargs)
```

`add_note`

`Exception.add_note(note)` – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

AgentExecutionException Class

An error occurred while executing the agent.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
AgentExecutionException()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

AgentFrameworkException Class

Base exceptions for the Agent Framework.

Automatically logs the message as debug.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
AgentFrameworkException()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	<code>str</code>
inner_exception	<code>Exception None</code> Default value: <code>None</code>
log_level	<code>Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None</code> Default value: <code>10</code>
args Required*	<code>Any</code>
kwargs Required*	<code>Any</code>

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```
Python
```

```
with_traceback()
```

Attributes

args

AgentInitializationError Class

An error occurred while initializing the agent.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
AgentInitializationError()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

AgentThreadException Class

An error occurred while managing the agent thread.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
AgentThreadException()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

Any Class

Special type indicating an unconstrained type.

- Any is compatible with every type.
- Any assumed to have all methods.
- All values assumed to be instances of Any.

Note that all the above statements are true from the point of view of static type checkers. At runtime, Any should not be used with instance checks.

Constructor

Python
<code>Any()</code>

Methods

 Expand table

<code>__init__</code>
<code>__new__</code>

`__init__`

Python
<code>__init__()</code>

`__new__`

Python
<code>static __new__(cls, *args, **kwargs)</code>

ChatClientException Class

An error occurred while dealing with a chat client.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ChatClientException()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

 Expand table

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ChatClientInitializationError Class

An error occurred while initializing the chat client.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ChatClientInitializationError()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

 Expand table

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	<code>str</code>
inner_exception	<code>Exception None</code> Default value: <code>None</code>
log_level	<code>Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None</code> Default value: <code>10</code>
args Required*	<code>Any</code>
kwargs Required*	<code>Any</code>

`__new__`

Python

```
__new__(**kwargs)
```

`add_note`

`Exception.add_note(note)` – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ContentError Class

An error occurred while processing content.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ContentError()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

 Expand table

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

MiddlewareException Class

An error occurred during middleware execution.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
MiddlewareException()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	<code>str</code>
inner_exception	<code>Exception None</code> Default value: <code>None</code>
log_level	<code>Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None</code> Default value: <code>10</code>
args Required*	<code>Any</code>
kwargs Required*	<code>Any</code>

`__new__`

Python

```
__new__(**kwargs)
```

`add_note`

`Exception.add_note(note)` – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ServiceContentFilterException Class

An error was raised by the content filter of the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ServiceContentFilterException()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

 Expand table

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ServiceException Class

Base class for all service exceptions.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
ServiceException()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ServiceInitializationError Class

An error occurred while initializing the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
ServiceInitializationError()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```


with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ServiceInvalidAuthError Class

An error occurred while authenticating the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
ServiceInvalidAuthError()

Parameters

 Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

 Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ServiceInvalidExecutionSettingsError Class

An error occurred while validating the execution settings of the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ServiceInvalidExecutionSettingsError()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

 Expand table

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	<code>str</code>
inner_exception	<code>Exception None</code> Default value: <code>None</code>
log_level	<code>Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None</code> Default value: <code>10</code>
args Required*	<code>Any</code>
kwargs Required*	<code>Any</code>

`__new__`

Python

```
__new__(**kwargs)
```

`add_note`

`Exception.add_note(note)` – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ServiceInvalidRequestError Class

An error occurred while validating the request to the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ServiceInvalidRequestError()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

 Expand table

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	<code>str</code>
inner_exception	<code>Exception None</code> Default value: <code>None</code>
log_level	<code>Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None</code> Default value: <code>10</code>
args Required*	<code>Any</code>
kwargs Required*	<code>Any</code>

`__new__`

Python

```
__new__(**kwargs)
```

`add_note`

`Exception.add_note(note)` – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ServiceInvalidResponseError Class

An error occurred while validating the response from the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ServiceInvalidResponseError()</code>

Parameters

[Expand table](#)

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

[Expand table](#)

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ServiceResponseException Class

Base class for all service response exceptions.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ServiceResponseException()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

 Expand table

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	<code>str</code>
inner_exception	<code>Exception None</code> Default value: <code>None</code>
log_level	<code>Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None</code> Default value: <code>10</code>
args Required*	<code>Any</code>
kwargs Required*	<code>Any</code>

`__new__`

Python

```
__new__(**kwargs)
```

`add_note`

`Exception.add_note(note)` – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ToolException Class

An error occurred while executing a tool.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ToolException()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

 Expand table

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	str
inner_exception	Exception None Default value: None
log_level	Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

ToolExecutionException Class

An error occurred while executing a tool.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
<code>ToolExecutionException()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>inner_exception</code>	Default value: None
<code>log_level</code>	Default value: 10

Methods

 Expand table

<code>__init__</code>	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
message Required*	<code>str</code>
inner_exception	<code>Exception None</code> Default value: <code>None</code>
log_level	<code>Literal[0] typing.Literal[10] typing.Literal[20] typing.Literal[30] typing.Literal[40] typing.Literal[50] None</code> Default value: <code>10</code>
args Required*	<code>Any</code>
kwargs Required*	<code>Any</code>

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

observability Module

Enums

 Expand table

OtelAttr Enum to capture the attributes used in OpenTelemetry for Generative AI.

Based on: <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/> and <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/>

Functions

configure_otel_providers

Configure otel providers and enable instrumentation for the application with OpenTelemetry.

This method creates the exporters and providers for the application based on the provided values and environment variables and enables instrumentation.

Call this method once during application startup, before any telemetry is captured. DO NOT call this method multiple times, as it may lead to unexpected behavior.

The function automatically reads standard OpenTelemetry environment variables:

- OTEL_EXPORTER_OTLP_ENDPOINT: Base OTLP endpoint for all signals
- OTEL_EXPORTER_OTLP_TRACES_ENDPOINT: OTLP endpoint for traces
- OTEL_EXPORTER_OTLP_METRICS_ENDPOINT: OTLP endpoint for metrics
- OTEL_EXPORTER_OTLP_LOGS_ENDPOINT: OTLP endpoint for logs
- OTEL_EXPORTER_OTLP_PROTOCOL: Protocol (grpc/http)
- OTEL_EXPORTER_OTLP_HEADERS: Headers for all signals
- ENABLE_CONSOLE_EXPORTERS: Enable console output for telemetry

Note

Since you can only setup one provider per signal type (logs, traces, metrics), you can choose to use this method and take the exporter and provider that we created.

Alternatively, you can setup the providers yourself, or through another library

(e.g., Azure Monitor) and just call `enable_instrumentation()` to enable instrumentation.

Note

By default, the Agent Framework emits metrics with the prefixes `agent_framework` and `gen_ai` (OpenTelemetry GenAI semantic conventions). You can use the `views` parameter to filter which metrics are collected and exported. You can also use the `create_metric_views()` helper function to get default views.

-[References]-

- <https://opentelemetry.io/docs/languages/sdk-configuration/general/>
- <https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/>

Python

```
configure_otel_providers(*, enable_sensitive_data: bool | None = None, exporters:
list['LogRecordExporter' | 'SpanExporter' | 'MetricExporter'] | None = None, views:
list['View'] | None = None, vs_code_extension_port: int | None = None,
env_file_path: str | None = None, env_file_encoding: str | None = None) -> None
```

Parameters

 Expand table

Name	Description
enable_sensitive_data Required*	<code>bool</code> <code>None</code>
exporters Required*	<code>list[<xref:SpanExporter> <xref:MetricExporter'>]</code> <code>None</code>
views Required*	<code>list[<xref:'View'>]</code> <code>None</code>
vs_code_extension_port Required*	<code>int</code> <code>None</code>
env_file_path Required*	<code>str</code> <code>None</code>

Name	Description
<code>env_file_encoding</code> Required*	str None

Keyword-Only Parameters

 Expand table

Name	Description
<code>enable_sensitive_data</code>	Enable OpenTelemetry sensitive events. Overrides the environment variable <code>ENABLE_SENSITIVE_DATA</code> if set. Default is <code>None</code> . Default value: <code>None</code>
<code>exporters</code>	A list of custom exporters for logs, metrics or spans, or any combination. These will be added in addition to exporters configured via environment variables. Default is <code>None</code> . Default value: <code>None</code>
<code>views</code>	Optional list of OpenTelemetry views for metrics configuration. Views allow filtering and customizing which metrics are collected. Default is <code>None</code> (empty list). Default value: <code>None</code>
<code>vs_code_extension_port</code>	The port the AI Toolkit or Azure AI Foundry VS Code extensions are listening on. When set, additional OTEL exporters will be created with endpoint <code>http://localhost:{vs_code_extension_port}</code> . Overrides the environment variable <code>VS_CODE_EXTENSION_PORT</code> if set. Default is <code>None</code> . Default value: <code>None</code>
<code>env_file_path</code>	An optional path to a <code>.env</code> file to load environment variables from. Default is <code>None</code> . Default value: <code>None</code>
<code>env_file_encoding</code>	The encoding to use when loading the <code>.env</code> file. Default is <code>None</code> which uses the system default encoding. Default value: <code>None</code>

Returns

 Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.observability import configure_otel_providers

# Using environment variables (recommended)
# Set ENABLE_INSTRUMENTATION=true
# Set OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
configure_otel_providers()

# Enable console output for debugging
# Set ENABLE_CONSOLE_EXPORTERS=true
configure_otel_providers()

# With custom exporters
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter
from opentelemetry.exporter.otlp.proto.grpc._log_exporter import
OTLPLogExporter

configure_otel_providers(
    exporters=[
        OTLPSpanExporter(endpoint="http://custom:4317"),
        OTLPLogExporter(endpoint="http://custom:4317"),
    ],
)

# VS Code extension integration
configure_otel_providers(
    vs_code_extension_port=4317, # Connects to AI Toolkit
)

# Enable sensitive data logging (development only)
configure_otel_providers(
    enable_sensitive_data=True,
)

# With custom metrics views
from opentelemetry.sdk.metrics.view import View

configure_otel_providers(
    views=[
        View(instrument_name="agent_framework*"),
        View(instrument_name="gen_ai*"),
    ],
)
```

This example shows how to first setup your providers, and then ensure Agent Framework emits traces, logs and metrics

Python

```
# when azure monitor is installed
from agent_framework.observability import enable_instrumentation
from azure.monitor.opentelemetry import configure_azure_monitor

connection_string = "InstrumentationKey=your_instrumentation_key_here;..."
configure_azure_monitor(connection_string=connection_string)
enable_instrumentation()
```

create_metric_views

Create the default OpenTelemetry metric views for Agent Framework.

Python

```
create_metric_views() -> list['View']
```

Returns

[Expand table](#)

Type	Description
<code>list['<xref:View>']</code>	

create_resource

Create an OpenTelemetry Resource from environment variables and parameters.

This function reads standard OpenTelemetry environment variables to configure the resource, which identifies your service in telemetry backends.

The following environment variables are read:

- OTEL_SERVICE_NAME: The name of the service (defaults to "agent_framework")
- OTEL_SERVICE_VERSION: The version of the service (defaults to package version)
- OTEL_RESOURCE_ATTRIBUTES: Additional resource attributes as key=value pairs

Python

```
create_resource(service_name: str | None = None, service_version: str | None =
None, env_file_path: str | None = None, env_file_encoding: str | None = None,
**attributes: Any) -> Resource
```

Parameters

[Expand table](#)

Name	Description
service_name	str None Override the service name. If not provided, reads from OTEL_SERVICE_NAME environment variable or defaults to "agent_framework". Default value: None
service_version	str None Override the service version. If not provided, reads from OTEL_SERVICE_VERSION environment variable or defaults to the package version. Default value: None
env_file_path	str None Path to a .env file to load environment variables from. Default is None, which loads from '.env' if present. Default value: None
env_file_encoding	str None Encoding to use when reading the .env file. Default is None, which uses the system default encoding. Default value: None
**attributes Required*	Any Additional resource attributes to include. These will be merged with attributes from OTEL_RESOURCE_ATTRIBUTES environment variable.

Returns

[Expand table](#)

Type	Description
<xref:opentelemetry.sdk.resources.Resource>	A configured OpenTelemetry Resource instance.

Examples

Python

```
from agent_framework.observability import create_resource

# Use defaults from environment variables
resource = create_resource()

# Override service name
```

```
resource = create_resource(service_name="my_service")

# Add custom attributes
resource = create_resource(
    service_name="my_service", service_version="1.0.0",
    deployment_environment="production"
)

# Load from custom .env file
resource = create_resource(env_file_path="config/.env")
```

enable_instrumentation

Enable instrumentation for your application.

Calling this method implies you want to enable observability in your application.

This method does not configure exporters or providers. It only updates the global variables that trigger the instrumentation code. If you have already set the environment variable `ENABLE_INSTRUMENTATION=true`, calling this method has no effect, unless you want to enable or disable sensitive data events.

Python

```
enable_instrumentation(*, enable_sensitive_data: bool | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>enable_sensitive_data</code> Required*	<code>bool</code> <code>None</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>enable_sensitive_data</code>	Enable OpenTelemetry sensitive events. Overrides the environment variable <code>ENABLE_SENSITIVE_DATA</code> if set. Default is <code>None</code> . Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
None	

get_meter

Returns a Meter for Agent Framework.

This is a convenience wrapper for metrics.get_meter() replicating the behavior of opentelemetry.metrics.get_meter().

Python

```
get_meter(name: str = 'agent_framework', version: str = version_info, schema_url: str | None = None, attributes: dict[str, Any] | None = None) -> Meter
```

Parameters

[Expand table](#)

Name	Description
name	str The name of the instrumenting library. Default is "agent_framework". Default value: "agent_framework"
version	str The version of agent_framework. Default is the current version of the package. Default value: version_info
schema_url	str None Optional schema URL of the emitted telemetry. Default value: None
attributes	dict[str, Any] None Optional attributes associated with the emitted telemetry. Default value: None

Returns

[Expand table](#)

Type	Description
<xref:opentelemetry.metrics.Meter>	A Meter instance for recording metrics.

Examples

Python

```

from agent_framework import get_meter

# Get default meter
meter = get_meter()

# Create a counter metric
request_counter = meter.create_counter(
    name="requests",
    description="Number of requests",
    unit="1",
)
request_counter.add(1, {"endpoint": "/api/chat"})

# Create a histogram metric
duration_histogram = meter.create_histogram(
    name="request_duration",
    description="Request duration in seconds",
    unit="s",
)
duration_histogram.record(0.125, {"status": "success"})

```

get_tracer

Returns a Tracer for use by the given instrumentation library.

This function is a convenience wrapper for `trace.get_tracer()` replicating the behavior of `opentelemetry.trace.TracerProvider.get_tracer`. If `tracer_provider` is omitted the current configured one is used.

Python

```

get_tracer(instrumenting_module_name: str = 'agent_framework',
instrumenting_library_version: str = version_info, schema_url: str | None = None,
attributes: dict[str, Any] | None = None) -> Tracer

```

Parameters

Name	Description
<code>instrumenting_module_name</code>	<code>str</code> The name of the instrumenting library. Default is "agent_framework". Default value: "agent_framework"
<code>instrumenting_library_version</code>	<code>str</code> The version of the instrumenting library. Default is the current agent_framework version. Default value: version_info
<code>schema_url</code>	<code>str</code> <code>None</code> Optional schema URL for the emitted telemetry. Default value: None
<code>attributes</code>	<code>dict[str, Any]</code> <code>None</code> Optional attributes associated with the emitted telemetry. Default value: None

Returns

 Expand table

Type	Description
<code><xref:opentelemetry.trace.Tracer></code>	A Tracer instance for creating spans.

Examples

Python

```
from agent_framework import get_tracer

# Get default tracer
tracer = get_tracer()

# Use tracer to create spans
with tracer.start_as_current_span("my_operation") as span:
    span.set_attribute("custom.attribute", "value")
    # Your operation here
    pass

# Get tracer with custom module name
custom_tracer = get_tracer(
    instrumenting_module_name="my_custom_module",
    instrumenting_library_version="1.0.0",
)
```


use_agent_instrumentation

Class decorator that enables OpenTelemetry observability for an agent.

This decorator automatically traces agent run requests, captures events, and logs interactions for the decorated agent class.

ⓘ **Note**

This decorator must be applied to the agent class itself, not an instance.

The agent class should have a class variable AGENT_PROVIDER_NAME to set the proper system name for telemetry.

Python

```
use_agent_instrumentation(agent: type[TAgent] | None = None, *, capture_usage: bool = True) -> type[TAgent] | Callable[[type[TAgent]], type[TAgent]]
```

Parameters

 Expand table

Name	Description
agent	<code>type[<xref:TAgent>] None</code> The agent class to enable observability for. Default value: None
capture_usage Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
capture_usage	Whether to capture token usage as a span attribute. Defaults to True, set to False when the agent has underlying traces that already capture token usage to avoid double counting. Default value: True

Returns

[Expand table](#)

Type	Description
<code>type[~<xref:TAgent>] Callable[[type[~<xref:TAgent>]], type[~<xref:TAgent>]]</code>	The decorated agent class with observability enabled.

Exceptions

[Expand table](#)

Type	Description
<code>AgentInitializationError</code>	If the agent does not have required methods (run, run_stream).

Examples

Python

```
from agent_framework import use_agent_instrumentation, configure_otel_providers
from agent_framework._agents import AgentProtocol

# Decorate a custom agent class
@use_agent_instrumentation
class MyCustomAgent:
    AGENT_PROVIDER_NAME = "my_agent_system"

    async def run(self, messages=None, *, thread=None, **kwargs):
        # Your implementation
        pass

    async def run_stream(self, messages=None, *, thread=None, **kwargs):
        # Your implementation
        pass

# Setup observability
configure_otel_providers(otlp_endpoint="http://localhost:4317")

# Now all agent runs will be traced
agent = MyCustomAgent()
response = await agent.run("Perform a task")
```

use_instrumentation

Class decorator that enables OpenTelemetry observability for a chat client.

This decorator automatically traces chat completion requests, captures metrics, and logs events for the decorated chat client class.

 Note

This decorator must be applied to the class itself, not an instance.

The chat client class should have a class variable `OTEL_PROVIDER_NAME` to set the proper provider name for telemetry.

Python

```
use_instrumentation(chat_client: type[TChatClient]) -> type[TChatClient]
```

Parameters

 Expand table

Name	Description
chat_client Required*	<code>type[<xref:TChatClient>]</code> The chat client class to enable observability for.

Returns

 Expand table

Type	Description
<code>type[~<xref:TChatClient>]</code>	The decorated chat client class with observability enabled.

Exceptions

 Expand table

Type	Description
<code>ChatClientInitializationError</code>	If the chat client does not have required methods (<code>get_response</code> , <code>get_streaming_response</code>).

Examples

Python

```
from agent_framework import use_instrumentation, configure_otel_providers
from agent_framework import ChatClientProtocol

# Decorate a custom chat client class
@use_instrumentation
class MyCustomChatClient:
    OTEL_PROVIDER_NAME = "my_provider"

    async def get_response(self, messages, **kwargs):
        # Your implementation
        pass

    async def get_streaming_response(self, messages, **kwargs):
        # Your implementation
        pass

# Setup observability
configure_otel_providers(otlp_endpoint="http://localhost:4317")

# Now all calls will be traced
client = MyCustomChatClient()
response = await client.get_response("Hello")
```

OtelAttr Enum

Enum to capture the attributes used in OpenTelemetry for Generative AI.

Based on: <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/> and <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/>

Fields

 Expand table

ADDRESS
AGENT_CREATE_OPERATION
AGENT_DESCRIPTION
AGENT_FRAMEWORK_GEN_AI_SYSTEM
AGENT_ID
AGENT_INVOKE_OPERATION
AGENT_NAME
ASSISTANT_MESSAGE
BUILD_COMPLETED
BUILD_ERROR
BUILD_ERROR_MESSAGE
BUILD_ERROR_TYPE
BUILD_STARTED
BUILD_VALIDATION_COMPLETED
CHAT_COMPLETION_OPERATION
CHOICE
CHOICE_COUNT
CONVERSATION_ID
DATA_SOURCE_ID

DURATION_UNIT
EDGE_GROUP_DELIVERED
EDGE_GROUP_DELIVERY_STATUS
EDGE_GROUP_ID
EDGE_GROUP_PROCESS_SPAN
EDGE_GROUP_TYPE
ENCODING_FORMATS
ERROR_TYPE
EVENT_NAME
EXECUTOR_ID
EXECUTOR_PROCESS_SPAN
EXECUTOR_TYPE
FINISH_REASONS
FREQUENCY_PENALTY
INPUT_MESSAGES
INPUT_TOKENS
MEASUREMENT_FUNCTION_INVOCATION_DURATION
MEASUREMENT_FUNCTION_TAG_NAME
MESSAGE_DESTINATION_EXECUTOR_ID
MESSAGE_PAYLOAD_TYPE
MESSAGE_SEND_SPAN
MESSAGE_SOURCE_ID
MESSAGE_TARGET_ID
MESSAGE_TYPE
OPERATION
OUTPUT_MESSAGES
OUTPUT_TOKENS

OUTPUT_TYPE
PORT
PRESENCE_PENALTY
PROVIDER_NAME
RESPONSE_ID
SEED
SPAN_ID
STOP_SEQUENCES
SYSTEM_INSTRUCTIONS
SYSTEM_MESSAGE
TOOL_ARGUMENTS
TOOL_CALL_ID
TOOL_DEFINITIONS
TOOL_DESCRIPTION
TOOL_EXECUTION_OPERATION
TOOL_MESSAGE
TOOL_NAME
TOOL_RESULT
TOOL_TYPE
TOP_K
TRACE_ID
T_TYPE
T_TYPE_INPUT
T_TYPE_OUTPUT
T_UNIT
USER_MESSAGE
WORKFLOW_BUILD_SPAN

WORKFLOW_COMPLETED
WORKFLOW_DEFINITION
WORKFLOW_DESCRIPTION
WORKFLOW_ERROR
WORKFLOW_ID
WORKFLOW_NAME
WORKFLOW_RUN_SPAN
WORKFLOW_STARTED

AIFunction Class

A tool that wraps a Python function to make it callable by AI models.

This class wraps a Python function to make it callable by AI models with automatic parameter validation and JSON schema generation.

Initialize the AIFunction.

Constructor

Python
AIFunction()

Keyword-Only Parameters

 Expand table

Name	Description
name	The name of the function.
description	A description of the function.
approval_mode	Whether or not approval is required to run this tool. Default is that approval is not needed. Default value: None
max_invocations	The maximum number of times this function can be invoked. If None, there is no limit. Should be at least 1. Default value: None
max_invocation_exceptions	The maximum number of exceptions allowed during invocations. If None, there is no limit. Should be at least 1. Default value: None
additional_properties	Additional properties to set on the function. Default value: None
func	The function to wrap. Default value: None
input_model	The Pydantic model that defines the input parameters for the function. This can also be a JSON schema dictionary. If not provided, it will be inferred from the function signature.

Name	Description
	Default value: None
**kwargs	Additional keyword arguments.

Examples

Python

```
from typing import Annotated
from pydantic import BaseModel, Field
from agent_framework import AIFunction, ai_function

# Using the decorator with string annotations
@ai_function
def get_weather(
    location: Annotated[str, "The city name"],
    unit: Annotated[str, "Temperature unit"] = "celsius",
) -> str:
    '''Get the weather for a location.'''
    return f"Weather in {location}: 22°{unit[0].upper()}"

# Using direct instantiation with Field
class WeatherArgs(BaseModel):
    location: Annotated[str, Field(description="The city name")]
    unit: Annotated[str, Field(description="Temperature unit")] = "celsius"

weather_func = AIFunction(
    name="get_weather",
    description="Get the weather for a location",
    func=lambda location, unit="celsius": f"Weather in {location}: 22°
{unit[0].upper()}",
    approval_mode="never_require",
    input_model=WeatherArgs,
)

# Invoke the function
result = await weather_func.invoke(arguments=WeatherArgs(location="Seattle"))
```

Methods

 Expand table

<code>__init__</code>	Initialize the AIFunction.
-----------------------	----------------------------

<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>invoke</code>	Run the AI function with the provided arguments as a Pydantic model.
<code>parameters</code>	Create the JSON schema of the parameters.
<code>to_dict</code>	
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.
<code>to_json_schema_spec</code>	Convert a AIFunction to the JSON Schema function specification format.

`__init__`

Initialize the AIFunction.

Python

```
__init__(*, name: str, description: str = '', approval_mode: Literal['always_require', 'never_require'] | None = None, max_invocations: int | None = None, max_invocation_exceptions: int | None = None, additional_properties: dict[str, Any] | None = None, func: Callable[[...], Awaitable[ReturnT] | ReturnT] | None = None, input_model: type[ArgsT] | Mapping[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
name Required*	str
description Required*	str
approval_mode Required*	Literal ['always_require', 'never_require'] None
max_invocations Required*	int None
max_invocation_exceptions Required*	int None
additional_properties Required*	dict [str , Any] None
func Required*	Callable [[...] , Awaitable [<xref:agent_framework_tools.ReturnT>] <xref:agent_framework_tools.ReturnT>] None
input_model Required*	type [<xref:ArgsT>] Mapping [str , Any] None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
name	The name of the function.
description	A description of the function. Default value: ""
approval_mode	Whether or not approval is required to run this tool. Default is that approval is not needed. Default value: None
max_invocations	The maximum number of times this function can be invoked. If None, there is no limit. Should be at least 1. Default value: None
max_invocation_exceptions	The maximum number of exceptions allowed during invocations. If None, there is no limit. Should be at least 1.

Name	Description
	Default value: None
additional_properties	Additional properties to set on the function. Default value: None
func	The function to wrap. Default value: None
input_model	The Pydantic model that defines the input parameters for the function. This can also be a JSON schema dictionary. If not provided, it will be inferred from the function signature. Default value: None
**kwargs	Additional keyword arguments.

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"
```

```

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
<code>from_json()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TCIass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
<code>JSONDecodeError</code>	If the JSON string is malformed.
<code>ValueError</code>	If the parsed data doesn't contain a valid 'type' field.

invoke

Run the AI function with the provided arguments as a Pydantic model.

Python

```
async invoke(*, arguments: ArgsT | None = None, **kwargs: Any) -> ReturnT
```

Parameters

[Expand table](#)

Name	Description
arguments Required*	<code><xref:agent_framework_tools.ArgsT></code> <code>None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
arguments	A Pydantic model instance containing the arguments for the function. Default value: None

Name	Description
kwargs	Keyword arguments to pass to the function, will not be used if arguments is provided.

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_tools.ReturnT>	The result of the function execution.

Exceptions

[Expand table](#)

Type	Description
TypeError	If arguments is not an instance of the expected input model.

parameters

Create the JSON schema of the parameters.

Python
parameters() -> dict[str, Any]

Returns

[Expand table](#)

Type	Description
dict[str, Any]	A dictionary containing the JSON schema for the function's parameters. The result is cached after the first call for performance.

to_dict

Python
to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]

Parameters

 Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Default value: None
<code>exclude_none</code>	Default value: True

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

to_json_schema_spec

Convert a `AIFunction` to the JSON Schema function specification format.

Python

```
to_json_schema_spec() -> dict[str, Any]
```

Returns

Type	Description
<code>dict[str, Any]</code>	A dictionary containing the function specification in JSON Schema format.

Attributes

`declaration_only`

Indicate whether the function is declaration only (i.e., has no implementation).

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'_cached_parameters',
'_invocation_duration_histogram', 'input_model'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'func'}
```

AgentExecutor Class

built-in executor that wraps an agent for handling messages.

AgentExecutor adapts its behavior based on the workflow execution mode:

- `run_stream()`: Emits incremental `AgentRunUpdateEvent` events as the agent produces tokens
- `run()`: Emits a single `AgentRunEvent` containing the complete response

The executor automatically detects the mode via `WorkflowContext.is_streaming()`.

Initialize the executor with a unique identifier.

Constructor

Python
<code>AgentExecutor()</code>

Parameters

 Expand table

Name	Description
agent Required*	The agent to be wrapped by this executor.
agent_thread Required*	The thread to use for running the agent. If None, a new thread will be created.
output_response Required*	Whether to yield an <code>AgentRunResponse</code> as a workflow output when the agent completes.
id Required*	A unique identifier for the executor. If None, the agent's name will be used if available.

Keyword-Only Parameters

 Expand table

Name	Description
agent_thread	Default value: None
output_response	Default value: False
id	Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the executor with a unique identifier.
<code>__new__</code>	
<code>can_handle</code>	Check if the executor can handle a given message type.
<code>clone</code>	
<code>execute</code>	Execute the executor with a given message and context parameters. - Do not call this method directly - it is invoked by the workflow engine. - Do not override this method. Instead, define handlers using <code>@handler</code> decorator.
<code>from_dict</code>	
<code>from_json</code>	
<code>from_message</code>	Accept a single ChatMessage as input.
<code>from_messages</code>	Accept a list of chat inputs (strings or ChatMessage) as conversation context.
<code>from_response</code>	Enable seamless chaining: accept a prior AgentExecutorResponse as input. Strategy: treat the prior response's messages as the conversation state and immediately run the agent to produce a new response.
<code>from_str</code>	Accept a raw user prompt string and run the agent (one-shot).
<code>handle_user_input_response</code>	Handle user input responses for function approvals during agent execution. This will hold the executor's execution until all pending user input requests are resolved.
<code>is_request_supported</code>	Check if the executor supports request of the given type and handling a response of the given type.
<code>on_checkpoint_restore</code>	Restore executor state from checkpoint.

on_checkpoint_save	Capture current executor state for checkpointing. NOTE: if the thread storage is on the server side, the full thread state may not be serialized locally. Therefore, we are relying on the server-side to ensure the thread state is preserved and immutable across checkpoints. This is not the case for AzureAI Agents, but works for the Responses API.
reset	Reset the internal cache of the executor.
run	Handle an AgentExecutorRequest (canonical input). This is the standard path: extend cache with provided messages; if should_respond run the agent and emit an AgentExecutorResponse downstream.
to_dict	Serialize executor definition for workflow topology export.
to_json	

`__init__`

Initialize the executor with a unique identifier.

Python

```
__init__(agent: AgentProtocol, *, agent_thread: AgentThread | None = None,
output_response: bool = False, id: str | None = None)
```

Parameters

 Expand table

Name	Description
agent Required*	AgentProtocol The agent to be wrapped by this executor.
agent_thread Required*	AgentThread None The thread to use for running the agent. If None, a new thread will be created.
output_response Required*	bool Whether to yield an AgentRunResponse as a workflow output when the agent completes.
id Required*	str None A unique identifier for the executor. If None, the agent's name will be used if available.

Keyword-Only Parameters

 Expand table

Name	Description
agent_thread	Default value: None
output_response	Default value: False
id	Default value: None

__new__

Python
<code>__new__(**kwargs)</code>

can_handle

Check if the executor can handle a given message type.

Python
<code>can_handle(message: Message) -> bool</code>

Parameters

 Expand table

Name	Description
message Required*	Message The message to check.

Returns

 Expand table

Type	Description
bool	True if the executor can handle the message type, False otherwise.

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

[Expand table](#)

Name	Description
deep Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
deep	Default value: True

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using @handler decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state: SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] | None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] None Optional source span IDs from multiple sources for linking. Default value: None

Returns

[Expand table](#)

Type	Description
None	An awaitable that resolves to the result of the execution.

from_dict

Python

```
from_dict(data: dict[str, Any]) -> TModel
```

Parameters

[Expand table](#)

Name	Description
data Required*	dict[str, Any]

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_workflows_model_utils.TModel>	

from_json

Python
<pre>from_json(raw: str) -> TModel</pre>

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_workflows_model_utils.TModel>	

from_message

Accept a single ChatMessage as input.

Python
<pre>async from_message(message: ChatMessage, ctx: WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None</pre>

Parameters

[Expand table](#)

Name	Description
message Required*	ChatMessage
ctx Required*	WorkflowContext [AgentExecutorResponse , AgentRunResponse]

Returns

[Expand table](#)

Type	Description
None	

from_messages

Accept a list of chat inputs (strings or ChatMessage) as conversation context.

Python

```
async from_messages(messages: list[str | ChatMessage], ctx:
WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None
```

Parameters

[Expand table](#)

Name	Description
messages Required*	list [str ChatMessage]
ctx Required*	WorkflowContext [AgentExecutorResponse , AgentRunResponse]

Returns

[Expand table](#)

Type	Description
None	

from_response

Enable seamless chaining: accept a prior AgentExecutorResponse as input.

Strategy: treat the prior response's messages as the conversation state and immediately run the agent to produce a new response.

Python

```
async from_response(prior: AgentExecutorResponse, ctx:
WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None
```

Parameters

[Expand table](#)

Name	Description
prior Required*	AgentExecutorResponse
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse]

Returns

[Expand table](#)

Type	Description
None	

from_str

Accept a raw user prompt string and run the agent (one-shot).

Python

```
async from_str(text: str, ctx: WorkflowContext[AgentExecutorResponse,
AgentRunResponse]) -> None
```

Parameters

[Expand table](#)

Name	Description
text Required*	str
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse]

Returns

[Expand table](#)

Type	Description
None	

handle_user_input_response

Handle user input responses for function approvals during agent execution.

This will hold the executor's execution until all pending user input requests are resolved.

Python

```
async def handle_user_input_response(original_request: FunctionApprovalRequestContent,
    response: FunctionApprovalResponseContent, ctx:
    WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None
```

Parameters

[Expand table](#)

Name	Description
original_request Required*	FunctionApprovalRequestContent The original function approval request sent by the agent.
response Required*	FunctionApprovalResponseContent The user's response to the function approval request.
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse] The workflow context for emitting events and outputs.

Returns

[Expand table](#)

Type	Description
None	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

Python

```
is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool
```

Parameters

[Expand table](#)

Name	Description
request_type Required*	type[Any] The type of the request message
response_type Required*	type[Any] The type of the expected response message

Returns

[Expand table](#)

Type	Description
bool	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Restore executor state from checkpoint.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
state Required*	<code>dict[str, Any]</code> Checkpoint data dict

Returns

[Expand table](#)

Type	Description
<code>None</code>	

on_checkpoint_save

Capture current executor state for checkpointing.

NOTE: if the thread storage is on the server side, the full thread state may not be serialized locally. Therefore, we are relying on the server-side to ensure the thread state is preserved and immutable across checkpoints. This is not the case for AzureAI Agents, but works for the Responses API.

Python

```
async on_checkpoint_save() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dict containing serialized cache and thread state

reset

Reset the internal cache of the executor.

Python

```
reset() -> None
```

Returns

[Expand table](#)

Type	Description
None	

run

Handle an AgentExecutorRequest (canonical input).

This is the standard path: extend cache with provided messages; if should_respond run the agent and emit an AgentExecutorResponse downstream.

Python

```
async run(request: AgentExecutorRequest, ctx:
WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None
```

Parameters

[Expand table](#)

Name	Description
request Required*	AgentExecutorRequest
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse]

Returns

[Expand table](#)

Type	Description
None	

to_dict

Serialize executor definition for workflow topology export.

Python

```
to_dict() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

to_json

Python

```
to_json() -> str
```

Returns

[Expand table](#)

Type	Description
<code>str</code>	

Attributes

description

Get the description of the underlying agent.

input_types

Get the list of input types that this executor can handle.

Returns

[Expand table](#)

Type	Description
	A list of the message types that this executor's handlers can process.

output_response

Whether this executor yields AgentRunResponse as workflow output when complete.

output_types

Get the list of output types that this executor can produce via send_message().

Returns

 Expand table

Type	Description
	A list of the output types inferred from the handlers' WorkflowContext[T] annotations.

workflow_output_types

AgentExecutorRequest Class

A request to an agent executor.

Constructor

Python
<code>AgentExecutorRequest()</code>

Parameters

[Expand table](#)

Name	Description
<code>messages</code> Required*	
<code>should_respond</code>	Default value: True

Methods

[Expand table](#)

<code>__init__</code>
<code>__new__</code>

`__init__`

Python
<code>__init__(messages: list[ChatMessage], should_respond: bool = True) -> None</code>

Parameters

[Expand table](#)

Name	Description
messages Required*	<code>list[ChatMessage]</code>
should_respond	<code>bool</code> Default value: True

Returns

 Expand table

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

Attributes

messages

A list of chat messages to be processed by the agent.

Python
<code>messages: list[ChatMessage]</code>

should_respond

A flag indicating whether the agent should respond to the messages. If False, the messages will be saved to the executor's cache but not sent to the agent.

Python
<code>should_respond: bool = True</code>

AgentExecutorResponse Class

A response from an agent executor.

Constructor

Python
<code>AgentExecutorResponse()</code>

Parameters

[Expand table](#)

Name	Description
<code>executor_id</code> Required*	
<code>agent_run_response</code> Required*	
<code>full_conversation</code>	Default value: None

Methods

[Expand table](#)

<code>__init__</code>
<code>__new__</code>

`__init__`

Python
<code>__init__(executor_id: str, agent_run_response: AgentRunResponse, full_conversation: list[ChatMessage] None = None) -> None</code>

Parameters

[Expand table](#)

Name	Description
executor_id Required*	str
agent_run_response Required*	AgentRunResponse
full_conversation	list[ChatMessage] None Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

Attributes

executor_id

The ID of the executor that generated the response.

Python
<code>executor_id: str</code>

agent_run_response

The underlying agent run response (unaltered from client).

Python
<code>agent_run_response: AgentRunResponse</code>

full_conversation

The full conversation context (prior inputs + all assistant/tool outputs) that should be used when chaining to another AgentExecutor. This prevents downstream agents losing user prompts while keeping the emitted AgentRunEvent text faithful to the raw agent output.

Python

```
full_conversation: list[ChatMessage] | None = None
```

AgentInputRequest Class

Request for human input before an agent runs in high-level builder workflows.

Emitted via RequestInfoEvent when a workflow pauses before an agent executes. The response is injected into the conversation as a user message to steer the agent's behavior.

This is the standard request type used by `.with_request_info()` on SequentialBuilder, ConcurrentBuilder, GroupChatBuilder, and HandoffBuilder.

Constructor

Python
<code>AgentInputRequest()</code>

Parameters

 Expand table

Name	Description
<code>target_agent_id</code> Required*	
<code>conversation</code>	Default value: <factory>
<code>instruction</code>	Default value: None
<code>metadata</code>	Default value: <factory>

Methods

 Expand table

<code>__init__</code>
<code>__new__</code>

`__init__`

Python

```
__init__(target_agent_id: str | None, conversation:
list[agent_framework._types.ChatMessage] = <factory>, instruction: str | None =
None, metadata: dict[str, typing.Any] = <factory>) -> None
```

Parameters

 Expand table

Name	Description
target_agent_id Required*	str None
conversation	list[ChatMessage] Default value: <factory>
instruction	str None Default value: None
metadata	dict[str, Any] Default value: <factory>

Returns

 Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

Attributes

target_agent_id

ID of the agent that is about to run

Python

```
target_agent_id: str | None
```

conversation

Current conversation history the agent will receive

```
Python
```

```
conversation: list[ChatMessage]
```

instruction

Optional instruction from the orchestrator (e.g., manager in GroupChat)

```
Python
```

```
instruction: str | None = None
```

metadata

Builder-specific context (stores internal state for resume)

```
Python
```

```
metadata: dict[str, Any]
```

AgentMiddleware Class

Abstract base class for agent middleware that can intercept agent invocations.

Agent middleware allows you to intercept and modify agent invocations before and after execution. You can inspect messages, modify context, override results, or terminate execution early.

⚠ Note

AgentMiddleware is an abstract base class. You must subclass it and implement the `process()` method to create custom agent middleware.

Constructor

Python

```
AgentMiddleware()
```

Examples

Python

```
from agent_framework import AgentMiddleware, AgentRunContext, ChatAgent

class RetryMiddleware(AgentMiddleware):
    def __init__(self, max_retries: int = 3):
        self.max_retries = max_retries

    async def process(self, context: AgentRunContext, next):
        for attempt in range(self.max_retries):
            await next(context)
            if context.result and not context.result.is_error:
                break
            print(f"Retry {attempt + 1}/{self.max_retries}")

# Use with an agent
agent = ChatAgent(chat_client=client, name="assistant",
middleware=RetryMiddleware())
```

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>process</code>	Process an agent invocation.

 **Note**

Middleware should not return anything. All data manipulation should happen within the context object. Set `context.result` to override execution, or observe `context.result` after calling `next()` for actual results.

`__init__`

Python
<code>__init__()</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`process`

Process an agent invocation.

 Note Middleware should not return anything. All data manipulation should happen within the context object. Set <code>context.result</code> to override execution, or observe <code>context.result</code> after calling <code>next()</code> for actual results.

Python

```
abstract async process(context: AgentRunContext, next: Callable[[AgentRunContext],  
Awaitable[None]]) -> None
```

Parameters

 Expand table

Name	Description
context Required*	AgentRunContext Agent invocation context containing agent, messages, and metadata. Use context.is_streaming to determine if this is a streaming call. Middleware can set context.result to override execution, or observe the actual execution result after calling next(). For non-streaming: AgentRunResponse For streaming: AsyncIterable[AgentRunResponseUpdate]
next Required*	Callable[[AgentRunContext], Awaitable[None]] Function to call the next middleware or final agent execution. Does not return anything - all data flows through the context.

Returns

 Expand table

Type	Description
None	

AgentProtocol Class

A protocol for an agent that can be invoked.

This protocol defines the interface that all agents must implement, including properties for identification and methods for execution.

⚠ Note

Protocols use structural subtyping (duck typing). Classes don't need to explicitly inherit from this protocol to be considered compatible.

This allows you to create completely custom agents without using any Agent Framework base classes.

Constructor

Python

```
AgentProtocol()
```

Examples

Python

```
from agent_framework import AgentProtocol

# Any class implementing the required methods is compatible
# No need to inherit from AgentProtocol or use any framework classes
class CustomAgent:
    def __init__(self):
        self._id = "custom-agent-001"
        self._name = "Custom Agent"

    @property
    def id(self) -> str:
        return self._id

    @property
    def name(self) -> str | None:
```

```

        return self._name

    @property
    def display_name(self) -> str:
        return self.name or self.id

    @property
    def description(self) -> str | None:
        return "A fully custom agent implementation"

    async def run(self, messages=None, *, thread=None, **kwargs):
        # Your custom implementation
        from agent_framework import AgentRunResponse

        return AgentRunResponse(messages=[], response_id="custom-response")

    def run_stream(self, messages=None, *, thread=None, **kwargs):
        # Your custom streaming implementation
        async def _stream():
            from agent_framework import AgentRunResponseUpdate

            yield AgentRunResponseUpdate()

        return _stream()

    def get_new_thread(self, **kwargs):
        # Return your own thread implementation
        return {"id": "custom-thread", "messages": []}

# Verify the instance satisfies the protocol
instance = CustomAgent()
assert isinstance(instance, AgentProtocol)

```

Methods

[Expand table](#)

<code>__init__</code>	
<code>__new__</code>	
<code>get_new_thread</code>	Creates a new conversation thread for the agent.
<code>run</code>	Get a response from the agent. This method returns the final result of the agent's execution as a single <code>AgentRunResponse</code> object. The caller is blocked until the final result is available. Note: For streaming responses, use the <code>run_stream</code> method, which returns intermediate steps and the final result as a stream of <code>AgentRunResponseUpdate</code> objects. Streaming

only the final result is not feasible because the timing of the final result's availability is unknown, and blocking the caller until then is undesirable in streaming scenarios.

`run_stream`

Run the agent as a stream.

This method will return the intermediate steps and final results of the agent's execution as a stream of `AgentRunResponseUpdate` objects to the caller.

Note: An `AgentRunResponseUpdate` object contains a chunk of a message.

`__init__`

Python

```
__init__(*args, **kwargs)
```

`__new__`

Python

```
__new__(**kwargs)
```

`get_new_thread`

Creates a new conversation thread for the agent.

Python

```
get_new_thread(**kwargs: Any) -> AgentThread
```

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
AgentThread	

run

Get a response from the agent.

This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available.

Note: For streaming responses, use the run_stream method, which returns intermediate steps and the final result as a stream of AgentRunResponseUpdate objects. Streaming only the final result is not feasible because the timing of the final result's availability is unknown, and blocking the caller until then is undesirable in streaming scenarios.

Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, **kwargs: Any) -> AgentRunResponse
```

Parameters

 Expand table

Name	Description
messages	str ChatMessage list[str] list[ChatMessage] None The message(s) to send to the agent. Default value: None
thread Required*	AgentThread None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None

Name	Description
kwargs	Additional keyword arguments.

Returns

[Expand table](#)

Type	Description
AgentRunResponse	An agent response item.

run_stream

Run the agent as a stream.

This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller.

Note: An AgentRunResponseUpdate object contains a chunk of a message.

Python

```
run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, **kwargs: Any) -> AsyncIterable[AgentRunResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages	str ChatMessage list[str] list[ChatMessage] None The message(s) to send to the agent. Default value: None
thread Required*	AgentThread None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

[Expand table](#)

Type	Description
AsyncIterable [AgentRunResponseUpdate]	

Attributes

description

Returns the description of the agent.

display_name

Returns the display name of the agent.

id

Returns the ID of the agent.

name

Returns the name of the agent.

AgentRunContext Class

Context object for agent middleware invocations.

This context is passed through the agent middleware pipeline and contains all information about the agent invocation.

Initialize the AgentRunContext.

Constructor

Python
<code>AgentRunContext()</code>

Parameters

 Expand table

Name	Description
agent Required*	The agent being invoked.
messages Required*	The messages being sent to the agent.
thread	The agent thread for this invocation, if any. Default value: None
is_streaming	Whether this is a streaming invocation. Default value: False
metadata	Metadata dictionary for sharing data between agent middleware. Default value: None
result	Agent execution result. Default value: None
terminate	A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	Additional keyword arguments passed to the agent run method. Default value: None

Examples

Python

```
from agent_framework import AgentMiddleware, AgentRunContext

class LoggingMiddleware(AgentMiddleware):
    async def process(self, context: AgentRunContext, next):
        print(f"Agent: {context.agent.name}")
        print(f"Messages: {len(context.messages)}")
        print(f"Thread: {context.thread}")
        print(f"Streaming: {context.is_streaming}")

        # Store metadata
        context.metadata["start_time"] = time.time()

        # Continue execution
        await next(context)

        # Access result after execution
        print(f"Result: {context.result}")
```

Methods

 Expand table

<code>__init__</code>	Initialize the AgentRunContext.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

`to_json` Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize the AgentRunContext.

Python

```
__init__(agent: AgentProtocol, messages: list[ChatMessage], thread: AgentThread | None = None, is_streaming: bool = False, metadata: dict[str, Any] | None = None, result: AgentRunResponse | AsyncIterable[AgentRunResponseUpdate] | None = None, terminate: bool = False, kwargs: dict[str, Any] | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
agent Required*	AgentProtocol The agent being invoked.
messages Required*	list[ChatMessage] The messages being sent to the agent.
thread	AgentThread None The agent thread for this invocation, if any. Default value: None
is_streaming	bool Whether this is a streaming invocation. Default value: False
metadata	dict[str, Any] None Metadata dictionary for sharing data between agent middleware. Default value: None
result	AgentRunResponse AsyncIterable[AgentRunResponseUpdate] None Agent execution result. Default value: None

Name	Description
terminate	<code>bool</code> A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	<code>dict[str, Any] None</code> Additional keyword arguments passed to the agent run method. Default value: None

Returns

[Expand table](#)

Type	Description
<code>None</code>	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value	

Name	Description
Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
```

```

function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
<code>from_json()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->  
dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>

Name	Description
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

agent

The agent being invoked.

messages

The messages being sent to the agent.

thread

The agent thread for this invocation, if any.

is_streaming

Whether this is a streaming invocation.

metadata

Metadata dictionary for sharing data between agent middleware.

result

Agent execution result. Can be observed after calling `next()` to see the actual execution result or can be set to override the execution result. For non-streaming: should be `AgentRunResponse`. For streaming: should be `AsyncIterable[AgentRunResponseUpdate]`.

terminate

A flag indicating whether to terminate execution after current middleware. When set to `True`, execution will stop as soon as control returns to framework.

kwargs

Additional keyword arguments passed to the agent run method.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'agent', 'result', 'thread'}
```

AgentRunEvent Class

Event triggered when an agent run is completed.

Initialize the agent run event.

Constructor

Python
<code>AgentRunEvent()</code>

Parameters

 Expand table

Name	Description
<code>executor_id</code> Required*	
<code>data</code>	Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the agent run event.
<code>__new__</code>	

`__init__`

Initialize the agent run event.

Python
<code>__init__(executor_id: str, data: AgentRunResponse None = None)</code>

Parameters

 Expand table

Name	Description
executor_id Required*	str
data	AgentRunResponse None Default value: None

__new__

Python
<code>__new__(**kwargs)</code>

Attributes

data

Python
<code>data: AgentRunResponse None</code>

AgentRunResponse Class

Represents the response to an Agent run request.

Provides one or more response messages and metadata about the response. A typical response will contain a single message, but may contain multiple messages in scenarios involving function calls, RAG retrievals, or complex logic.

Initialize an AgentRunResponse.

Constructor

Python
<code>AgentRunResponse()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>messages</code>	The list of chat messages in the response. Default value: None
<code>response_id</code>	The ID of the chat response. Default value: None
<code>created_at</code>	A timestamp for the chat response. Default value: None
<code>usage_details</code>	The usage details for the chat response. Default value: None
<code>value</code>	The structured output of the agent run response, if applicable. Default value: None
<code>additional_properties</code>	Any additional properties associated with the chat response. Default value: None
<code>raw_representation</code>	The raw representation of the chat response from an underlying implementation. Default value: None
<code>**kwargs</code>	Additional properties to set on the response.

Examples

Python

```
from agent_framework import AgentRunResponse, ChatMessage

# Create agent response
msg = ChatMessage(role="assistant", text="Task completed successfully.")
response = AgentRunResponse(messages=[msg], response_id="run_123")
print(response.text) # "Task completed successfully."

# Access user input requests
user_requests = response.user_input_requests
print(len(user_requests)) # 0

# Combine streaming updates
updates = [...] # List of AgentRunResponseUpdate objects
response = AgentRunResponse.from_agent_run_response_updates(updates)

# Serialization - to_dict and from_dict
response_dict = response.to_dict()
# {'type': 'agent_run_response', 'messages': [...], 'response_id': 'run_123',
#  'additional_properties': {}}
restored_response = AgentRunResponse.from_dict(response_dict)
print(restored_response.response_id) # "run_123"

# Serialization - to_json and from_json
response_json = response.to_json()
# '{"type": "agent_run_response", "messages": [...], "response_id": "run_123",
...}'
restored_from_json = AgentRunResponse.from_json(response_json)
print(restored_from_json.text) # "Task completed successfully."
```

Methods

 Expand table

<code>__init__</code>	Initialize an AgentRunResponse.
<code>__new__</code>	
<code>from_agent_response_generator</code>	Joins multiple updates into a single AgentRunResponse.
<code>from_agent_run_response_updates</code>	Joins multiple updates into a single AgentRunResponse.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It

	supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.
<code>try_parse_value</code>	If there is a value, does nothing, otherwise tries to parse the text into the value.

`__init__`

Initialize an AgentRunResponse.

Python

```
__init__(*, messages: ChatMessage | list[ChatMessage] | MutableMapping[str, Any] | list[MutableMapping[str, Any]] | None = None, response_id: str | None = None, created_at: str | None = None, usage_details: UsageDetails | MutableMapping[str, Any] | None = None, value: Any | None = None, raw_representation: Any | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

Name	Description
messages Required*	ChatMessage list[ChatMessage] MutableMapping[str, Any] list[MutableMapping[str, Any]] None
response_id Required*	str None
created_at Required*	str None
usage_details Required*	UsageDetails MutableMapping[str, Any] None
value Required*	Any None
raw_representation Required*	Any None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
messages	The list of chat messages in the response. Default value: None
response_id	The ID of the chat response. Default value: None
created_at	A timestamp for the chat response. Default value: None
usage_details	The usage details for the chat response. Default value: None
value	The structured output of the agent run response, if applicable. Default value: None
additional_properties	Any additional properties associated with the chat response. Default value: None

Name	Description
<code>raw_representation</code>	The raw representation of the chat response from an underlying implementation. Default value: None
<code>**kwargs</code>	Additional properties to set on the response.

Returns

[Expand table](#)

Type	Description
None	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_agent_response_generator`

Joins multiple updates into a single AgentRunResponse.

Python
<pre>async classmethod from_agent_response_generator(updates: AsyncIterable[AgentRunResponseUpdate], *, output_format_type: type[BaseModel] None = None) -> TAgentRunResponse</pre>

Parameters

[Expand table](#)

Name	Description
<code>updates</code> Required*	AsyncIterable[AgentRunResponseUpdate] An async iterable of AgentRunResponseUpdate objects to combine.
<code>output_format_type</code> Required*	type[<xref:pydantic.main.BaseModel>] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
output_format_type	Optional Pydantic model type to parse the response text into structured data Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._types.TAgentRunResponse></code>	

from_agent_run_response_updates

Joins multiple updates into a single AgentRunResponse.

Python

```
from_agent_run_response_updates(updates: Sequence[AgentRunResponseUpdate], *,  
output_format_type: type[BaseModel] | None = None) -> TAgentRunResponse
```

Parameters

[Expand table](#)

Name	Description
updates Required*	Sequence[AgentRunResponseUpdate] A sequence of AgentRunResponseUpdate objects to combine.
output_format_type Required*	type[<xref:pydantic.main.BaseModel>] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
output_format_type	Optional Pydantic model type to parse the response text into structured data. Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._types.TAgentRunResponse>	

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any]</code> <code>None</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```

```

    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}

```

```

context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with `'_'`).

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
**kwargs	

Name	Description
	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

try_parse_value

If there is a value, does nothing, otherwise tries to parse the text into the value.

Python
<pre>try_parse_value(output_format_type: type[BaseModel]) -> None</pre>

Parameters

[Expand table](#)

Name	Description
<code>output_format_type</code> Required*	<code>type[<xref:pydantic.main.BaseModel>]</code>

Returns

[Expand table](#)

Type	Description
<code>None</code>	

Attributes

text

Get the concatenated text of all messages.

user_input_requests

Get all BaseUserInputRequest messages from the response.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

AgentRunResponseUpdate Class

Represents a single streaming response chunk from an Agent.

Initialize an AgentRunResponseUpdate.

Constructor

Python
<code>AgentRunResponseUpdate()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>contents</code>	Optional list of BaseContent items or dicts to include in the update. Default value: None
<code>text</code>	Optional text content of the update. Default value: None
<code>role</code>	The role of the author of the response update (Role, string, or dict Default value: None
<code>author_name</code>	Optional name of the author of the response update. Default value: None
<code>response_id</code>	Optional ID of the response of which this update is a part. Default value: None
<code>message_id</code>	Optional ID of the message of which this update is a part. Default value: None
<code>created_at</code>	Optional timestamp for the chat response update. Default value: None
<code>additional_properties</code>	Optional additional properties associated with the chat response update. Default value: None
<code>raw_representation</code>	Optional raw representation of the chat response update. Default value: None
<code>kwargs</code>	will be combined with additional_properties if provided.

Examples

Python

```
from agent_framework import AgentRunResponseUpdate, TextContent

# Create an agent run update
update = AgentRunResponseUpdate(
    contents=[TextContent(text="Processing...")],
    role="assistant",
    response_id="run_123",
)
print(update.text) # "Processing..."

# Check for user input requests
user_requests = update.user_input_requests

# Serialization - to_dict and from_dict
update_dict = update.to_dict()
# {'type': 'agent_run_response_update', 'contents': [{'type': 'text', 'text':
'Processing...'}],
#  'role': {'type': 'role', 'value': 'assistant'}, 'response_id': 'run_123'}
restored_update = AgentRunResponseUpdate.from_dict(update_dict)
print(restored_update.response_id) # "run_123"

# Serialization - to_json and from_json
update_json = update.to_json()
# '{"type": "agent_run_response_update", "contents": [{"type": "text", "text":
"Processing..."}], ...}'
restored_from_json = AgentRunResponseUpdate.from_json(update_json)
print(restored_from_json.text) # "Processing..."
```

Methods

 Expand table

<code>__init__</code>	Initialize an AgentRunResponseUpdate.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

`to_dict` Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

`to_json` Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an `AgentRunResponseUpdate`.

Python

```
__init__(*, contents: Sequence[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent | MutableMapping[str, Any]] | None = None, text: TextContent | str | None = None, role: Role | MutableMapping[str, Any] | str | None = None, author_name: str | None = None, response_id: str | None = None, message_id: str | None = None, created_at: str | None = None, additional_properties: MutableMapping[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
contents Required*	<code>Sequence[TextContent DataContent TextReasoningContent UriContent FunctionCallContent FunctionResultContent ErrorContent UsageContent HostedFileContent HostedVectorStoreContent FunctionApprovalRequestContent FunctionApprovalResponseContent MutableMapping[str, Any]] None</code>
text	<code>TextContent str None</code>

Name	Description
Required*	
role Required*	Role MutableMapping[str, Any] str None
author_name Required*	str None
response_id Required*	str None
message_id Required*	str None
created_at Required*	str None
additional_properties Required*	MutableMapping[str, Any] None
raw_representation Required*	Any None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
contents	Optional list of BaseContent items or dicts to include in the update. Default value: None
text	Optional text content of the update. Default value: None
role	The role of the author of the response update (Role, string, or dict) Default value: None
author_name	Optional name of the author of the response update. Default value: None
response_id	Optional ID of the response of which this update is a part. Default value: None
message_id	Optional ID of the message of which this update is a part. Default value: None

Name	Description
<code>created_at</code>	Optional timestamp for the chat response update. Default value: None
<code>additional_properties</code>	Optional additional properties associated with the chat response update. Default value: None
<code>raw_representation</code>	Optional raw representation of the chat response update. Default value: None
<code>kwargs</code>	will be combined with <code>additional_properties</code> if provided.

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) -> str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
```

```

function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
<code>from_json()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TCIass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
<code>JSONDecodeError</code>	If the JSON string is malformed.
<code>ValueError</code>	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str]</code> <code>None</code>
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>

Name	Description
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

text

Get the concatenated text of all `TextContent` objects in contents.

user_input_requests

Get all `BaseUserInputRequest` messages from the response.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```


AgentRunUpdateEvent Class

Event triggered when an agent is streaming messages.

Initialize the agent streaming event.

Constructor

Python
<code>AgentRunUpdateEvent()</code>

Parameters

 Expand table

Name	Description
<code>executor_id</code> Required*	
<code>data</code>	Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the agent streaming event.
<code>__new__</code>	

`__init__`

Initialize the agent streaming event.

Python
<code>__init__(executor_id: str, data: AgentRunResponseUpdate None = None)</code>

Parameters

 Expand table

Name	Description
executor_id Required*	str
data	AgentRunResponseUpdate None Default value: None

__new__

Python
<code>__new__(**kwargs)</code>

Attributes

data

Python
<code>data: AgentRunResponseUpdate None</code>

AgentThread Class

The Agent thread class, this can represent both a locally managed thread or a thread managed by the service.

An `AgentThread` maintains the conversation state and message history for an agent interaction. It can either use a service-managed thread (via `service_thread_id`) or a local message store (via `message_store`), but not both.

Initialize an `AgentThread`, do not use this method manually, always use: `agent.get_new_thread()`.

ⓘ **Note**

Either `service_thread_id` or `message_store` may be set, but not both.

Constructor

Python
<code>AgentThread()</code>

Parameters

 Expand table

Name	Description
<code>service_thread_id</code> Required*	The optional ID of the thread managed by the agent service.
<code>message_store</code> Required*	The optional ChatMessageStore implementation for managing chat messages.
<code>context_provider</code> Required*	The optional ContextProvider for the thread.

Keyword-Only Parameters

 Expand table

Name	Description
service_thread_id	Default value: None
message_store	Default value: None
context_provider	Default value: None

Examples

Python

```
from agent_framework import ChatAgent, ChatMessageStore
from agent_framework.openai import OpenAIChatClient

client = OpenAIChatClient(model="gpt-4o")

# Create agent with service-managed threads using a service_thread_id
service_agent = ChatAgent(name="assistant", client=client)
service_thread = await
service_agent.get_new_thread(service_thread_id="thread_abc123")

# Create agent with service-managed threads using conversation_id
conversation_agent = ChatAgent(name="assistant", client=client,
conversation_id="thread_abc123")
conversation_thread = await conversation_agent.get_new_thread()

# Create agent with custom message store factory
local_agent = ChatAgent(name="assistant", client=client,
chat_message_store_factory=ChatMessageStore)
local_thread = await local_agent.get_new_thread()

# Serialize and restore thread state
state = await local_thread.serialize()
restored_thread = await local_agent.deserialize_thread(state)
```

Methods

 Expand table

`__init__`

Initialize an AgentThread, do not use this method manually, always use:
`agent.get_new_thread()`.

 Note

Either `service_thread_id` or `message_store` may be set, but not both.

`__new__`

<code>deserialize</code>	Deserializes the state from a dictionary into a new <code>AgentThread</code> instance.
<code>on_new_messages</code>	Invoked when a new message has been contributed to the chat by any participant.
<code>serialize</code>	Serializes the current object's state.
<code>update_from_thread_state</code>	Deserializes the state from a dictionary into the thread properties.

`__init__`

Initialize an `AgentThread`, do not use this method manually, always use:

```
agent.get_new_thread().
```

⚠ Note

Either `service_thread_id` or `message_store` may be set, but not both.

Python

```
__init__(*, service_thread_id: str | None = None, message_store: ChatMessageStoreProtocol | None = None, context_provider: AggregateContextProvider | None = None) -> None
```

Parameters

 Expand table

Name	Description
<code>service_thread_id</code> Required*	<code>str</code> <code>None</code> The optional ID of the thread managed by the agent service.
<code>message_store</code> Required*	<code>ChatMessageStoreProtocol</code> <code>None</code> The optional <code>ChatMessageStore</code> implementation for managing chat messages.
<code>context_provider</code> Required*	<code>AggregateContextProvider</code> <code>None</code> The optional <code>ContextProvider</code> for the thread.

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>service_thread_id</code>	Default value: None
<code>message_store</code>	Default value: None
<code>context_provider</code>	Default value: None

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python

```
__new__(**kwargs)
```

deserialize

Deserializes the state from a dictionary into a new AgentThread instance.

Python

```
async classmethod deserialize(serialized_thread_state: MutableMapping[str, Any],  
*, message_store: ChatMessageStoreProtocol | None = None, **kwargs: Any) ->  
AgentThread
```

Parameters

[Expand table](#)

Name	Description
<code>serialized_thread_state</code> Required*	<code>MutableMapping[str, Any]</code> The serialized thread state as a dictionary.
<code>message_store</code> Required*	<code>ChatMessageStoreProtocol None</code>

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
message_store	Optional ChatMessageStoreProtocol to use for managing messages. If not provided, a new ChatMessageStore will be created if needed. Default value: None
**kwargs	Additional arguments for deserialization.

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_threads.TAgentThread>	A new AgentThread instance with properties set from the serialized state.

on_new_messages

Invoked when a new message has been contributed to the chat by any participant.

Python

```
async on_new_messages(new_messages: ChatMessage | Sequence[ChatMessage]) -> None
```

Parameters

[Expand table](#)

Name	Description
new_messages Required*	ChatMessage Sequence[ChatMessage] The new ChatMessage or sequence of ChatMessage objects to add to the thread.

Returns

[Expand table](#)

Type	Description
None	

serialize

Serializes the current object's state.

Python

```
async serialize(**kwargs: Any) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Arguments for serialization.

Returns

[Expand table](#)

Type	Description
dict[str, Any]	

update_from_thread_state

Deserializes the state from a dictionary into the thread properties.

Python


```
async update_from_thread_state(serialized_thread_state: MutableMapping[str, Any],
                               **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
serialized_thread_state Required*	<code>MutableMapping[str, Any]</code> The serialized thread state as a dictionary.
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional arguments for deserialization.

Returns

[Expand table](#)

Type	Description
<code>None</code>	

Attributes

is_initialized

Indicates if the thread is initialized.

This means either the `service_thread_id` or the `message_store` is set.

message_store

Gets the `ChatMessageStoreProtocol` used by this thread.

service_thread_id

Gets the ID of the current thread to support cases where the thread is owned by the agent service.

AggregateContextProvider Class

A ContextProvider that contains multiple context providers.

It delegates events to multiple context providers and aggregates responses from those events before returning. This allows you to combine multiple context providers into a single provider.

 Note

An AggregateContextProvider is created automatically when you pass a single context provider or a sequence of context providers to the agent constructor.

Initialize the AggregateContextProvider with context providers.

Constructor

Python
AggregateContextProvider()

Parameters

 Expand table

Name	Description
context_providers	The context provider(s) to add. Default value: None

Examples

Python
<pre>from agent_framework import AggregateContextProvider, ChatAgent # Create multiple context providers provider1 = CustomContextProvider1() provider2 = CustomContextProvider2() provider3 = CustomContextProvider3()</pre>

```
# Pass them to the agent - AggregateContextProvider is created automatically
agent = ChatAgent(chat_client=client, name="assistant", context_providers=
[provider1, provider2, provider3])

# Verify that an AggregateContextProvider was created
assert isinstance(agent.context_providers, AggregateContextProvider)

# Add additional providers to the agent
provider4 = CustomContextProvider4()
agent.context_providers.add(provider4)
```

Methods

[Expand table](#)

<code>__init__</code>	Initialize the AggregateContextProvider with context providers.
<code>__new__</code>	
<code>add</code>	Add a new context provider.
<code>invoked</code>	
<code>invoking</code>	
<code>thread_created</code>	

`__init__`

Initialize the AggregateContextProvider with context providers.

Python

```
__init__(context_providers: ContextProvider | Sequence[ContextProvider] | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>context_providers</code>	<code>ContextProvider</code> <code>Sequence[ContextProvider]</code> <code>None</code> The context provider(s) to add. Default value: None

Returns

[Expand table](#)

Type	Description
None	

`__new__`

Python
<code>__new__(**kwargs)</code>

`add`

Add a new context provider.

Python
<code>add(context_provider: ContextProvider) -> None</code>

Parameters

[Expand table](#)

Name	Description
<code>context_provider</code> Required*	ContextProvider The context provider to add.

Returns

[Expand table](#)

Type	Description
None	

`invoked`

Python
<code>async invoked(request_messages: ChatMessage Sequence[ChatMessage], response_messages: ChatMessage Sequence[ChatMessage] None = None,</code>

```
invoke_exception: Exception | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
request_messages Required*	ChatMessage Sequence[ChatMessage]
response_messages	ChatMessage Sequence[ChatMessage] None Default value: None
invoke_exception	Exception None Default value: None
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
None	

invoking

Python

```
async invoking(messages: ChatMessage | MutableSequence[ChatMessage], **kwargs:  
Any) -> Context
```

Parameters

[Expand table](#)

Name	Description
messages Required*	ChatMessage MutableSequence[ChatMessage]
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
Context	

thread_created

Python
<pre>async thread_created(thread_id: str None = None) -> None</pre>

Parameters

[Expand table](#)

Name	Description
thread_id	str None Default value: None

Returns

[Expand table](#)

Type	Description
None	

Attributes

DEFAULT_CONTEXT_PROMPT

Python
<pre>DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories when answering user questions:'</pre>

BaseAgent Class

Base class for all Agent Framework agents.

This class provides core functionality for agent implementations, including context providers, middleware support, and thread management.

 Note

BaseAgent cannot be instantiated directly as it doesn't implement the `run()`, `run_stream()`, and other methods required by AgentProtocol.

Use a concrete implementation like ChatAgent or create a subclass.

Initialize a BaseAgent instance.

Constructor

Python
<code>BaseAgent()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>id</code>	The unique identifier of the agent. If no id is provided, a new UUID will be generated. Default value: None
<code>name</code>	The name of the agent, can be None. Default value: None
<code>description</code>	The description of the agent. Default value: None
<code>context_providers</code>	The collection of multiple context providers to include during agent invocation. Default value: None
<code>middleware</code>	List of middleware to intercept agent and function invocations.

Name	Description
	Default value: None
<code>additional_properties</code>	Additional properties set on the agent. Default value: None
<code>kwargs</code>	Additional keyword arguments (merged into <code>additional_properties</code>).

Examples

Python

```
from agent_framework import BaseAgent, AgentThread, AgentRunResponse

# Create a concrete subclass that implements the protocol
class SimpleAgent(BaseAgent):
    async def run(self, messages=None, *, thread=None, **kwargs):
        # Custom implementation
        return AgentRunResponse(messages=[], response_id="simple-response")

    def run_stream(self, messages=None, *, thread=None, **kwargs):
        async def _stream():
            # Custom streaming implementation
            yield AgentRunResponseUpdate()

        return _stream()

# Now instantiate the concrete subclass
agent = SimpleAgent(name="my-agent", description="A simple agent implementation")

# Create with specific ID and additional properties
agent = SimpleAgent(
    id="custom-id-123",
    name="configured-agent",
    description="An agent with custom configuration",
    additional_properties={"version": "1.0", "environment": "production"},
)

# Access agent properties
print(agent.id) # Custom or auto-generated UUID
print(agent.display_name) # Returns name or id
```

Methods

<code>__init__</code>	Initialize a BaseAgent instance.
<code>__new__</code>	
<code>as_tool</code>	Create an AIFunction tool that wraps this agent.
<code>deserialize_thread</code>	Deserialize a thread from its serialized state.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_new_thread</code>	Return a new AgentThread instance that is compatible with the agent.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with '_').
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize a BaseAgent instance.

Python

```
__init__(*, id: str | None = None, name: str | None = None, description: str | None = None, context_providers: ContextProvider | Sequence[ContextProvider] | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext], Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None] | FunctionMiddleware | Callable[[FunctionInvocationContext], Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None] | ChatMiddleware | Callable[[ChatContext], Callable[[ChatContext], Awaitable[None]]],
```

```
Awaitable[None]] | Sequence[AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None = None, additional_properties: MutableMapping[str, Any] |
None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
id Required*	str None
name Required*	str None
description Required*	str None
context_providers Required*	ContextProvider Sequence[ContextProvider] None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] Sequence[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] None
additional_properties Required*	MutableMapping[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
id	The unique identifier of the agent. If no id is provided, a new UUID will be generated. Default value: None
name	The name of the agent, can be None. Default value: None
description	The description of the agent. Default value: None
context_providers	The collection of multiple context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
additional_properties	Additional properties set on the agent. Default value: None
kwargs	Additional keyword arguments (merged into additional_properties).

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

as_tool

Create an AIFunction tool that wraps this agent.

Python
<pre>as_tool(*, name: str None = None, description: str None = None, arg_name: str = 'task', arg_description: str None = None, stream_callback:</pre>

```
Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate],
Awaitable[None]] | None = None) -> AIFunction[BaseModel, str]
```

Parameters

 Expand table

Name	Description
name Required*	str None
description Required*	str None
arg_name Required*	str
arg_description Required*	str None
stream_callback Required*	Callable[[AgentRunResponseUpdate], None] Callable[[AgentRunResponseUpdate], Awaitable[None]] None

Keyword-Only Parameters

 Expand table

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task"). Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

[Expand table](#)

Type	Description
AIFunction [<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

[Expand table](#)

Type	Description
TypeError	If the agent does not implement AgentProtocol.
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
from agent_framework import ChatAgent

# Create an agent
agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

# Convert the agent to a tool
research_tool = agent.as_tool()

# Use the tool with another agent
coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

[Expand table](#)

Name	Description
serialized_thread Required*	Any The serialized thread data.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Additional keyword arguments.

Returns

[Expand table](#)

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value	

Name	Description
Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) -> str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
```

```

function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Return a new AgentThread instance that is compatible with the agent.

Python
<code>get_new_thread(**kwargs: Any) -> AgentThread</code>

Parameters

[Expand table](#)

Name	Description
<code>kwargs</code> Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>kwargs</code>	Additional keyword arguments passed to AgentThread.

Returns

[Expand table](#)

Type	Description
AgentThread	A new AgentThread instance configured with the agent's context provider.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	

Name	Description
	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

display_name

Returns the display name of the agent.

This is the name if present, otherwise the id.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

BaseAnnotation Class

Base class for all AI Annotation types.

Initialize BaseAnnotation.

Constructor

Python
BaseAnnotation()

Keyword-Only Parameters

[Expand table](#)

Name	Description
annotated_regions	A list of regions that have been annotated. Can be region objects or dicts. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content from an underlying implementation. Default value: None
**kwargs	Additional keyword arguments (merged into additional_properties).

Methods

[Expand table](#)

__init__	Initialize BaseAnnotation.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize BaseAnnotation.

Python

```
__init__(*, annotated_regions: list[TextSpanRegion] | list[MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>annotated_regions</code> Required*	<code>list[TextSpanRegion] list[MutableMapping[str, Any]] None</code>
<code>additional_properties</code> Required*	<code>dict[str, Any] None</code>
<code>raw_representation</code> Required*	<code>Any None</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
annotated_regions	A list of regions that have been annotated. Can be region objects or dicts. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content from an underlying implementation. Default value: None
**kwargs	Additional keyword arguments (merged into additional_properties).

Returns

 Expand table

Type	Description
None	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}

```

```

function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
<code>JSONDecodeError</code>	If the JSON string is malformed.
<code>ValueError</code>	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python
<pre>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
    'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

BaseChatClient Class

Base class for chat clients.

This abstract base class provides core functionality for chat client implementations, including middleware support, message preparation, and tool normalization.

 Note

BaseChatClient cannot be instantiated directly as it's an abstract base class.

Subclasses must implement `_inner_get_response()` and `_inner_get_streaming_response()`.

Initialize a BaseChatClient instance.

Constructor

Python
<code>BaseChatClient()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>middleware</code>	Middleware for the client. Default value: None
<code>additional_properties</code>	Additional properties for the client. Default value: None
<code>kwargs</code>	Additional keyword arguments (merged into <code>additional_properties</code>).

Examples

Python
<pre>from agent_framework import BaseChatClient, ChatResponse, ChatMessage from collections.abc import AsyncIterable</pre>

```

class CustomChatClient(BaseChatClient):
    async def _inner_get_response(self, *, messages, chat_options, **kwargs):
        # Your custom implementation
        return ChatResponse(
            messages=[ChatMessage(role="assistant", text="Hello!")],
            response_id="custom-response"
        )

    async def _inner_get_streaming_response(self, *, messages, chat_options,
**kwargs):
        # Your custom streaming implementation
        from agent_framework import ChatResponseUpdate

        yield ChatResponseUpdate(role="assistant", contents=[{"type": "text",
"text": "Hello!"}])

# Create an instance of your custom client
client = CustomChatClient()

# Use the client to get responses
response = await client.get_response("Hello, how are you?")

```

Methods

 Expand table

<code>__init__</code>	Initialize a BaseChatClient instance.
<code>__new__</code>	
<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.

<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize a `BaseChatClient` instance.

Python

```
__init__(*, middleware: ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | list[ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]]] | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
middleware Required*	ChatMiddleware Callable [[ChatContext , Callable [[ChatContext , Awaitable [None]]], Awaitable [None]]] FunctionMiddleware Callable [[FunctionInvocationContext , Callable [[FunctionInvocationContext , Awaitable [None]]], Awaitable [None]]] list [ChatMiddleware Callable [[ChatContext , Callable [[ChatContext , Awaitable [None]]], Awaitable [None]]] FunctionMiddleware Callable [[FunctionInvocationContext , Callable [[FunctionInvocationContext , Awaitable [None]]], Awaitable [None]]], Awaitable [None]]] None
additional_properties Required*	dict [str , Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
middleware	Middleware for the client. Default value: None
additional_properties	Additional properties for the client. Default value: None
kwargs	Additional keyword arguments (merged into additional_properties).

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str, int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

[Expand table](#)

Name	Description
id Required*	str None
name Required*	str None
description Required*	str None
instructions	str None

Name	Description
Required*	
chat_message_store_factory Required*	Callable [[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list [ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable [[AgentRunContext , Callable [[AgentRunContext , Awaitable [None]]], Awaitable [None]]] FunctionMiddleware Callable [[FunctionInvocationContext , Callable [[FunctionInvocationContext , Awaitable [None]]], Awaitable [None]]] ChatMiddleware Callable [[ChatContext , Callable [[ChatContext , Awaitable [None]]], Awaitable [None]]] list [AgentMiddleware Callable [[AgentRunContext , Callable [[AgentRunContext , Awaitable [None]]], Awaitable [None]]] FunctionMiddleware Callable [[FunctionInvocationContext , Callable [[FunctionInvocationContext , Awaitable [None]]], Awaitable [None]]] ChatMiddleware Callable [[ChatContext , Callable [[ChatContext , Awaitable [None]]], Awaitable [None]]]]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict [str int , float] None
max_tokens Required*	int None
metadata Required*	dict [str , Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type [< xref:pydantic.main.BaseModel >] None
seed Required*	int None

Name	Description
stop Required*	<code>str</code> <code>Sequence[str]</code> <code>None</code>
store Required*	<code>bool</code> <code>None</code>
temperature Required*	<code>float</code> <code>None</code>
tool_choice Required*	<code>ToolMode</code> <code>Literal['auto', 'required', 'none']</code> <code>dict[str, typing.Any]</code> <code>None</code>
tools Required*	<code>ToolProtocol</code> <code>Callable[[...], Any]</code> <code>MutableMapping[str, Any]</code> <code>Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]]</code> <code>None</code>
top_p Required*	<code>float</code> <code>None</code>
user Required*	<code>str</code> <code>None</code>
additional_chat_options Required*	<code>dict[str, Any]</code> <code>None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: <code>None</code>
name	The name of the agent. Default value: <code>None</code>
description	A brief description of the agent's purpose. Default value: <code>None</code>
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: <code>None</code>

Name	Description
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request.

Name	Description
	Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

 Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
    temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any] None</code>

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: - Simple injection: <code>{"<type>": {"<parameter>": value}}</code> - Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code>

Name	Description
	Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
```

```

my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

 Expand table

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None

Name	Description
stop Required*	<code>str</code> <code>Sequence[str]</code> <code>None</code>
store Required*	<code>bool</code> <code>None</code>
temperature Required*	<code>float</code> <code>None</code>
tool_choice Required*	<code>ToolMode</code> <code>Literal['auto', 'required', 'none']</code> <code>dict[str, typing.Any]</code> <code>None</code>
tools Required*	<code>ToolProtocol</code> <code>Callable[[...], Any]</code> <code>MutableMapping[str, Any]</code> <code>Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]]</code> <code>None</code>
top_p Required*	<code>float</code> <code>None</code>
user Required*	<code>str</code> <code>None</code>
additional_properties Required*	<code>dict[str, Any]</code> <code>None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: <code>None</code>
frequency_penalty	The frequency penalty to use. Default value: <code>None</code>
logit_bias	The logit bias to use. Default value: <code>None</code>
max_tokens	The maximum number of tokens to generate. Default value: <code>None</code>
metadata	Additional metadata to include in the request. Default value: <code>None</code>

Name	Description
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

 Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

 Expand table

Name	Description
messages Required*	<code>str ChatMessage list[str] list[ChatMessage]</code> The message or messages to send to the model.
allow_multiple_tool_calls Required*	<code>bool None</code>
frequency_penalty Required*	<code>float None</code>
logit_bias Required*	<code>dict[str int, float] None</code>
max_tokens Required*	<code>int None</code>
metadata Required*	<code>dict[str, Any] None</code>
model_id	<code>str None</code>

Name	Description
Required*	
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use.

Name	Description
	Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
AsyncIterable [ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python
<pre>service_url() -> str</pre>

Returns

[Expand table](#)

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python
<pre>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str]</code> <code>None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python
<pre>DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}</pre>

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'unknown'
```

BaseContent Class

Represents content used by AI services.

Initialize BaseContent.

Constructor

Python
<code>BaseContent()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>annotations</code>	Optional annotations associated with the content. Can be annotation objects or dicts. Default value: None
<code>additional_properties</code>	Optional additional properties associated with the content. Default value: None
<code>raw_representation</code>	Optional raw representation of the content from an underlying implementation. Default value: None
<code>**kwargs</code>	Additional keyword arguments (merged into <code>additional_properties</code>).

Methods

 Expand table

<code>__init__</code>	Initialize BaseContent.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection

	to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code> . All the same serialization rules apply as in <code>to_dict()</code> , including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize BaseContent.

Python

```
__init__(*, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
annotations Required*	<code>Sequence[CitationAnnotation MutableMapping[str, Any]] None</code>
additional_properties Required*	<code>dict[str, Any] None</code>
raw_representation Required*	<code>Any None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
annotations	Optional annotations associated with the content. Can be annotation objects or dicts. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content from an underlying implementation. Default value: None
**kwargs	Additional keyword arguments (merged into additional_properties).

Returns

[Expand table](#)

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"
```

```

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
<code>JSONDecodeError</code>	If the JSON string is malformed.
<code>ValueError</code>	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python
<pre>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

annotations

Optional annotations associated with the content.

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
    'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

Case Class

Runtime wrapper combining a switch-case predicate with its target.

Each *Case* couples a boolean predicate with the executor that should handle the message when the predicate evaluates to *True*. The runtime keeps this lightweight container separate from the serialisable *SwitchCaseEdgeGroupCase* so that execution can operate with live callables without polluting persisted state.

Constructor

Python
<code>Case()</code>

Parameters

 Expand table

Name	Description
condition Required*	
target Required*	

Examples

Python
<pre>class JsonExecutor(Executor): def __init__(self) -> None: super().__init__(id="json", defer_discovery=True) processor = JsonExecutor() case = Case(condition=lambda payload: payload["kind"] == "json", target=processor) assert case.target.id == "json"</pre>

Methods

[Expand table](#)[__init__](#)[__new__](#)

[__init__](#)

Python

```
__init__(condition: Callable[[Any], bool], target: Executor | str) -> None
```

Parameters

[Expand table](#)

Name	Description
condition Required*	Callable[[Any], bool]
target Required*	Executor str

Returns

[Expand table](#)

Type	Description
None	

[__new__](#)

Python

```
__new__(**kwargs)
```

Attributes

condition

Python

condition: Callable[[Any], bool]

target

Python

target: Executor | str

ChatAgent Class

A Chat Client Agent.

This is the primary agent implementation that uses a chat client to interact with language models. It supports tools, context providers, middleware, and both streaming and non-streaming responses.

Initialize a ChatAgent instance.

ⓘ **Note**

The set of parameters from frequency_penalty to request_kwargs are used to call the chat client. They can also be passed to both run methods.

When both are set, the ones passed to the run methods take precedence.

Constructor

Python
<code>ChatAgent()</code>

Parameters

 Expand table

Name	Description
<code>chat_client</code> Required*	The chat client to use for the agent.
<code>instructions</code>	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None

Keyword-Only Parameters

 Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	The collection of multiple context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
conversation_id	The conversation ID for service-managed threads. Cannot be used together with chat_message_store_factory. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. This overrides the model_id set in the chat client if it contains one. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: auto
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Examples

Basic usage:

Python

```
from agent_framework import ChatAgent
from agent_framework.clients import OpenAIChatClient

# Create a basic chat agent
client = OpenAIChatClient(model_id="gpt-4")
agent = ChatAgent(chat_client=client, name="assistant", description="A helpful assistant")

# Run the agent with a simple message
```

```
response = await agent.run("Hello, how are you?")
print(response.text)
```

With tools and streaming:

Python

```
# Create an agent with tools and instructions
def get_weather(location: str) -> str:
    return f"The weather in {location} is sunny."

agent = ChatAgent(
    chat_client=client,
    name="weather-agent",
    instructions="You are a weather assistant.",
    tools=get_weather,
    temperature=0.7,
    max_tokens=500,
)

# Use streaming responses
async for update in agent.run_stream("What's the weather in Paris?"):
    print(update.text, end="")
```

With additional provider specific options:

Python

```
agent = ChatAgent(
    chat_client=client,
    name="reasoning-agent",
    instructions="You are a reasoning assistant.",
    model_id="gpt-5",
    temperature=0.7,
    max_tokens=500,
    additional_chat_options={
        "reasoning": {"effort": "high", "summary": "concise"}
    }, # OpenAI Responses specific.
)

# Use streaming responses
async for update in agent.run_stream("How do you prove the pythagorean theorem?"):
    print(update.text, end="")
```

Methods

<code>__init__</code>	Initialize a ChatAgent instance. Note The set of parameters from <code>frequency_penalty</code> to <code>request_kwargs</code> are used to call the chat client. They can also be passed to both run methods. When both are set, the ones passed to the run methods take precedence.
<code>__new__</code>	
<code>as_mcp_server</code>	Create an MCP server from an agent instance. This function automatically creates a MCP server from an agent instance, it uses the provided arguments to configure the server and exposes the agent as a single MCP tool.
<code>as_tool</code>	Create an AIFunction tool that wraps this agent.
<code>deserialize_thread</code>	Deserialize a thread from its serialized state.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_new_thread</code>	Get a new conversation thread for the agent. If you supply a <code>service_thread_id</code>, the thread will be marked as service managed. If you don't supply a <code>service_thread_id</code> but have a <code>conversation_id</code> configured on the agent, that <code>conversation_id</code> will be used to create a service-managed thread. If you don't supply a <code>service_thread_id</code> but have a <code>chat_message_store_factory</code> configured on the agent, that factory will be used to create a message store for the thread and the thread will be managed locally.

When neither is present, the thread will be created without a service ID or message store. This will be updated based on usage when you run the agent with this thread. If you run with `store=True`, the response will include a `thread_id` and that will be set. Otherwise a message store is created from the default factory.

`run`

Run the agent with the given messages and options.

ⓘ Note

Since you won't always call `agent.run()` directly (it gets called through workflows), it is advised to set your default values for all the chat client parameters in the agent constructor.

If both parameters are used, the ones passed to the run methods take precedence.

`run_stream`

Stream the agent with the given messages and options.

ⓘ Note

Since you won't always call `agent.run_stream()` directly (it gets called through orchestration), it is advised to set your default values for all the chat client parameters in the agent constructor.

If both parameters are used, the ones passed to the run methods take precedence.

`to_dict`

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with `'_'`).

`to_json`

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Initialize a ChatAgent instance.

⚠ Note

The set of parameters from `frequency_penalty` to `request_kwargs` are used to call the chat client. They can also be passed to both run methods.

When both are set, the ones passed to the run methods take precedence.

Python

```
__init__(chat_client: ChatClientProtocol, instructions: str | None = None, *, id: str | None = None, name: str | None = None, description: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
chat_client Required*	ChatClientProtocol The chat client to use for the agent.

Name	Description
instructions	<code>str</code> <code>None</code> Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: <code>None</code>
id Required*	<code>str</code> <code>None</code>
name Required*	<code>str</code> <code>None</code>
description Required*	<code>str</code> <code>None</code>
chat_message_store_factory Required*	<code>Callable[[], ChatMessageStoreProtocol]</code> <code>None</code>
context_providers Required*	<code>ContextProvider</code> <code>list[ContextProvider]</code> <code>AggregateContextProvider</code> <code>None</code>
middleware Required*	<code>AgentMiddleware</code> <code>Callable[[AgentRunContext, Callable[[AgentRunContext, Awaitable[None]]], Awaitable[None]]</code> <code>FunctionMiddleware</code> <code>Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext, Awaitable[None]]], Awaitable[None]]</code> <code>ChatMiddleware</code> <code>Callable[[ChatContext, Callable[[ChatContext, Awaitable[None]]], Awaitable[None]]</code> <code>list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext, Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext, Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext, Awaitable[None]]], Awaitable[None]]]]</code> <code>None</code>
allow_multiple_tool_calls Required*	<code>bool</code> <code>None</code>
conversation_id Required*	<code>str</code> <code>None</code>
frequency_penalty Required*	<code>float</code> <code>None</code>
logit_bias Required*	<code>dict[str int, float]</code> <code>None</code>
max_tokens Required*	<code>int</code> <code>None</code>
metadata	<code>dict[str, Any]</code> <code>None</code>

Name	Description
Required*	
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
id	

Name	Description
	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	The collection of multiple context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
conversation_id	The conversation ID for service-managed threads. Cannot be used together with chat_message_store_factory. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. This overrides the model_id set in the chat client if it contains one. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the <code>chat_client</code> <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

[Expand table](#)

Type	Description
None	

Exceptions

[Expand table](#)

Type	Description
AgentInitializationError	If both <code>conversation_id</code> and <code>chat_message_store_factory</code> are provided.

__new__

Python

```
__new__(**kwargs)
```

as_mcp_server

Create an MCP server from an agent instance.

This function automatically creates a MCP server from an agent instance, it uses the provided arguments to configure the server and exposes the agent as a single MCP tool.

Python

```
as_mcp_server(*, server_name: str = 'Agent', version: str | None = None,
instructions: str | None = None, lifespan: Callable[[Server[Any, Any]],
AbstractAsyncContextManager[Any]] | None = None, **kwargs: Any) -> Server[Any,
Any]
```

Parameters

 Expand table

Name	Description
server_name Required*	str
version Required*	str None
instructions Required*	str None
lifespan Required*	Callable [[<xref:mcp.server.lowlevel.server.Server> [Any, Any]], AbstractAsyncContextManager [Any]] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
server_name	The name of the server. Default value: "Agent"
version	The version of the server. Default value: None
instructions	The instructions to use for the server. Default value: None
lifespan	The lifespan of the server. Default value: None
**kwargs	Any extra arguments to pass to the server creation.

Returns

[Expand table](#)

Type	Description
<xref:mcp.server.lowlevel.server.Server> Any , Any	The MCP server instance.

as_tool

Create an AIFunction tool that wraps this agent.

Python

```
as_tool(*, name: str | None = None, description: str | None = None, arg_name: str = 'task', arg_description: str | None = None, stream_callback: Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate], Awaitable[None]] | None = None) -> AIFunction[BaseModel, str]
```

Parameters

[Expand table](#)

Name	Description
name Required*	str None
description Required*	str None

Name	Description
arg_name Required*	str
arg_description Required*	str None
stream_callback Required*	Callable [[AgentRunResponseUpdate], None] Callable [[AgentRunResponseUpdate], Awaitable [None]] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task"). Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

[Expand table](#)

Type	Description
AIFunction [<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

[Expand table](#)

Type	Description
TypeError	If the agent does not implement AgentProtocol.
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
from agent_framework import ChatAgent

# Create an agent
agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

# Convert the agent to a tool
research_tool = agent.as_tool()

# Use the tool with another agent
coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

 Expand table

Name	Description
serialized_thread Required*	Any The serialized thread data.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Additional keyword arguments.

Returns

[Expand table](#)

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any]

Name	Description
	The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

```
Python
```



```

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected

```

```
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Get a new conversation thread for the agent.

If you supply a `service_thread_id`, the thread will be marked as service managed.

If you don't supply a `service_thread_id` but have a `conversation_id` configured on the agent, that `conversation_id` will be used to create a service-managed thread.

If you don't supply a `service_thread_id` but have a `chat_message_store_factory` configured on the agent, that factory will be used to create a message store for the thread and the thread will be managed locally.

When neither is present, the thread will be created without a service ID or message store. This will be updated based on usage when you run the agent with this thread. If you run with `store=True`, the response will include a `thread_id` and that will be set. Otherwise a message store is created from the default factory.

Python

```
get_new_thread(*, service_thread_id: str | None = None, **kwargs: Any) -> AgentThread
```

Parameters

[Expand table](#)

Name	Description
<code>service_thread_id</code> Required*	str None
<code>kwargs</code> Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>service_thread_id</code>	Optional service managed thread ID. Default value: None
<code>kwargs</code>	Not used at present.

Returns

[Expand table](#)

Type	Description
AgentThread	A new AgentThread instance.

run

Run the agent with the given messages and options.

ⓘ Note

Since you won't always call `agent.run()` directly (it gets called through workflows), it is advised to set your default values for all the chat client parameters in the agent constructor.

If both parameters are used, the ones passed to the run methods take precedence.

Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | list[ToolProtocol | Callable[..., Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> AgentRunResponse
```

Parameters

 Expand table

Name	Description
messages	<code>str</code> <code>ChatMessage</code> <code>list[str]</code> <code>list[ChatMessage]</code> <code>None</code> The messages to process. Default value: <code>None</code>
thread Required*	<code>AgentThread</code> <code>None</code>
allow_multiple_tool_calls Required*	<code>bool</code> <code>None</code>
frequency_penalty Required*	<code>float</code> <code>None</code>
logit_bias Required*	<code>dict[str int, float]</code> <code>None</code>
max_tokens Required*	<code>int</code> <code>None</code>
metadata Required*	<code>dict[str, Any]</code> <code>None</code>
model_id Required*	<code>str</code> <code>None</code>
presence_penalty Required*	<code>float</code> <code>None</code>
response_format Required*	<code>type[<xref:pydantic.main.BaseModel>]</code> <code>None</code>
seed Required*	<code>int</code> <code>None</code>
stop Required*	<code>str</code> <code>Sequence[str]</code> <code>None</code>
store Required*	<code>bool</code> <code>None</code>
temperature Required*	<code>float</code> <code>None</code>
tool_choice Required*	<code>ToolMode</code> <code>Literal['auto', 'required', 'none']</code> <code>dict[str, typing.Any]</code> <code>None</code>

Name	Description
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] list[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
thread	The thread to use for the agent. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	Additional properties to include in the request. Use this field for provider-specific parameters. Default value: None
kwargs	Additional keyword arguments for the agent. Will only be passed to functions that are called.

Returns

[Expand table](#)

Type	Description
AgentRunResponse	An AgentRunResponse containing the agent's response.

run_stream

Stream the agent with the given messages and options.

ⓘ Note

Since you won't always call `agent.run_stream()` directly (it gets called

through orchestration), it is advised to set your default values for

all the chat client parameters in the agent constructor.

If both parameters are used, the ones passed to the run methods take precedence.

Python

```
run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | list[ToolProtocol | Callable[...], Any] | MutableMapping[str, Any] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> AsyncIterable[AgentRunResponseUpdate]
```

Parameters

 Expand table

Name	Description
messages	<code>str ChatMessage list[str] list[ChatMessage] None</code> The messages to process. Default value: None
thread Required*	<code>AgentThread None</code>
allow_multiple_tool_calls Required*	<code>bool None</code>
frequency_penalty Required*	<code>float None</code>
logit_bias Required*	<code>dict[str int, float] None</code>
max_tokens Required*	<code>int None</code>
metadata Required*	<code>dict[str, Any] None</code>

Name	Description
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type [<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal ['auto', 'required', 'none'] dict [str, typing.Any] None
tools Required*	ToolProtocol Callable [..., Any] MutableMapping [str, Any] list [ToolProtocol Callable [..., Any] MutableMapping [str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict [str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
thread	The thread to use for the agent. Default value: None

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	Additional properties to include in the request. Use this field for provider-specific parameters.

Name	Description
	Default value: None
kwargs	Any additional keyword arguments. Will only be passed to functions that are called.

Returns

[Expand table](#)

Type	Description
<code>AsyncIterable[AgentRunResponseUpdate]</code>	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python
<pre>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

display_name

Returns the display name of the agent.

This is the name if present, otherwise the id.

AGENT_PROVIDER_NAME

Python
AGENT_PROVIDER_NAME: ClassVar[str] = 'microsoft.agent_framework'

DEFAULT_EXCLUDE

--

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

additional_properties

Python

```
additional_properties: dict[str, Any]
```

middleware

Python

```
middleware: list[AgentMiddleware | Callable[[AgentRunContext,  
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |  
FunctionMiddleware | Callable[[FunctionInvocationContext,  
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |  
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],  
Awaitable[None]]] | None
```

ChatClientProtocol Class

A protocol for a chat client that can generate responses.

This protocol defines the interface that all chat clients must implement, including methods for generating both streaming and non-streaming responses.

⚠ Note

Protocols use structural subtyping (duck typing). Classes don't need to explicitly inherit from this protocol to be considered compatible.

Constructor

Python

```
ChatClientProtocol()
```

Examples

Python

```
from agent_framework import ChatClientProtocol, ChatResponse, ChatMessage

# Any class implementing the required methods is compatible
class CustomChatClient:
    @property
    def additional_properties(self) -> dict[str, Any]:
        return {}

    async def get_response(self, messages, **kwargs):
        # Your custom implementation
        return ChatResponse(messages=[], response_id="custom")

    def get_streaming_response(self, messages, **kwargs):
        async def _stream():
            from agent_framework import ChatResponseUpdate

            yield ChatResponseUpdate()

        return _stream()
```



```
# Verify the instance satisfies the protocol
client = CustomChatClient()
assert isinstance(client, ChatClientProtocol)
```

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>get_response</code>	Send input and return the response.
<code>get_streaming_response</code>	Send input messages and stream the response.

`__init__`

Python

```
__init__(*args, **kwargs)
```

`__new__`

Python

```
__new__(**kwargs)
```

`get_response`

Send input and return the response.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
```

```
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

[Expand table](#)

Name	Description
messages Required*	<code>str</code> <code>ChatMessage</code> <code>list[str]</code> <code>list[ChatMessage]</code> The sequence of input messages to send.
frequency_penalty Required*	<code>float</code> <code>None</code>
logit_bias Required*	<code>dict[str int, float]</code> <code>None</code>
max_tokens Required*	<code>int</code> <code>None</code>
metadata Required*	<code>dict[str, Any]</code> <code>None</code>
model_id Required*	<code>str</code> <code>None</code>
presence_penalty Required*	<code>float</code> <code>None</code>
response_format Required*	<code>type[<xref:pydantic.main.BaseModel>]</code> <code>None</code>
seed Required*	<code>int</code> <code>None</code>
stop Required*	<code>str</code> <code>Sequence[str]</code> <code>None</code>
store Required*	<code>bool</code> <code>None</code>
temperature Required*	<code>float</code> <code>None</code>
tool_choice Required*	<code>ToolMode</code> <code>Literal['auto', 'required', 'none']</code> <code>dict[str, typing.Any]</code> <code>None</code>
tools Required*	<code>ToolProtocol</code> <code>Callable[[...], Any]</code> <code>MutableMapping[str, Any]</code> <code>Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]]</code> <code>None</code>

Name	Description
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None

Name	Description
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Default value: None
kwargs	Any additional keyword arguments. Will only be passed to functions that are called.

Returns

 Expand table

Type	Description
ChatResponse	The response messages generated by the client.

Exceptions

 Expand table

Type	Description
ValueError	If the input message sequence is <code>None</code>

get_streaming_response

Send input messages and stream the response.

Python

```
get_streaming_response(messages: str | ChatMessage | list[str] |  
list[ChatMessage], *, frequency_penalty: float | None = None, logit_bias: dict[str
```

```
| int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> AsyncIterable[ChatResponseUpdate]
```

Parameters

 Expand table

Name	Description
messages Required*	<code>str</code> <code>ChatMessage</code> <code>list[str]</code> <code>list[ChatMessage]</code> The sequence of input messages to send.
frequency_penalty Required*	<code>float</code> <code>None</code>
logit_bias Required*	<code>dict[str int, float]</code> <code>None</code>
max_tokens Required*	<code>int</code> <code>None</code>
metadata Required*	<code>dict[str, Any]</code> <code>None</code>
model_id Required*	<code>str</code> <code>None</code>
presence_penalty Required*	<code>float</code> <code>None</code>
response_format Required*	<code>type[<xref:pydantic.main.BaseModel>]</code> <code>None</code>
seed Required*	<code>int</code> <code>None</code>
stop Required*	<code>str</code> <code>Sequence[str]</code> <code>None</code>
store Required*	<code>bool</code> <code>None</code>
temperature	<code>float</code> <code>None</code>

Name	Description
Required*	
tool_choice Required*	ToolMode Literal ['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable [..., Any] MutableMapping [str, Any] Sequence [ToolProtocol Callable [..., Any] MutableMapping [str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict [str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use.

Name	Description
	Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Default value: None
kwargs	Any additional keyword arguments. Will only be passed to functions that are called.

Returns

[Expand table](#)

Type	Description
AsyncIterable[ChatResponseUpdate]	

Exceptions

[Expand table](#)

Type	Description
ValueError	If the input message sequence is <code>None</code>

Attributes

additional_properties

Get additional properties associated with the client.

ChatContext Class

Context object for chat middleware invocations.

This context is passed through the chat middleware pipeline and contains all information about the chat request.

Initialize the ChatContext.

Constructor

Python
<code>ChatContext()</code>

Parameters

 Expand table

Name	Description
chat_client Required*	The chat client being invoked.
messages Required*	The messages being sent to the chat client.
chat_options Required*	The options for the chat request.
is_streaming	Whether this is a streaming invocation. Default value: False
metadata	Metadata dictionary for sharing data between chat middleware. Default value: None
result	Chat execution result. Default value: None
terminate	A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	Additional keyword arguments passed to the chat client. Default value: None

Examples

Python

```
from agent_framework import ChatMiddleware, ChatContext

class TokenCounterMiddleware(ChatMiddleware):
    async def process(self, context: ChatContext, next):
        print(f"Chat client: {context.chat_client.__class__.__name__}")
        print(f"Messages: {len(context.messages)}")
        print(f"Model: {context.chat_options.model_id}")

        # Store metadata
        context.metadata["input_tokens"] = self.count_tokens(context.messages)

        # Continue execution
        await next(context)

        # Access result and count output tokens
        if context.result:
            context.metadata["output_tokens"] = self.count_tokens(context.result)
```

Methods

 Expand table

<code>__init__</code>	Initialize the ChatContext.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

to_json Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize the ChatContext.

Python

```
__init__(chat_client: ChatClientProtocol, messages: MutableSequence[ChatMessage],
chat_options: ChatOptions, is_streaming: bool = False, metadata: dict[str, Any] |
None = None, result: ChatResponse | AsyncIterable[ChatResponseUpdate] | None =
None, terminate: bool = False, kwargs: dict[str, Any] | None = None) -> None
```

Parameters

 Expand table

Name	Description
chat_client Required*	ChatClientProtocol The chat client being invoked.
messages Required*	MutableSequence[ChatMessage] The messages being sent to the chat client.
chat_options Required*	ChatOptions The options for the chat request.
is_streaming	bool Whether this is a streaming invocation. Default value: False
metadata	dict[str, Any] None Metadata dictionary for sharing data between chat middleware. Default value: None
result	ChatResponse AsyncIterable[ChatResponseUpdate] None Chat execution result. Default value: None
terminate	bool A flag indicating whether to terminate execution after current middleware.

Name	Description
	Default value: False
kwargs	<code>dict[str, Any] None</code> Additional keyword arguments passed to the chat client. Default value: None

Returns

 Expand table

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
<code>value</code> Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) -> str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```

```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	

Name	Description
	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

`chat_client`

The chat client being invoked.

`messages`

The messages being sent to the chat client.

`chat_options`

The options for the chat request.

`is_streaming`

Whether this is a streaming invocation.

`metadata`

Metadata dictionary for sharing data between chat middleware.

result

Chat execution result. Can be observed after calling `next()` to see the actual execution result or can be set to override the execution result. For non-streaming: should be `ChatResponse`. For streaming: should be `AsyncIterable[ChatResponseUpdate]`.

terminate

A flag indicating whether to terminate execution after current middleware. When set to `True`, execution will stop as soon as control returns to framework.

kwargs

Additional keyword arguments passed to the chat client.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'chat_client', 'result'}
```

ChatMessage Class

Represents a chat message.

Initialize ChatMessage.

Constructor

Python
<code>ChatMessage()</code>

Parameters

[Expand table](#)

Name	Description
role Required*	The role of the author of the message (Role, string, or dict).

Keyword-Only Parameters

[Expand table](#)

Name	Description
text	Optional text content of the message. Default value: None
contents	Optional list of BaseContent items or dicts to include in the message. Default value: None
author_name	Optional name of the author of the message. Default value: None
message_id	Optional ID of the chat message. Default value: None
additional_properties	Optional additional properties associated with the chat message. Additional properties are used within Agent Framework, they are not sent to services. Default value: None
raw_representation	Optional raw representation of the chat message.

Name	Description
	Default value: None
kwargs	will be combined with additional_properties if provided.

Examples

Python

```
from agent_framework import ChatMessage, TextContent

# Create a message with text
user_msg = ChatMessage(role="user", text="What's the weather?")
print(user_msg.text) # "What's the weather?"

# Create a message with role string
system_msg = ChatMessage(role="system", text="You are a helpful assistant.")

# Create a message with contents
assistant_msg = ChatMessage(
    role="assistant",
    contents=[TextContent(text="The weather is sunny!")],
)
print(assistant_msg.text) # "The weather is sunny!"

# Serialization - to_dict and from_dict
msg_dict = user_msg.to_dict()
# {'type': 'chat_message', 'role': {'type': 'role', 'value': 'user'},
#  'contents': [{'type': 'text', 'text': "What's the weather?"}]},
'additional_properties': {}}
restored_msg = ChatMessage.from_dict(msg_dict)
print(restored_msg.text) # "What's the weather?"

# Serialization - to_json and from_json
msg_json = user_msg.to_json()
# '{"type": "chat_message", "role": {"type": "role", "value": "user"}, "contents":
[...], ...}'
restored_from_json = ChatMessage.from_json(msg_json)
print(restored_from_json.role.value) # "user"
```

Methods

 Expand table

<code>__init__</code>	Initialize ChatMessage.
<code>__new__</code>	

<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize ChatMessage.

Python

```
__init__(role: Role | Literal['system', 'user', 'assistant', 'tool'], *, text: str, author_name: str | None = None, message_id: str | None = None, additional_properties: MutableMapping[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
<code>role</code> Required*	The role of the author of the message (Role, string, or dict).

Keyword-Only Parameters

 Expand table

Name	Description
text	Optional text content of the message. Default value: None
contents	Optional list of BaseContent items or dicts to include in the message. Default value: None
author_name	Optional name of the author of the message. Default value: None
message_id	Optional ID of the chat message. Default value: None
additional_properties	Optional additional properties associated with the chat message. Additional properties are used within Agent Framework, they are not sent to services. Default value: None
raw_representation	Optional raw representation of the chat message. Default value: None
kwargs	will be combined with additional_properties if provided.

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
```

```

# In real implementation, this would call a weather API
return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution

contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific).

Name	Description
	Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
<code>JSONDecodeError</code>	If the JSON string is malformed.
<code>ValueError</code>	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

text

Returns the text content of the message.

Remarks: This property concatenates the text of all `TextContent` objects in `Contents`.

role

The role of the author of the message.

contents

The chat message content items.

author_name

The name of the author of the message.

message_id

The ID of the chat message.

additional_properties

Any additional properties associated with the chat message. Additional properties are used within Agent Framework, they are not sent to services.

raw_representation

The raw representation of the chat message from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```


ChatMessageStore Class

An in-memory implementation of ChatMessageStoreProtocol that stores messages in a list.

This implementation provides a simple, list-based storage for chat messages with support for serialization and deserialization. It implements all the required methods of the `ChatMessageStoreProtocol` protocol.

The store maintains messages in memory and provides methods to serialize and deserialize the state for persistence purposes.

Create a ChatMessageStore for use in a thread.

Constructor

Python
<code>ChatMessageStore()</code>

Parameters

 Expand table

Name	Description
<code>messages</code>	The messages to store. Default value: None

Examples

Python
<pre>from agent_framework import ChatMessageStore, ChatMessage # Create an empty store store = ChatMessageStore() # Add messages message = ChatMessage(role="user", content="Hello") await store.add_messages([message]) # Retrieve messages messages = await store.list_messages()</pre>

```
# Serialize for persistence
state = await store.serialize()

# Deserialize from saved state
restored_store = await ChatMessageStore.deserialize(state)
```

Methods

 Expand table

<code>__init__</code>	Create a ChatMessageStore for use in a thread.
<code>__new__</code>	
<code>add_messages</code>	Add messages to the store.
<code>deserialize</code>	Create a new ChatMessageStore instance from serialized state data.
<code>list_messages</code>	Get all messages from the store in chronological order.
<code>serialize</code>	Serialize the current store state for persistence.
<code>update_from_state</code>	Update the current ChatMessageStore instance from serialized state data.

`__init__`

Create a ChatMessageStore for use in a thread.

Python

```
__init__(messages: Sequence[ChatMessage] | None = None)
```

Parameters

 Expand table

Name	Description
<code>messages</code>	<code>Sequence[ChatMessage]</code> <code>None</code> The messages to store. Default value: None

`__new__`

Python

```
__new__(**kwargs)
```

add_messages

Add messages to the store.

Python

```
async def add_messages(messages: Sequence[ChatMessage]) -> None
```

Parameters

[Expand table](#)

Name	Description
messages Required*	Sequence[ChatMessage] Sequence of ChatMessage objects to add to the store.

Returns

[Expand table](#)

Type	Description
None	

deserialize

Create a new ChatMessageStore instance from serialized state data.

Python

```
async classmethod deserialize(serialized_store_state: MutableMapping[str, Any],  
                               **kwargs: Any) -> TChatMessageStore
```

Parameters

[Expand table](#)

Name	Description
serialized_store_state	MutableMapping[str, Any]

Name	Description
Required*	Previously serialized state data containing messages.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional arguments for deserialization.

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._threads.TChatMessageStore>	A new ChatMessageStore instance populated with messages from the serialized state.

list_messages

Get all messages from the store in chronological order.

Python
<code>async list_messages() -> list[ChatMessage]</code>

Returns

[Expand table](#)

Type	Description
list[ChatMessage]	List of ChatMessage objects, ordered from oldest to newest.

serialize

Serialize the current store state for persistence.

--

Python

```
async serialize(**kwargs: Any) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional arguments for serialization.

Returns

[Expand table](#)

Type	Description
dict[str, Any]	Serialized state data that can be used with <code>deserialize_state</code> .

update_from_state

Update the current `ChatMessageStore` instance from serialized state data.

Python

```
async update_from_state(serialized_store_state: MutableMapping[str, Any],  
                        **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
serialized_store_state	MutableMapping[str, Any]

Name	Description
Required*	Previously serialized state data containing messages.
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
**kwargs	Additional arguments for deserialization.

Returns

 Expand table

Type	Description
None	

ChatMessageStoreProtocol Class

Defines methods for storing and retrieving chat messages associated with a specific thread.

Implementations of this protocol are responsible for managing the storage of chat messages, including handling large volumes of data by truncating or summarizing messages as necessary.

Constructor

Python

```
ChatMessageStoreProtocol()
```

Examples

Python

```
from agent_framework import ChatMessage

class MyMessageStore:
    def __init__(self):
        self._messages = []

    async def list_messages(self) -> list[ChatMessage]:
        return self._messages

    async def add_messages(self, messages: Sequence[ChatMessage]) -> None:
        self._messages.extend(messages)

    @classmethod
    async def deserialize(cls, serialized_store_state, **kwargs):
        store = cls()
        store._messages = serialized_store_state.get("messages", [])
        return store

    async def update_from_state(self, serialized_store_state, **kwargs) -> None:
        self._messages = serialized_store_state.get("messages", [])

    async def serialize(self, **kwargs):
        return {"messages": self._messages}

# Use the custom store
store = MyMessageStore()
```

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>add_messages</code>	Adds messages to the store.
<code>deserialize</code>	Creates a new instance of the store from previously serialized state. This method, together with <code>serialize()</code> can be used to save and load messages from a persistent store if this store only has messages in memory.
<code>list_messages</code>	Gets all the messages from the store that should be used for the next agent invocation. Messages are returned in ascending chronological order, with the oldest message first. If the messages stored in the store become very large, it is up to the store to truncate, summarize or otherwise limit the number of messages returned. When using implementations of <code>ChatMessageStoreProtocol</code>, a new one should be created for each thread since they may contain state that is specific to a thread.
<code>serialize</code>	Serializes the current object's state. This method, together with <code>deserialize()</code> can be used to save and load messages from a persistent store if this store only has messages in memory.
<code>update_from_state</code>	Update the current ChatMessageStore instance from serialized state data.

`__init__`

Python

```
__init__(*args, **kwargs)
```

`__new__`

Python

```
__new__(**kwargs)
```

`add_messages`

Adds messages to the store.

Python

```
async add_messages(messages: Sequence[ChatMessage]) -> None
```

Parameters

[Expand table](#)

Name	Description
messages Required*	Sequence[ChatMessage] The sequence of ChatMessage objects to add to the store.

Returns

[Expand table](#)

Type	Description
None	

deserialize

Creates a new instance of the store from previously serialized state.

This method, together with `serialize()` can be used to save and load messages from a persistent store if this store only has messages in memory.

Python

```
async classmethod deserialize(serialized_store_state: MutableMapping[str, Any],  
**kwargs: Any) -> ChatMessageStoreProtocol
```

Parameters

[Expand table](#)

Name	Description
serialized_store_state Required*	MutableMapping[str, Any] The previously serialized state data containing messages.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional arguments for deserialization.

Returns

[Expand table](#)

Type	Description
ChatMessageStoreProtocol	A new instance of the store populated with messages from the serialized state.

list_messages

Gets all the messages from the store that should be used for the next agent invocation.

Messages are returned in ascending chronological order, with the oldest message first.

If the messages stored in the store become very large, it is up to the store to truncate, summarize or otherwise limit the number of messages returned.

When using implementations of `ChatMessageStoreProtocol`, a new one should be created for each thread since they may contain state that is specific to a thread.

Python

```
async list_messages() -> list[ChatMessage]
```

Returns

[Expand table](#)

Type	Description
list[ChatMessage]	

serialize

Serializes the current object's state.

This method, together with `deserialize()` can be used to save and load messages from a persistent store if this store only has messages in memory.

Python

```
async serialize(**kwargs: Any) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Additional arguments for serialization.

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	The serialized state data that can be used with <code>deserialize()</code> .

update_from_state

Update the current ChatMessageStore instance from serialized state data.

Python

```
async update_from_state(serialized_store_state: MutableMapping[str, Any],  
                        **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
serialized_store_state Required*	MutableMapping[str, Any] Previously serialized state data containing messages.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Additional arguments for deserialization.

Returns

[Expand table](#)

Type	Description
None	

ChatMiddleware Class

Abstract base class for chat middleware that can intercept chat client requests.

Chat middleware allows you to intercept and modify chat client requests before and after execution. You can modify messages, add system prompts, log requests, or override chat responses.

⚠ Note

ChatMiddleware is an abstract base class. You must subclass it and implement the `process()` method to create custom chat middleware.

Constructor

Python

```
ChatMiddleware()
```

Examples

Python

```
from agent_framework import ChatMiddleware, ChatContext, ChatAgent

class SystemPromptMiddleware(ChatMiddleware):
    def __init__(self, system_prompt: str):
        self.system_prompt = system_prompt

    async def process(self, context: ChatContext, next):
        # Add system prompt to messages
        from agent_framework import ChatMessage

        context.messages.insert(0, ChatMessage(role="system",
        content=self.system_prompt))

        # Continue execution
        await next(context)

# Use with an agent
```

```
agent = ChatAgent(  
    chat_client=client, name="assistant", middleware=SystemPromptMiddleware("You  
are a helpful assistant.")  
)
```

Methods

 Expand table

`__init__`

`__new__`

process Process a chat client request.

ⓘ Note

Middleware should not return anything. All data manipulation should happen within the context object. Set `context.result` to override execution, or observe `context.result` after calling `next()` for actual results.

`__init__`

Python

```
__init__()
```

`__new__`

Python

```
__new__(**kwargs)
```

process

Process a chat client request.

ⓘ Note

Middleware should not return anything. All data manipulation should happen

within the context object. Set context.result to override execution, or observe context.result after calling next() for actual results.

Python

```
abstract async process(context: ChatContext, next: Callable[[ChatContext], Awaitable[None]]) -> None
```

Parameters

 Expand table

Name	Description
context Required*	ChatContext Chat invocation context containing chat client, messages, options, and metadata. Use context.is_streaming to determine if this is a streaming call. Middleware can set context.result to override execution, or observe the actual execution result after calling next(). For non-streaming: ChatResponse For streaming: AsyncIterable[ChatResponseUpdate]
next Required*	Callable [[ChatContext], Awaitable [None]] Function to call the next middleware or final chat execution. Does not return anything - all data flows through the context.

Returns

 Expand table

Type	Description
None	

ChatOptions Class

Common request settings for AI services.

Initialize ChatOptions.

Constructor

Python
<code>ChatOptions()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>model_id</code>	The AI model ID to use. Default value: None
<code>allow_multiple_tool_calls</code>	Whether to allow multiple tool calls. Default value: None
<code>conversation_id</code>	The conversation ID. Default value: None
<code>frequency_penalty</code>	The frequency penalty (must be between -2.0 and 2.0). Default value: None
<code>instructions</code>	the instructions, will be turned into a system or equivalent message. Default value: None
<code>logit_bias</code>	The logit bias mapping. Default value: None
<code>max_tokens</code>	The maximum number of tokens (must be > 0). Default value: None
<code>metadata</code>	Metadata mapping. Default value: None
<code>presence_penalty</code>	The presence penalty (must be between -2.0 and 2.0). Default value: None
<code>response_format</code>	Structured output response format schema. Must be a valid Pydantic model.

Name	Description
	Default value: None
seed	Random seed for reproducibility. Default value: None
stop	Stop sequences. Default value: None
store	Whether to store the conversation. Default value: None
temperature	The temperature (must be between 0.0 and 2.0). Default value: None
tool_choice	The tool choice mode. Default value: None
tools	List of available tools. Default value: None
top_p	The top-p value (must be between 0.0 and 1.0). Default value: None
user	The user ID. Default value: None
additional_properties	Provider-specific additional properties, can also be passed as kwargs. Default value: None
**kwargs	Additional properties to include in additional_properties.

Examples

Python

```

from agent_framework import ChatOptions, ai_function

# Create basic chat options
options = ChatOptions(
    model_id="gpt-4",
    temperature=0.7,
    max_tokens=1000,
)

# With tools
@ai_function
def get_weather(location: str) -> str:
    '''Get weather for a location.'''

```

```
return f"Weather in {location}"
```

```
options = ChatOptions(
    model_id="gpt-4",
    tools=get_weather,
    tool_choice="auto",
)

# Require a specific tool to be called
options_required = ChatOptions(
    model_id="gpt-4",
    tools=get_weather,
    tool_choice=ToolMode.REQUIRED(function_name="get_weather"),
)

# Combine options
base_options = ChatOptions(temperature=0.5)
extended_options = ChatOptions(max_tokens=500, tools=get_weather)
combined = base_options & extended_options
```

Methods

[Expand table](#)

<code>__init__</code>	Initialize ChatOptions.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).

`to_json` Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize ChatOptions.

Python

```
__init__(*, model_id: str | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, instructions: str | None = None, logit_bias: MutableMapping[str | int, float] | None = None, max_tokens: int | None = None, metadata: MutableMapping[str, str] | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | Mapping[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: MutableMapping[str, Any] | None = None, **kwargs: Any)
```

Parameters

[Expand table](#)

Name	Description
<code>model_id</code> Required*	<code>str</code> <code>None</code>
<code>allow_multiple_tool_calls</code> Required*	<code>bool</code> <code>None</code>
<code>conversation_id</code> Required*	<code>str</code> <code>None</code>
<code>frequency_penalty</code> Required*	<code>float</code> <code>None</code>
<code>instructions</code> Required*	<code>str</code> <code>None</code>
<code>logit_bias</code> Required*	<code>MutableMapping[str int, float]</code> <code>None</code>

Name	Description
max_tokens Required*	int None
metadata Required*	MutableMapping[str, str] None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] collections.abc.Mapping[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	MutableMapping[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
model_id	The AI model ID to use. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls. Default value: None
conversation_id	The conversation ID. Default value: None
frequency_penalty	The frequency penalty (must be between -2.0 and 2.0). Default value: None
instructions	the instructions, will be turned into a system or equivalent message. Default value: None
logit_bias	The logit bias mapping. Default value: None
max_tokens	The maximum number of tokens (must be > 0). Default value: None
metadata	Metadata mapping. Default value: None
presence_penalty	The presence penalty (must be between -2.0 and 2.0). Default value: None
response_format	Structured output response format schema. Must be a valid Pydantic model. Default value: None
seed	Random seed for reproducibility. Default value: None
stop	Stop sequences. Default value: None
store	Whether to store the conversation. Default value: None
temperature	The temperature (must be between 0.0 and 2.0). Default value: None
tool_choice	The tool choice mode. Default value: None
tools	List of available tools. Default value: None
top_p	The top-p value (must be between 0.0 and 1.0).

Name	Description
	Default value: None
user	The user ID. Default value: None
additional_properties	Provider-specific additional properties, can also be passed as kwargs. Default value: None
**kwargs	Additional properties to include in additional_properties.

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected

```



```
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	

Name	Description
	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

tools

Return the tools that are specified.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'_tools'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ChatResponse Class

Represents the response to a chat request.

Initializes a ChatResponse with the provided parameters.

Constructor

Python
<code>ChatResponse()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>messages</code>	A single ChatMessage or a sequence of ChatMessage objects to include in the response. Default value: None
<code>text</code>	The text content to include in the response. If provided, it will be added as a ChatMessage. Default value: None
<code>response_id</code>	Optional ID of the chat response. Default value: None
<code>conversation_id</code>	Optional identifier for the state of the conversation. Default value: None
<code>model_id</code>	Optional model ID used in the creation of the chat response. Default value: None
<code>created_at</code>	Optional timestamp for the chat response. Default value: None
<code>finish_reason</code>	Optional reason for the chat response. Default value: None
<code>usage_details</code>	Optional usage details for the chat response. Default value: None
<code>value</code>	Optional value of the structured output. Default value: None

Name	Description
response_format	Optional response format for the chat response. Default value: None
additional_properties	Optional additional properties associated with the chat response. Default value: None
raw_representation	Optional raw representation of the chat response from an underlying implementation. Default value: None
**kwargs	Any additional keyword arguments.

Examples

Python

```

from agent_framework import ChatResponse, ChatMessage

# Create a simple text response
response = ChatResponse(text="Hello, how can I help you?")
print(response.text) # "Hello, how can I help you?"

# Create a response with messages
msg = ChatMessage(role="assistant", text="The weather is sunny.")
response = ChatResponse(
    messages=[msg],
    finish_reason="stop",
    model_id="gpt-4",
)

# Combine streaming updates
updates = [...] # List of ChatResponseUpdate objects
response = ChatResponse.from_chat_response_updates(updates)

# Serialization - to_dict and from_dict
response_dict = response.to_dict()
# {'type': 'chat_response', 'messages': [...], 'model_id': 'gpt-4',
#  'finish_reason': {'type': 'finish_reason', 'value': 'stop'}}
restored_response = ChatResponse.from_dict(response_dict)
print(restored_response.model_id) # "gpt-4"

# Serialization - to_json and from_json
response_json = response.to_json()
# '{"type": "chat_response", "messages": [...], "model_id": "gpt-4", ...}'
restored_from_json = ChatResponse.from_json(response_json)
print(restored_from_json.text) # "The weather is sunny."

```

Methods

<code>__init__</code>	Initializes a ChatResponse with the provided parameters.
<code>__new__</code>	
<code>from_chat_response_generator</code>	Joins multiple updates into a single ChatResponse.
<code>from_chat_response_updates</code>	Joins multiple updates into a single ChatResponse.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.
<code>try_parse_value</code>	If there is a value, does nothing, otherwise tries to parse the text into the value.

`__init__`

Initializes a ChatResponse with the provided parameters.


```
__init__(*, messages: ChatMessage | MutableSequence[ChatMessage], response_id: str | None = None, conversation_id: str | None = None, model_id: str | None = None, created_at: str | None = None, finish_reason: FinishReason | None = None, usage_details: UsageDetails | None = None, value: Any | None = None, response_format: type[BaseModel] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Keyword-Only Parameters

 Expand table

Name	Description
messages	A single ChatMessage or a sequence of ChatMessage objects to include in the response. Default value: None
text	The text content to include in the response. If provided, it will be added as a ChatMessage. Default value: None
response_id	Optional ID of the chat response. Default value: None
conversation_id	Optional identifier for the state of the conversation. Default value: None
model_id	Optional model ID used in the creation of the chat response. Default value: None
created_at	Optional timestamp for the chat response. Default value: None
finish_reason	Optional reason for the chat response. Default value: None
usage_details	Optional usage details for the chat response. Default value: None
value	Optional value of the structured output. Default value: None
response_format	Optional response format for the chat response. Default value: None
additional_properties	Optional additional properties associated with the chat response. Default value: None
raw_representation	Optional raw representation of the chat response from an underlying implementation.

Name	Description
	Default value: None
**kwargs	Any additional keyword arguments.

__new__

Python
<code>__new__(**kwargs)</code>

from_chat_response_generator

Joins multiple updates into a single ChatResponse.

Python
<pre>async classmethod from_chat_response_generator(updates: AsyncIterable[ChatResponseUpdate], *, output_format_type: type[BaseModel] None = None) -> TChatResponse</pre>

Parameters

 Expand table

Name	Description
updates Required*	AsyncIterable[ChatResponseUpdate] An async iterable of ChatResponseUpdate objects to combine.
output_format_type Required*	type[<xref:pydantic.main.BaseModel>] None

Keyword-Only Parameters

 Expand table

Name	Description
output_format_type	Optional Pydantic model type to parse the response text into structured data. Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._types.TChatResponse></code>	

Examples

Python

```
from agent_framework import ChatResponse, ChatResponseUpdate, ChatClient

client = ChatClient() # should be a concrete implementation
response = await ChatResponse.from_chat_response_generator(
    client.get_streaming_response("Hello, how are you?")
)
print(response.text)
```

from_chat_response_updates

Joins multiple updates into a single ChatResponse.

Python

```
from_chat_response_updates(updates: Sequence[ChatResponseUpdate], *,
    output_format_type: type[BaseModel] | None = None) -> TChatResponse
```

Parameters

[Expand table](#)

Name	Description
updates Required*	<code>Sequence[ChatResponseUpdate]</code> A sequence of ChatResponseUpdate objects to combine.
output_format_type Required*	<code>type[<xref:pydantic.main.BaseModel>] None</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
output_format_type	Optional Pydantic model type to parse the response text into structured data. Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework._types.TChatResponse>	

Examples

Python

```
from agent_framework import ChatResponse, ChatResponseUpdate

# Create some response updates
updates = [
    ChatResponseUpdate(role="assistant", text="Hello"),
    ChatResponseUpdate(text=" How can I help you?"),
]

# Combine updates into a single ChatResponse
response = ChatResponse.from_chat_response_updates(updates)
print(response.text)  # "Hello How can I help you?"
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"
```

```

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TCIass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
<code>JSONDecodeError</code>	If the JSON string is malformed.
<code>ValueError</code>	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>

Name	Description
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

try_parse_value

If there is a value, does nothing, otherwise tries to parse the text into the value.

Python
<code>try_parse_value(output_format_type: type[BaseModel]) -> <code>None</code></code>

Parameters

[Expand table](#)

Name	Description
output_format_type Required*	type [<xref:pydantic.main.BaseModel>]

Returns

 Expand table

Type	Description
None	

Attributes

text

Returns the concatenated text of all messages in the response.

messages

The list of chat messages in the response.

response_id

The ID of the chat response.

conversation_id

An identifier for the state of the conversation.

model_id

The model ID used in the creation of the chat response.

created_at

A timestamp for the chat response.

finish_reason

The reason for the chat response.

usage_details

The usage details for the chat response.

structured_output

The structured output of the chat response, if applicable.

additional_properties

Any additional properties associated with the chat response.

raw_representation

The raw representation of the chat response from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ChatResponseUpdate Class

Represents a single streaming response chunk from a *ChatClient*.

Initializes a ChatResponseUpdate with the provided parameters.

Constructor

Python
<code>ChatResponseUpdate()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>contents</code>	Optional list of BaseContent items or dicts to include in the update. Default value: None
<code>text</code>	Optional text content to include in the update. Default value: None
<code>role</code>	Optional role of the author of the response update (Role, string, or dict Default value: None
<code>author_name</code>	Optional name of the author of the response update. Default value: None
<code>response_id</code>	Optional ID of the response of which this update is a part. Default value: None
<code>message_id</code>	Optional ID of the message of which this update is a part. Default value: None
<code>conversation_id</code>	Optional identifier for the state of the conversation of which this update is a part Default value: None
<code>model_id</code>	Optional model ID associated with this response update. Default value: None
<code>created_at</code>	Optional timestamp for the chat response update. Default value: None

Name	Description
finish_reason	Optional finish reason for the operation. Default value: None
additional_properties	Optional additional properties associated with the chat response update. Default value: None
raw_representation	Optional raw representation of the chat response update from an underlying implementation. Default value: None
**kwargs	Any additional keyword arguments.

Examples

Python

```
from agent_framework import ChatResponseUpdate, TextContent

# Create a response update
update = ChatResponseUpdate(
    contents=[TextContent(text="Hello")],
    role="assistant",
    message_id="msg_123",
)
print(update.text) # "Hello"

# Create update with text shorthand
update = ChatResponseUpdate(text="World!", role="assistant")

# Serialization - to_dict and from_dict
update_dict = update.to_dict()
# {'type': 'chat_response_update', 'contents': [{'type': 'text', 'text': 'Hello'}],
#  'role': {'type': 'role', 'value': 'assistant'}, 'message_id': 'msg_123'}
restored_update = ChatResponseUpdate.from_dict(update_dict)
print(restored_update.text) # "Hello"

# Serialization - to_json and from_json
update_json = update.to_json()
# '{"type": "chat_response_update", "contents": [{"type": "text", "text":
"Hello"}], ...}'
restored_from_json = ChatResponseUpdate.from_json(update_json)
print(restored_from_json.message_id) # "msg_123"
```

Methods

<code>__init__</code>	Initializes a ChatResponseUpdate with the provided parameters.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with '_').
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a ChatResponseUpdate with the provided parameters.

Python

```
__init__(*, contents: Sequence[TextContent | DataContent | TextReasoningContent |
UriContent | FunctionCallContent | FunctionResultContent | ErrorContent |
UsageContent | HostedFileContent | HostedVectorStoreContent |
FunctionApprovalRequestContent | FunctionApprovalResponseContent | dict[str, Any]]
| None = None, text: TextContent | str | None = None, role: Role |
Literal['system', 'user', 'assistant', 'tool'] | dict[str, Any] | None = None,
author_name: str | None = None, response_id: str | None = None, message_id: str |
None = None, conversation_id: str | None = None, model_id: str | None = None,
created_at: str | None = None, finish_reason: FinishReason | dict[str, Any] | None
= None, additional_properties: dict[str, Any] | None = None, raw_representation:
Any | None = None, **kwargs: Any) -> None
```


Parameters

 Expand table

Name	Description
contents Required*	Sequence [TextContent DataContent TextReasoningContent UriContent FunctionCallContent FunctionResultContent ErrorContent UsageContent HostedFileContent HostedVectorStoreContent FunctionApprovalRequestContent FunctionApprovalResponseContent dict[str, Any]] None
text Required*	TextContent str None
role Required*	Role Literal ['system', 'user', 'assistant', 'tool'] dict[str, typing.Any] None
author_name Required*	str None
response_id Required*	str None
message_id Required*	str None
conversation_id Required*	str None
model_id Required*	str None
created_at Required*	str None
finish_reason Required*	FinishReason dict[str, Any] None
additional_properties Required*	dict[str, Any] None
raw_representation Required*	Any None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
contents	Optional list of BaseContent items or dicts to include in the update. Default value: None
text	Optional text content to include in the update. Default value: None
role	Optional role of the author of the response update (Role, string, or dict Default value: None
author_name	Optional name of the author of the response update. Default value: None
response_id	Optional ID of the response of which this update is a part. Default value: None
message_id	Optional ID of the message of which this update is a part. Default value: None
conversation_id	Optional identifier for the state of the conversation of which this update is a part Default value: None
model_id	Optional model ID associated with this response update. Default value: None
created_at	Optional timestamp for the chat response update. Default value: None
finish_reason	Optional finish reason for the operation. Default value: None
additional_properties	Optional additional properties associated with the chat response update. Default value: None
raw_representation	Optional raw representation of the chat response update from an underlying implementation. Default value: None
**kwargs	Any additional keyword arguments.

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies	MutableMapping[str, Any] None

Name	Description
Required*	

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

```
Python

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```

```

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.

Type	Description
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None

Name	Description
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

text

Returns the concatenated text of all contents in the update.

contents

The chat response update content items.

role

The role of the author of the response update.

author_name

The name of the author of the response update.

response_id

The ID of the response of which this update is a part.

message_id

The ID of the message of which this update is a part.

conversation_id

An identifier for the state of the conversation of which this update is a part.

model_id

The model ID associated with this response update.

created_at

A timestamp for the chat response update.

finish_reason

The finish reason for the operation.

additional_properties

Any additional properties associated with the chat response update.

raw_representation

The raw representation of the chat response update from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

CheckpointStorage Class

Protocol for checkpoint storage backends.

Constructor

Python
<code>CheckpointStorage()</code>

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>delete_checkpoint</code>	Delete a checkpoint by ID.
<code>list_checkpoint_ids</code>	List checkpoint IDs. If workflow_id is provided, filter by that workflow.
<code>list_checkpoints</code>	List checkpoint objects. If workflow_id is provided, filter by that workflow.
<code>load_checkpoint</code>	Load a checkpoint by ID.
<code>save_checkpoint</code>	Save a checkpoint and return its ID.

`__init__`

Python
<code>__init__(*args, **kwargs)</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`delete_checkpoint`

Delete a checkpoint by ID.

Python

```
async delete_checkpoint(checkpoint_id: str) -> bool
```

Parameters

[Expand table](#)

Name	Description
checkpoint_id Required*	str

Returns

[Expand table](#)

Type	Description
bool	

list_checkpoint_ids

List checkpoint IDs. If workflow_id is provided, filter by that workflow.

Python

```
async list_checkpoint_ids(workflow_id: str | None = None) -> list[str]
```

Parameters

[Expand table](#)

Name	Description
workflow_id	str None Default value: None

Returns

[Expand table](#)

Type	Description
<code>list[str]</code>	

list_checkpoints

List checkpoint objects. If `workflow_id` is provided, filter by that workflow.

Python
<pre>async list_checkpoints(workflow_id: str None = None) -> list[WorkflowCheckpoint]</pre>

Parameters

[Expand table](#)

Name	Description
<code>workflow_id</code>	<code>str None</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code>list[WorkflowCheckpoint]</code>	

load_checkpoint

Load a checkpoint by ID.

Python
<pre>async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint None</pre>

Parameters

[Expand table](#)

Name	Description
checkpoint_id Required*	str

Returns

[Expand table](#)

Type	Description
WorkflowCheckpoint None	

save_checkpoint

Save a checkpoint and return its ID.

Python
<pre>async save_checkpoint(checkpoint: WorkflowCheckpoint) -> str</pre>

Parameters

[Expand table](#)

Name	Description
checkpoint Required*	WorkflowCheckpoint

Returns

[Expand table](#)

Type	Description
str	

CitationAnnotation Class

Represents a citation annotation.

Initialize CitationAnnotation.

Constructor

Python
CitationAnnotation()

Keyword-Only Parameters

 Expand table

Name	Description
title	The title of the cited content. Default value: None
url	The URL of the cited content. Default value: None
file_id	The file identifier of the cited content, if applicable. Default value: None
tool_name	The name of the tool that generated the citation, if applicable. Default value: None
snippet	A snippet of the cited content, if applicable. Default value: None
annotated_regions	A list of regions that have been annotated with this citation. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content from an underlying implementation. Default value: None
**kwargs	Additional keyword arguments.

Examples

Python

```
from agent_framework import CitationAnnotation, TextSpanRegion

# Create a citation annotation
citation = CitationAnnotation(
    title="Agent Framework Documentation",
    url="https://example.com/docs",
    snippet="This is a relevant excerpt...",
    annotated_regions=[TextSpanRegion(start_index=0, end_index=25)],
)
print(citation.title) # "Agent Framework Documentation"
```

Methods

 Expand table

<code>__init__</code>	Initialize CitationAnnotation.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize CitationAnnotation.

Python

```
__init__(*, title: str | None = None, url: str | None = None, file_id: str | None = None, tool_name: str | None = None, snippet: str | None = None, annotated_regions: list[TextSpanRegion] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
title Required*	str None
url Required*	str None
file_id Required*	str None
tool_name Required*	str None
snippet Required*	str None
annotated_regions Required*	list[TextSpanRegion] None
additional_properties Required*	dict[str, Any] None
raw_representation Required*	Any None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
title	The title of the cited content. Default value: None

Name	Description
<code>url</code>	The URL of the cited content. Default value: None
<code>file_id</code>	The file identifier of the cited content, if applicable. Default value: None
<code>tool_name</code>	The name of the tool that generated the citation, if applicable. Default value: None
<code>snippet</code>	A snippet of the cited content, if applicable. Default value: None
<code>annotated_regions</code>	A list of regions that have been annotated with this citation. Default value: None
<code>additional_properties</code>	Optional additional properties associated with the content. Default value: None
<code>raw_representation</code>	Optional raw representation of the content from an underlying implementation. Default value: None
<code>**kwargs</code>	Additional keyword arguments.

Returns

[\[\]](#) Expand table

Type	Description
None	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection

to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

```

```
# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

type

The type of content, which is always "citation" for this class.

title

The title of the cited content.

url

The URL of the cited content.

file_id

The file identifier of the cited content, if applicable.

tool_name

The name of the tool that generated the citation, if applicable.

snippet

A snippet of the cited content, if applicable.

annotated_regions

A list of regions that have been annotated with this citation.

Python

```
annotated_regions: list[TextSpanRegion] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```


ConcurrentBuilder Class

High-level builder for concurrent agent workflows.

- *participants([...])* accepts a list of AgentProtocol (recommended) or Executor.
- *register_participants([...])* accepts a list of factories for AgentProtocol (recommended) or Executor factories
- *build()* wires: dispatcher -> fan-out -> participants -> fan-in -> aggregator.
- *with_aggregator(...)* overrides the default aggregator with an Executor or callback.
- *register_aggregator(...)* accepts a factory for an Executor as custom aggregator.

Usage:

Python

```
from agent_framework import ConcurrentBuilder

# Minimal: use default aggregator (returns list[ChatMessage])
workflow = ConcurrentBuilder().participants([agent1, agent2, agent3]).build()

# With agent factories
workflow = ConcurrentBuilder().register_participants([create_agent1, create_agent2,
create_agent3]).build()

# Custom aggregator via callback (sync or async). The callback receives
# list[AgentExecutorResponse] and its return value becomes the workflow's output.
def summarize(results: list[AgentExecutorResponse]) -> str:
    return " | ".join(r.agent_run_response.messages[-1].text for r in results)

workflow = ConcurrentBuilder().participants([agent1, agent2,
agent3]).with_aggregator(summarize).build()

# Custom aggregator via a factory
class MyAggregator(Executor):
    @handler
    async def aggregate(self, results: list[AgentExecutorResponse], ctx:
WorkflowContext[Never, str]) -> None:
        await ctx.yield_output(" | ".join(r.agent_run_response.messages[-1].text
for r in results))

workflow = (
    ConcurrentBuilder()
```

```

        .register_participants([create_agent1, create_agent2, create_agent3])
        .register_aggregator(lambda: MyAggregator(id="my_aggregator"))
        .build()
    )

```

```

    # Enable checkpoint persistence so runs can resume
    workflow = ConcurrentBuilder().participants([agent1, agent2,
agent3]).with_checkpointing(storage).build()

```

```

    # Enable request info before aggregation
    workflow = ConcurrentBuilder().participants([agent1,
agent2]).with_request_info().build()

```

Constructor

Python

`ConcurrentBuilder()`

Methods

[Expand table](#)

`__init__`

`__new__`

`build`

Build and validate the concurrent workflow.

Wiring pattern:

- Dispatcher (internal) fans out the input to all *participants*
- Fan-in collects *AgentExecutorResponse* objects from all participants
- If request info is enabled, the orchestration emits a request info event with outputs from all participants

before sending the outputs to the aggregator

- Aggregator yields output and the workflow becomes idle. The output is either:
 - list[ChatMessage] (default aggregator: one user + one assistant per agent) - custom payload from the provided aggregator

Example:

Python

```

workflow = ConcurrentBuilder().participants([agent1,

```

```
agent2])).build()
```

participants

Define the parallel participants for this concurrent workflow.

Accepts AgentProtocol instances (e.g., created by a chat client) or Executor instances. Each participant is wired as a parallel branch using fan-out edges from an internal dispatcher.

Example:

Python

```
wf = ConcurrentBuilder().participants([researcher_agent,
marketer_agent, legal_agent]).build()

# Mixing agent(s) and executor(s) is supported
wf2 = ConcurrentBuilder().participants([researcher_agent,
my_custom_executor]).build()
```

register_aggregator

Define a custom aggregator for this concurrent workflow.

Accepts a factory (callable) that returns an Executor instance. The executor should handle *list[AgentExecutorResponse]* and yield output using *ctx.yield_output(...)*.

Example: .. code-block:: python

```
class MyCustomExecutor(Executor): ...
```

```
wf = ( ConcurrentBuilder() .register_participants([create_researcher, create_marketer,
create_legal]) .register_aggregator(lambda: MyCustomExecutor(id="my_aggregator"))
.build()

)
```

register_participants

Define the parallel participants for this concurrent workflow.

Accepts factories (callables) that return AgentProtocol instances (e.g., created by a chat client) or Executor instances. Each participant created by a factory is wired as a parallel branch using fan-out edges from an internal dispatcher.

Example:

Python

```
def create_researcher() -> ChatAgent:
    return ...

def create_marketer() -> ChatAgent:
    return ...
```

```

def create_legal() -> ChatAgent:
    return ...

class MyCustomExecutor(Executor): ...

wf =
    ConcurrentBuilder().register_participants([create_researcher,
        create_marketer, create_legal]).build()

# Mixing agent(s) and executor(s) is supported
wf2 =
    ConcurrentBuilder().register_participants([create_researcher,
        MyCustomExecutor]).build()

```

with_aggregator

Override the default aggregator with an executor or a callback.

- Executor: must handle *list[AgentExecutorResponse]* and yield output using *ctx.yield_output(...)*
- Callback: sync or async callable with one of the signatures: *(results: list[AgentExecutorResponse]) -> Any | None* or *(results: list[AgentExecutorResponse], ctx: WorkflowContext) -> Any | None*. If the callback returns a non-None value, it becomes the workflow's output.

Example:

with_checkpointing

Enable checkpoint persistence using the provided storage backend.

with_request_info

Enable request info before aggregation in the workflow.

When enabled, the workflow pauses after all parallel agents complete, emitting a *RequestInfoEvent* that allows the caller to review and optionally modify the combined results before aggregation. The caller provides feedback via the standard *response_handler/request_info* pattern.

ⓘ Note

Unlike *SequentialBuilder* and *GroupChatBuilder*, *ConcurrentBuilder* does not

support per-agent filtering since all agents run in parallel and results are collected together. The pause occurs once with all agent outputs received.

Python

```
__init__() -> None
```

Returns

[Expand table](#)

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

build

Build and validate the concurrent workflow.

Wiring pattern:

- Dispatcher (internal) fans out the input to all *participants*
- Fan-in collects *AgentExecutorResponse* objects from all participants
- If request info is enabled, the orchestration emits a request info event with outputs from all participants

before sending the outputs to the aggregator

- Aggregator yields output and the workflow becomes idle. The output is either: -
list[ChatMessage] (default aggregator: one user + one assistant per agent) - custom
payload from the provided aggregator

Example:

Python

```
workflow = ConcurrentBuilder().participants([agent1, agent2]).build()
```

Python

```
build() -> Workflow
```

Returns

[Expand table](#)

Type	Description
Workflow	a ready-to-run workflow instance

Exceptions

[Expand table](#)

Type	Description
ValueError	if no participants were defined

participants

Define the parallel participants for this concurrent workflow.

Accepts AgentProtocol instances (e.g., created by a chat client) or Executor instances. Each participant is wired as a parallel branch using fan-out edges from an internal dispatcher.

Example:

Python

```
wf = ConcurrentBuilder().participants([researcher_agent, marketer_agent,
legal_agent]).build()
```

```
# Mixing agent(s) and executor(s) is supported
wf2 = ConcurrentBuilder().participants([researcher_agent,
my_custom_executor]).build()
```

Python

```
participants(participants: Sequence[AgentProtocol | Executor]) ->
ConcurrentBuilder
```

Parameters

[Expand table](#)

Name	Description
participants Required*	Sequence[AgentProtocol Executor] Sequence of AgentProtocol or Executor instances

Returns

[Expand table](#)

Type	Description
ConcurrentBuilder	

Exceptions

[Expand table](#)

Type	Description
ValueError	if <i>participants</i> is empty, contains duplicates, or <i>.register_participants()</i> or <i>.participants()</i> were already called
TypeError	if any entry is not AgentProtocol or Executor

register_aggregator

Define a custom aggregator for this concurrent workflow.

Accepts a factory (callable) that returns an Executor instance. The executor should handle *list[AgentExecutorResponse]* and yield output using *ctx.yield_output(...)*.

Example: .. code-block:: python

```
class MyCustomExecutor(Executor): ...
```

```
wf = ( ConcurrentBuilder() .register_participants([create_researcher, create_marketer,  
create_legal]) .register_aggregator(lambda: MyCustomExecutor(id="my_aggregator")) .build()  
)
```

Python

```
register_aggregator(aggregator_factory: Callable[[], Executor]) ->
ConcurrentBuilder
```

Parameters

[Expand table](#)

Name	Description
aggregator_factory Required*	Callable [[], Executor] Callable that returns an Executor instance

Returns

[Expand table](#)

Type	Description
ConcurrentBuilder	

register_participants

Define the parallel participants for this concurrent workflow.

Accepts factories (callables) that return AgentProtocol instances (e.g., created by a chat client) or Executor instances. Each participant created by a factory is wired as a parallel branch using fan-out edges from an internal dispatcher.

Example:

Python

```
def create_researcher() -> ChatAgent:
    return ...

def create_marketer() -> ChatAgent:
    return ...

def create_legal() -> ChatAgent:
    return ...

class MyCustomExecutor(Executor): ...
```

```

wf = ConcurrentBuilder().register_participants([create_researcher,
create_marketer, create_legal]).build()

# Mixing agent(s) and executor(s) is supported
wf2 = ConcurrentBuilder().register_participants([create_researcher,
MyCustomExecutor]).build()

```

Python

```

register_participants(participant_factories: Sequence[Callable[[], AgentProtocol |
Executor]]) -> ConcurrentBuilder

```

Parameters

[Expand table](#)

Name	Description
participant_factories Required*	Sequence[Callable[[], AgentProtocol Executor]] Sequence of callables returning AgentProtocol or Executor instances

Returns

[Expand table](#)

Type	Description
ConcurrentBuilder	

Exceptions

[Expand table](#)

Type	Description
ValueError	if <i>participant_factories</i> is empty or <i>.participants()</i> or <i>.register_participants()</i> were already called

with_aggregator

Override the default aggregator with an executor or a callback.

- Executor: must handle `list[AgentExecutorResponse]` and yield output using `ctx.yield_output(...)`
- Callback: sync or async callable with one of the signatures: *(results: list[AgentExecutorResponse]) -> Any | None* or *(results: list[AgentExecutorResponse], ctx: WorkflowContext) -> Any | None*. If the callback returns a non-None value, it becomes the workflow's output.

Example:

Python

```
with_aggregator(aggregator: Executor | Callable[[list[AgentExecutorResponse]], Any] | Callable[[list[AgentExecutorResponse], WorkflowContext[Never, Any]], Any])
-> ConcurrentBuilder
```

Parameters

[Expand table](#)

Name	Description
aggregator Required*	Executor Callable [[list [AgentExecutorResponse]], Any] Callable [[list [AgentExecutorResponse], WorkflowContext [Never , Any]], Any] Executor instance, or callback function

Returns

[Expand table](#)

Type	Description
ConcurrentBuilder	

with_checkpointing

Enable checkpoint persistence using the provided storage backend.

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> ConcurrentBuilder
```

Parameters

[Expand table](#)

Name	Description
checkpoint_storage Required*	CheckpointStorage CheckpointStorage instance for persisting workflow state

Returns

[Expand table](#)

Type	Description
ConcurrentBuilder	

with_request_info

Enable request info before aggregation in the workflow.

When enabled, the workflow pauses after all parallel agents complete, emitting a RequestInfoEvent that allows the caller to review and optionally modify the combined results before aggregation. The caller provides feedback via the standard response_handler/request_info pattern.

ⓘ Note

Unlike SequentialBuilder and GroupChatBuilder, ConcurrentBuilder does not support per-agent filtering since all agents run in parallel and results are collected together. The pause occurs once with all agent outputs received.

Python

```
with_request_info() -> ConcurrentBuilder
```

Returns

[Expand table](#)

Type	Description
<xref:self>	The builder instance for fluent chaining.

Context Class

A class containing any context that should be provided to the AI model as supplied by a ContextProvider.

Each ContextProvider has the ability to provide its own context for each invocation. The Context class contains the additional context supplied by the ContextProvider. This context will be combined with context supplied by other providers before being passed to the AI model. This context is per invocation, and will not be stored as part of the chat history.

Create a new Context object.

Constructor

Python
Context()

Parameters

 Expand table

Name	Description
instructions	The instructions to provide to the AI model. Default value: None
messages	The list of messages to include in the context. Default value: None
tools	The list of tools to provide to this run. Default value: None

Examples

Python
<pre>from agent_framework import Context, ChatMessage # Create context with instructions context = Context(instructions="Use a professional tone when responding.", messages=[ChatMessage(content="Previous context", role="user")],</pre>

```
tools=[my_tool],
)

# Access context properties
print(context.instructions)
print(len(context.messages))
```

Methods

 Expand table

<code>__init__</code>	Create a new Context object.
<code>__new__</code>	

`__init__`

Create a new Context object.

Python

```
__init__(instructions: str | None = None, messages: Sequence[ChatMessage] | None = None, tools: Sequence[ToolProtocol] | None = None)
```

Parameters

 Expand table

Name	Description
instructions	<code>str None</code> The instructions to provide to the AI model. Default value: None
messages	<code>Sequence[ChatMessage] None</code> The list of messages to include in the context. Default value: None
tools	<code>Sequence[ToolProtocol] None</code> The list of tools to provide to this run. Default value: None

`__new__`

Python

```
__new__(**kwargs)
```

ContextProvider Class

Base class for all context providers.

A context provider is a component that can be used to enhance the AI's context management. It can listen to changes in the conversation and provide additional context to the AI model just before invocation.

ⓘ Note

ContextProvider is an abstract base class. You must subclass it and implement the `invoking()` method to create a custom context provider. Ideally, you should also implement the `invoked()` and `thread_created()` methods to track conversation state, but these are optional.

Constructor

Python

```
ContextProvider()
```

Examples

Python

```
from agent_framework import ContextProvider, Context, ChatMessage

class CustomContextProvider(ContextProvider):
    async def invoking(self, messages, **kwargs):
        # Add custom instructions before each invocation
        return Context(instructions="Always be concise and helpful.", messages=[],
tools=[])

# Use with a chat agent
async with CustomContextProvider() as provider:
    agent = ChatAgent(chat_client=client, name="assistant",
context_providers=provider)
```

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>invoked</code>	Called after the agent has received a response from the underlying inference service. You can inspect the request and response messages, and update the state of the context provider.
<code>invoking</code>	Called just before the model/agent is invoked. Implementers can load any additional context required at this time, and they should return any context that should be passed to the agent.
<code>thread_created</code>	Called just after a new thread is created. Implementers can use this method to perform any operations required at the creation of a new thread. For example, checking long-term storage for any data that is relevant to the current session.

`__init__`

Python

```
__init__()
```

`__new__`

Python

```
__new__(**kwargs)
```

`invoked`

Called after the agent has received a response from the underlying inference service.

You can inspect the request and response messages, and update the state of the context provider.

Python

```
async invoked(request_messages: ChatMessage | Sequence[ChatMessage],
response_messages: ChatMessage | Sequence[ChatMessage] | None = None,
invoke_exception: Exception | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
request_messages Required*	ChatMessage Sequence[ChatMessage] The messages that were sent to the model/agent.
response_messages	ChatMessage Sequence[ChatMessage] None The messages that were returned by the model/agent. Default value: None
invoke_exception	Exception None The exception that was thrown, if any. Default value: None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
kwargs	Additional keyword arguments (not used at present).

Returns

 Expand table

Type	Description
None	

invoking

Called just before the model/agent is invoked.

Implementers can load any additional context required at this time, and they should return any context that should be passed to the agent.

Python

```
abstract async invoking(messages: ChatMessage | MutableSequence[ChatMessage],
**kwargs: Any) -> Context
```

Parameters

 Expand table

Name	Description
messages Required*	ChatMessage MutableSequence[ChatMessage] The most recent messages that the agent is being invoked with.
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
kwargs	Additional keyword arguments (not used at present).

Returns

 Expand table

Type	Description
Context	A Context object containing instructions, messages, and tools to include.

thread_created

Called just after a new thread is created.

Implementers can use this method to perform any operations required at the creation of a new thread. For example, checking long-term storage for any data that is relevant to the current session.

Python

```
async thread_created(thread_id: str | None) -> None
```

Parameters

 Expand table

Name	Description
thread_id Required*	<code>str</code> <code>None</code> The ID of the new thread.

Returns

 Expand table

Type	Description
<code>None</code>	

Attributes

DEFAULT_CONTEXT_PROMPT

Python

```
DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories\nwhen answering user questions:'
```


DataContent Class

Represents binary data content with an associated media type (also known as a MIME type).

 Important

This is for binary data that is represented as a data URI, not for online resources.

Use UriContent for online resources.

Initializes a DataContent instance.

 Important

This is for binary data that is represented as a data URI, not for online resources.

Use UriContent for online resources.

Constructor

Python
<code>DataContent()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>uri</code>	The URI of the data represented by this instance. Should be in the form: "data: {media_type};base64,{base64_data}". Default value: None
<code>data</code>	The binary data or base64-encoded string represented by this instance. If bytes, the data is transformed into a base64-encoded data URI. If str, it is assumed to be already base64-encoded and used directly. Default value: None
<code>media_type</code>	The media type of the data. Default value: None

Name	Description
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Examples

Python

```
from agent_framework import DataContent

# Create from binary data
image_data = b"raw image bytes"
data_content = DataContent(data=image_data, media_type="image/png")

# Create from base64-encoded string
base64_string = "iVBORw0KGgoAAAANS..."
data_content = DataContent(data=base64_string, media_type="image/png")

# Create from data URI
data_uri = "data:image/png;base64,iVBORw0KGgoAAAANS..."
data_content = DataContent(uri=data_uri)

# Check media type
if data_content.has_top_level_media_type("image"):
    print("This is an image")
```

Methods

 Expand table

<code>__init__</code>	Initializes a DataContent instance.  Important This is for binary data that is represented as a data URI, not for online resources.
---------------------------------------	---

Use UriContent for online resources.

`__new__`

`create_data_uri_from_base64` Create a data URI and media type from base64 image data.

`detect_image_format_from_base64` Detect image format from base64 data by examining the binary header.

`from_dict` Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

`from_json` Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

`get_data_bytes` Extracts and returns the binary data from the data URI.

`get_data_bytes_as_str` Extracts and returns the base64-encoded data from the data URI.

`has_top_level_media_type`

`to_dict` Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

`to_json` Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a DataContent instance.

Important

This is for binary data that is represented as a data URI, not for online resources.

Use UriContent for online resources.

Python

```
__init__(*, uri: str, annotations: Sequence[CitationAnnotation |  
MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] |  
None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Keyword-Only Parameters

[Expand table](#)

Name	Description
uri	The URI of the data represented by this instance. Should be in the form: "data:{media_type};base64,{base64_data}". Default value: None
data	The binary data or base64-encoded string represented by this instance. If bytes, the data is transformed into a base64-encoded data URI. If str, it is assumed to be already base64-encoded and used directly. Default value: None
media_type	The media type of the data. Default value: None
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

__new__

Python

```
__new__(**kwargs)
```

create_data_uri_from_base64

Create a data URI and media type from base64 image data.

Python

```
static create_data_uri_from_base64(image_base64: str) -> tuple[str, str]
```

Parameters

[Expand table](#)

Name	Description
image_base64 Required*	str Base64 encoded image data

Returns

[Expand table](#)

Type	Description
tuple [str , str]	Tuple of (data_uri, media_type)

detect_image_format_from_base64

Detect image format from base64 data by examining the binary header.

Python

```
static detect_image_format_from_base64(image_base64: str) -> str
```

Parameters

[Expand table](#)

Name	Description
image_base64 Required*	str Base64 encoded image data

Returns

[Expand table](#)

Type	Description
<code>str</code>	Image format as string (png, jpeg, webp, gif) with png as fallback

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
<code>value</code> Required*	

Parameters

[Expand table](#)

Name	Description
<code>value</code> Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
<code>dependencies</code> Required*	<code>MutableMapping[str, Any] None</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}
```

```

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",

```



```

    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TCI ass>	New instance of the class with any specified dependencies injected.

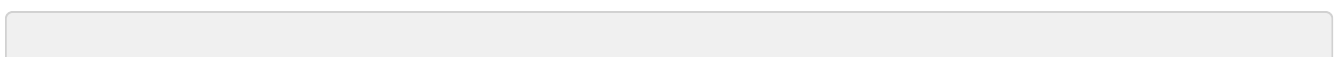
Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_data_bytes

Extracts and returns the binary data from the data URI.



Python

```
get_data_bytes() -> bytes
```

Returns

[Expand table](#)

Type	Description
bytes	The binary data as bytes.

get_data_bytes_as_str

Extracts and returns the base64-encoded data from the data URI.

Python

```
get_data_bytes_as_str() -> str
```

Returns

[Expand table](#)

Type	Description
str	The binary data as str.

has_top_level_media_type

Python

```
has_top_level_media_type(top_level_media_type: Literal['application', 'audio', 'image', 'text']) -> bool
```

Parameters

[Expand table](#)

Name	Description
top_level_media_type Required*	Literal ['application', 'audio', 'image', 'text']

Returns

[Expand table](#)

Type	Description
<code>bool</code>	

to_dict

Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Name	Description
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[↗](#) Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

uri

The URI of the data represented by this instance, typically in the form of a data URI. Should be in the form: "data:{media_type};base64,{base64_data}".

media_type

The media type of the data.

type

The type of content, which is always "data" for this class.

annotations

Optional annotations associated with the content.

Python
<code>annotations: list[CitationAnnotation] None</code>

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

Default Class

Runtime representation of the default branch in a switch-case group.

The default branch is invoked only when no other case predicates match. In practice it is guaranteed to exist so that routing never produces an empty target.

Constructor

Python
Default()

Parameters

 Expand table

Name	Description
target Required*	

Examples

Python
<pre>class DeadLetterExecutor(Executor): def __init__(self) -> None: super().__init__(id="dead_letter", defer_discovery=True) fallback = Default(target=DeadLetterExecutor()) assert fallback.target.id == "dead_letter"</pre>

Methods

 Expand table

<code>__init__</code>
<code>__new__</code>